

Appendices

Appendix A

AMUSE Fundamentals

You take the blue pill — the story ends, you wake up in your bed and believe whatever you want to believe. You take the red pill — you stay in Wonderland, and I show you how deep the rabbit hole goes.

Morpheus

We present implementation details of the AMUSE framework. AMUSE contains more than 70 highly optimized codes and more than 5000 handy routines to manipulate data. We would love to discuss them all, but this would result in a horribly boring text to write, and read. We limit ourselves to describing the most important and most-used methods, such as file I/O, data manipulation, how to tune the handling of units for diagnostics, and we list the most important parameters used in several of the domain-specific modules. We further list the most important packages currently interfaced with AMUSE.

A.1 Internal parameters, units and types

Most of the community modules in AMUSE have internal parameters that control the performance of the code. Some tune the accuracy or the details of the physics, while others inform the user of the state of the running module. Most are preset to reasonable values, often the values initially set by the original authors of the package, and for which the module has been validated. Under certain circumstances, you may want to change these parameters.

As discussed in Section [2.3.3](#), community codes can be queried for information on parameter settings, but even then some parameters stay hidden deep within the source code. Here we present a brief overview of some of the parameters that appear in this book, and for which a reference may be needed.

A.1.1 Non-tunable parameters

There are a number of non-tunable parameters in the AMUSE community codes. These often inform the user of the state of the underlying code. An example is the `stellar_type` parameter in the stellar evolution codes, as discussed in Section [5.3.12](#).

Table A.1: Stellar classifications in AMUSE.

stellar_type	physical description
0	fully convective low mass main-sequence star
1	main-sequence star
2	Hertzsprung gap
3	first giant branch
4	core helium burning
5	first asymptotic giant branch
6	second asymptotic giant branch
7	main-sequence naked helium star
8	Hertzsprung gap naked helium star
9	giant branch naked helium star
10	helium white dwarf
11	carbon/oxygen white dwarf
12	oxygen/neon white dwarf
13	neutron star
14	black hole
15	massless post-supernova object

The standard AMUSE stellar types are listed in Table A.1. In Section 5.2.2 we demonstrated how to query the stellar evolution type from a running stellar evolution code.

We have adopted the designation of stellar types used in SSE (Hurley *et al.*, 2000). These designations are used to indicate the astrophysical state of a star, even though they are not 100% accurate—these types should be homogeneous across codes, but it is hard to guarantee consistency. While implementing the other stellar evolution codes, we attempted to port the SSE designations, but this did not always lead to a perfect classification. One example is in Figure 5.3, where an asymptotic giant branch star is misidentified as a stellar remnant. Another is in MESA (Paxton *et al.*, 2010, 2011, 2013, 2015, 2018; Gaburov *et al.*, 2018), where the interface fails to report the core-helium burning stage even though this phase is actually properly modeled in the code.

A.1.2 Tunable parameters

Many of the code-specific parameters can be tuned. Examples include the integration-controlling time-step parameter in gravity (see Section 4.3.6), metallicity of the star in stellar evolution (see Section 5.2.3), the adiabatic index (γ) in hydrodynamics (see Section 6.2.3), or the number of photons in radiative transfer (see Chapter 7).

The following tables, Tables A.2 to A.6, present the tunable and non-tunable parameters for the core models in gravity, stellar evolution, hydrodynamics and radiative transfer.

In Table A.4, for example, we present an overview of the tunable parameters in the SPH code Fi. Some of them are quite similar to those used in other codes, such as `timestep`, `verbosity`, and `epsilon_squared`, which have similar meaning across

Table A.2: Overview of the most common tunable and non-tunable gravity-specific parameters.

Tunable parameters		Gravitational dynamics	
parameter name	value	unit	explanation
begin_time	0	time	Star time for integrator.
epsilon_squared	0	length ²	Gravitational softening.
force_sync	0	none	Force synchronization.
gpu_id	-1	none	identity of the GPU.
sync_time	0	time	
timestep_parameter	0.14	none	Time-step parameter.
use_gpu	0	bool	Use the GPU.
zero_step_mode	0	none	Do a zero time step.
Non-tunable parameters			
parameter name	value	unit	explanation
block_steps	0	none	Number of block time steps.
gpu_id	0	none	identity of the GPU.
total_steps	0	none	number of integration steps.

Table A.3: Overview of the most common tunable and non-tunable stellar evolution codes.

Tunable parameters		Stellar evolution	
parameter name	value	unit	explanation
supernova_kick_velocity	600	km/s	Supernova kick velocity.
is_logging_of_evolve_enabled	false	bool	Log stellar evolution.
OB_wind_setting	1	bool	O-star wind parameter.
RGB_wind_setting	1	bool	RGB wind parameter.
AGB_wind_setting	1	bool	AGB wind parameter.
convective_overshoot_parameter	0.12	none	convective overshoot length.
import_model_entropy_accuracy	0.0001	none	entropy solver accuracy.
import_model_entropy_force	20	none	Entropy forcing efficiency.
max_age_stop_condition	$2 \cdot 10^{12}$	yr	Stopping after a time.
maximum_number_of_stars	10	none	number of stars.
metallicity	0.02	none	Stellar metalicity.
min_timestep_stop_condition	10^6	s	Minimal time step.
mixing_length_ratio	2	none	Mixing length parameter.
number_of_ionization_elements	5	none	Numer of ionizing elements.
path_to_data	/data/input		Output directory path.
semi_convection_efficiency	0.04	none	Convection efficieny.
thermohaline_efficiency	1	none	Efficiency of the thermohalinety.
verbosity	True	bool	Maximizal output.

disciplines. Others may sound somewhat obscure, but if you are an SPH expert, they actually make a lot of sense. Now, it is not our intention to turn the reader into an SPH expert, and we will explain a few of the more obscure names. We refer to [Springel \(2010b\)](#) for an excellent overview of the free parameters in SPH. Some physical parameters tune the behavior of the gas, rather than the numerics. Setting the viscosity of a gas (`alphah`) allows one to model various density domains, or gas compositions. Although it is a numerical parameter, its interpretation 234567890-is physical, but regrettably not calculable from first principles.

Let us just provide an example of the sort of discussion one of these parameters leads to. For example, the parameter `alpha` (a common name for a parameter in astrophysics) is an artificial viscosity parameter, with a typical value of 0.5. The parameter `beta`, between 0 and 1, refers to another artificial viscosity, often simply called the “beta parameter”, the value of which is $2 * alpha$, with a typical value of 1. Similarly One might wonder why two artificial viscosity parameters, (`alpha` and `beta`) are needed, but numerically this seems to make sense.

Another example is (as you might guess) `gamma`. This parameter has a completely different meaning from `alpha` and `beta`, and its effect on the computational results is profound. The parameter `gamma` is the adiabatic index for the equation of state of the gas,

$$P = K\rho^\gamma. \quad (\text{A.1})$$

Here K is a proportionality constant and γ typically has a value of $5/3$. For historical reasons, Poisson’s equation for a gas of adiabatic index γ is often written in dimensionless form as the *Lane-Emden equation* ([Lane, 1870](#); [Emden, 1907](#)).

$$\frac{1}{\xi^2} \frac{d}{d\xi} \left(\xi^2 \frac{d\theta}{d\xi} \right) + \theta^n = 0, \quad (\text{A.2})$$

where ξ is dimensionless radius, $\theta = \phi/\phi_0$ is the dimensionless potential, and the polytropic index $n = 1/(\gamma - 1)$. Solutions to the Lane-Emden equation result in finite-radius stars for $0 < n < 5$, while stars with $n \geq 5$ have infinite radii and stars with $n > 5$ have infinite mass. The Eddington model for radiative stars, such as the Sun, corresponds to $n = 3$. The adopted value for n in Fi in AMUSE is $n = 3/2$, or $\gamma = 5/2$, which ensures energy conservation in systems without shocks or dissipative processes. We note that hydro codes, such as `Fi`, contain many such parameters, each of which might be worth an extensive discussion.

Apart from the adiabatic indices, the most important free parameters in SPH include `n_smooth`, the number of neighbors over which the results are smoothed, `sph_h_const`, specifying a fixed smoothing length, if desired, and `sph_artificial_viscosity_eps`, and `sph_viscosity`, controlling the viscosity of the simulated gas. The first, `n_smooth`, is a tunable parameter, which is set to a relatively small number (64 by default). For modern high-resolution simulations, one might wish to set this to a higher value.

Some hydro codes have, in addition to the already mentioned long list of free parameters, specific time-step parameters to control drifts in velocity and acceleration. These parameters are named `free_timestep_crit_constant_a`, `..._aexp`, and similarly for the velocity with a `..._v` instead of `..._a`, for the velocity and acceleration.

Table A.4: Overview of the most common tunable and non-tunable hydrodynamics codes. We omit explaining the details of the parameter, as most of them are rather detailed, require considerable explanation and are probably hardly ever changed. The best way to learn what these parameters do is to look in the source code, and try them out. Just run a simple example script, such as `amuse.examples.stellar_minimal`, change one of the parameters and see what it does in the output. Some of the more obscure names include `beta`, `gamma`, `courant`, `feedback`, `heat_par1`, `heat_par2`.

Tunable parameters	Hydrodynamics	
parameter name	value	unit
<code>acc_timestep_crit_constant</code>	0.25	none
<code>artificial_viscosity_alpha</code>	0.5	none
<code>beta</code>	1	none
<code>code_length_unit</code>	1	au
<code>code_mass_unit</code>	1	$M_{\odot}$
<code>cool_par</code>	1	none
<code>courant</code>	0.3	none
<code>epsilon_squared</code>	0	m^2
<code>feedback</code>	fuv	none
<code>fi_data_directory</code>	data/input/	
<code><code>_cell_opening_constant</code>	0.01	none
<code>gamma</code>	5/3	none
<code>gas_epsilon</code>	0.005	au
<code>max_density</code>	100	$M_{\odot}/\text{au}^3$
<code>n_smooth</code>	64	none
<code>n_smooth_tol</code>	0.1	none
<code>nn_tol</code>	0.1	none
<code>opening_angle</code>	0.5	none
<code>optical_depth</code>	0	none
<code>periodic_box_size</code>	1	au
<code>sph_artificial_viscosity_eps</code>	0.01	none
<code>sph_h_const</code>	0.2	au
<code>sph_viscosity</code>	sph	none
<code>timestep</code>	58	day
<code>verbosity</code>	0	none

Table A.5: Overview of the Boolean parameters in SPH codes.

Boolean value	Boolean parameter name
True	acc_timestep_flag, conservative_sph_flag, eps_is_h_flag, gadget_cell_opening_flag, integrate_entropy_flag, sph_dens_init_flag, use_hydro_flag, self_gravity_flag.
False	adaptive_smoothing_flag, balsara_flag, direct_sum_flag, enforce_min_sph_grav_softening_flag, fixed_halo_flag, freeform_timestep_flag, freeform_timestep_flag, isothermal_flag, periodic_boundaries_flag, quadrupole_moments_flag, radiation_flag, smooth_input_flag, square_root_timestep_flag, star_formation_flag.

Table A.6: Overview of the most common tunable and non-tunable radiative transfer-specific parameters.

Tunable parameters		Radiative transfer	
parameter name	value	unit	explanation
boundary_condition	0	none	See below.
box_size	13	kpc	box size, if bound.
default_spectral_type	-1	none	radiation spectrum.
helium_case_A_flag	True	bool	Helium recombination.
hydrogen_case_A_flag	True	bool	Hydrogen recombination.
isothermal_flag	False	bool	Evolve temperature or not.
momentum_kicks_flag	False	bool	use the momentum kick.
number_of_rays	1000	yr ⁻¹	number of rays.
spectral_file	./spectra/thermal30k.cdf		

Some of the flags may sound unfamiliar, such as **balsara_flag**. This parameter in the SPH code Fi refers to the activation of the Balsara switch (or Balsara limiter), a numerical technique used to reduce the effect of artificial viscosity in regions of effective shear, while still allowing artificial viscosity to be effective. Enabling the Balsara flag is important in simulations of differentially rotating disks, galaxies, or accretion flows, where shear is important and artificial viscosity otherwise dampens real structures.

In AMUSE the Balsara limiter is turned off by default (see Table A.5), but it is one of these parameters that make the dissipation and viscosity in a disk simulation more realistic. We emphasize that default values are what they are for a reason, and changing them is risky, but we recognize that not changing them in certain circumstances may be risky too.

A.1.2.1 Tunable radiative transfer parameters

A parameter not mentioned in Table A.6 is `ionization_temperature_solver`, which is really a flag for various options in solvers. The default value of 2 here indicates the backward differentiation formula (or bdf) method for solving the equations of motion (Gear, 1967). It is an implicit time integration scheme, which solves the time-dependent radiative transfer equations on smoothed particle hydrodynamics (SPH) data, the bdf solver is used to advance the solution of the radiative transfer equations in time, particularly when dealing with stiff equations that can arise in radiative processes. An alternative to the bdf solver would be to set `ionization_temperature_solver` to 1, in which case an explicit Euler solver is used.

The bdf solver allows for larger time steps and improved numerical stability compared to explicit Euler methods, especially when the radiative transfer equations involve rapid changes or processes with widely varying timescales. This makes it suitable for accurately and efficiently modeling the evolution of ionization and temperature in simulations where radiative transfer is coupled to hydrodynamics.

Another common parameter in radiative transfer is `global_helium_mass_fraction`. By default this fraction is set to 0, because in astrophysics we often deal with Hydrogen, rather than Helium-rich interstellar gas. It is one of these parameters that are important to set properly depending on your specific application.

The `isothermal_flag` controls whether to evolve temperature (0) or keep the gas isothermal (1). The `boundary_condition` flag can either be reflective (use a value of -1), vacuum (0), periodic (-1). In a more modern code, one would probably opt for strings, rather than integers for specifying such options.

A.1.3 Input/output file types

In Section 2.2.4 we discussed how to write and read particle sets from permanent storage using `write_set_to_file` and `read_set_from_file`. Normally we adopt hdf5 binary formatted files for input and output, but there are circumstances in which another format might be desirable. Table A.7 presents an overview of the file types recognized in AMUSE. There are restrictions to some file formats: ASCII files, for example, will not be able to represent pointers other than by showing the address which is not very useful when reading the data from permanent storage. Of course, we could add other file formats, but we think that with this repertoire covers some of the most common ones. The default AMUSE file format is hdf5, which is widely accepted within the astronomical community.

Each of these storage format settings has its own derived class and associated Python file. The module `amuse.io.starlab` specifies the starlab data format, whereas the module `amuse.io.gadget` does the same for gadget. If you want to make your own derived data format, the best way to start is to copy one of the existing derived class files and edit it to your needs, filling in the specific requirements of your module.

Table A.7: Recognized read/write file formats. For a full appreciation of this routine we refer to the source codes in `amuse.io`, in particular in `base` for the base class of the storage module, and `store` for the specifics.

keyword	brief description
<code>csv</code>	comma-separated values file
<code>dyn</code>	Starlab binary structured file
<code>starlab</code>	extensive Starlab data format
<code>gadget</code>	Gadget binary data file
<code>hdf5</code>	HDF5 file
<code>nemo</code>	NEMO binary structured file
<code>tsf</code>	NEMO binary structured file
<code>txt</code>	text file containing a table of values separated by a space
<code>vtu</code>	text file containing a table of values separated by a predefined character
<code>vtu</code>	text file of unstructured XML VTK-formatted data
<code>amuse</code>	default selection (uses HDF5)

A.2 Initial Conditions

A.2.1 Generalized initial condition generating routines

In Chapter 3 we discussed a few examples of initial conditions. Of course, any output from any high-fidelity simulation can be considered as input for another simulation. Here we present a list of the most common initial conditions generating functions and routines (as of the publication of this book). Some of these create new data, others append or manipulate data. Many of these generator functions are part of the standard AMUSE sandbox, but a large number of additional routines can be found on GitHub at <https://github.com/amusecode/initial-conditions-generator>.

These routines are used as follows (see also Figure A.1):

```
python make_plummer_sphere.py --nstars 16 --radius 0.01 -Q 0.3 -F Pl16.amuse
python name_stars.py -f Pl16.amuse --nstars 12 --name system -F PlN.amuse
python add_planets_oligarch.py -f PlN.amuse --fplanets 1 --name system -F PlN0.amuse
python add_debris_disk.py -f PlN0.amuse --name system --ndisk 300 -F PlN0A.amuse
python rotate.py -f PlN0A.amuse -F Pl_N16N0_A1000R.amuse --type star
```

This daunting series of calls results in a Plummer distribution of 16 stars, 12 of which have a Oligarchic planetary system, totaling 44 planets, including a debris disk of 3816 test particles each in random orientations in space. The results are stored in a series of intermediate files `Pl16.amuse`, `PlN.amuse`, `PlN0.amuse`, `PlN0A.amuse`, and eventually the final file `Pl_N16N0_A1000R.amuse` containing the initial realization of the simulation. Figure A.1 shows this cluster.

The shell script `make_cluster_with_planets.sh` generates a cluster of stars of which some stars have planets and a disk of planetesimals. The resulting data file (`Pl_N16N0_A1000R.amuse`) can be plotted using the script `amuse.examples.plot_plummer_stars_and_planets`, resulting in Figure A.1.

Table A.8: Overview of the routines for generating and manipulating initial conditions.

Particle-set generators	
Stars	make_single_star.py, make_binary_stars.py, make_stars_in_mass_function.py, make_cluster_of_stars.py, make_cube_of_stars.py, make_plummer_sphere.py, make_gas_plummer.py
Planets	make_solar_system_planets.py, make_exoplanetary_system.py, make_nice_like_system.py, make_planets_oligarch.py, new_solar_system_with_sun_and_planets.py.
complexes	make_hierarchical_cluster_with_planets.py.
Append to particle-set	
Planets	add_asteroids_to_solar_system.py, add_comets.py, add_debris_disk.py, add_moons_to_solar_system_planets.py, add_planets_oligarch.py, add_proto_oort_cloud.py, add_scattered_asteroids.py.
Stars	add_binary_companion.py, add_star_in_orbit.py.
complexes	add_perturbation_to_model.py.
Manipulators	
Manipulators	move_all_particles.py, move_bodies_to_stellar_position.py, rotate.py.
Diagnostics	determine_orbital_elements.py, orbital_elements_to_cartesian.py.
Converters	conv_amuse_to_brutus.py, name_stars.py, write_set_as_ascii.py.
plotting	plot_bodies.py, print_snapshot.py.

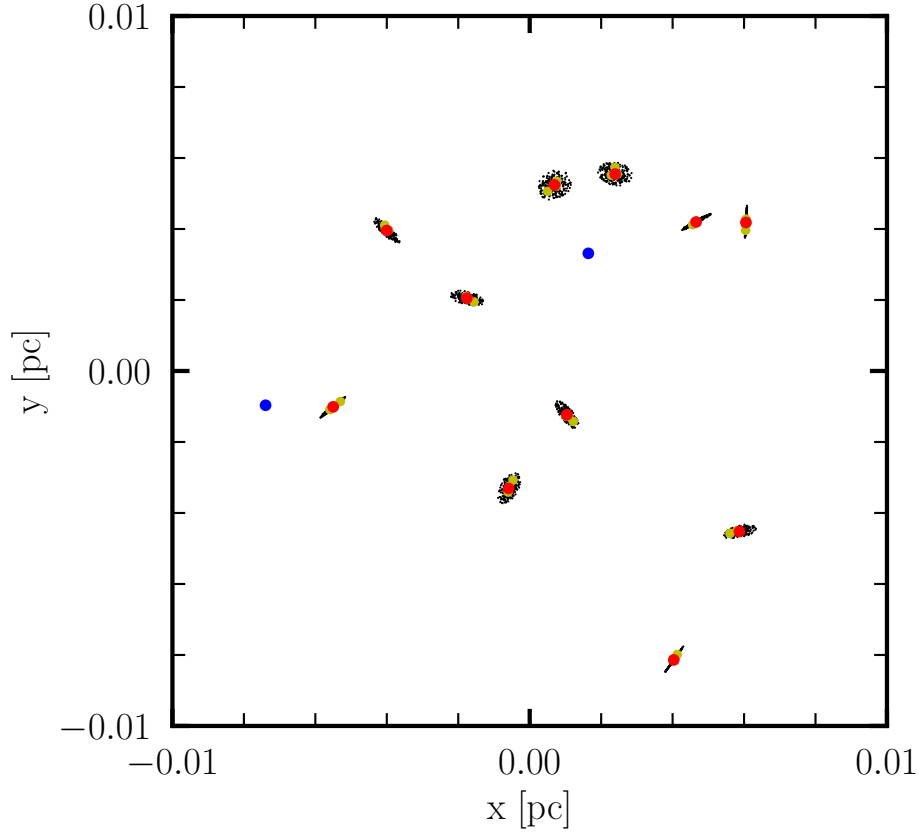


Figure A.1: Representation of a simple 16 star cluster with planetary systems and disks. Single stars are blue bullets, the host star for a planetary system is red. Planets are yellow (hardly visible), but they can be perceived since the stars are surrounded by tiny black dots (the debris). The script to generate these initial conditions is the shell-script `make_cluster_with_planets.sh` (which we copied below), and the code to plot the results is `amuse.examples.plot_plummer_stars_and_planets`.

A.2.1.1 Uniform gas distributions for SPH

Some codes for generating initial conditions are quite versatile, and can take numerous arguments to tune their function. The classes provided by `spherical_model` in `amuse/ext` are examples of such rich functionality. The Python class `UniformSphericalDistribution` in `amuse.ext.spherical_model` forms the basis for a number of associated routines, including `new_uniform_spherical_particle_distribution`, `new_spherical_particle_distribution`, and `new_plummer_spatial_distribution`. The underlying class generates a uniform spherical grid of particles, which can then be used for more useful initial distribution functions. Some of the most important parameters used in the class are listed in Table A.9.

To generate a spherical quasi-random distribution of $N = 10000$ particles with radius $R = 1 \mid \text{units.km}$ and mass $M = 1 \mid \text{units.kg}$, we write

```
from amuse.ext.spherical_model import new_uniform_spherical_particle_distribution
particles = new_uniform_spherical_particle_distribution(N, R, M, type="sobol")
```

The class has two more options: `offset` and `target_rms`. Option `offset` is only used for the regular grids ("cubic", "bcc", "fcc") and should be specified using a tuple of

Table A.9: Additional options used in the class `UniformSphericalDistribution` to tune the generation of initial gas-density distributions. For the sobol method, low-discrepancy indicates that the points are on-average close-to equally distributed.

keyword	brief description
<code>cubic</code>	cubic grid with particles at each corner
<code>bcc</code>	body-centered cubic, with additional particles at the center of each cube
<code>fcc</code>	face-centered cubic, but with additional particles on the face of each cube
<code>random</code>	particles distributed randomly using <code>numpy.random.uniform</code>
<code>glass</code>	as for <code>random</code> , but stabilized using hydro pressure without gravity
<code>sobol</code>	3D sobol sequence (low discrepancy, quasi-random) (see Sobol, 1967)

three numbers in the range $[0, 1)$, which define the offset between the origin of the grid and the corner of the unit cell, normalized to the unit cell size. Option `target_rms` is only used for `glass` as the rms density criterion for convergence (see Section 9.3.2.1).

A.2.2 Writing composite datafiles

Simulations such as those presented in Section 9.4.4 and Chapter 10 often require writing composite files which in Section 9.4.4 include both gravitating and gaseous particles. There is no best or optimal way to write composite data files, but we recommend keeping the format as simple as possible. In the cases just referenced we accomplish this by creating a common particle set containing all particles, and writing this to file. This particle set contains a variety of attributes, depending on the types of particles resulting from several independently operating codes, which can later be disentangled to recover the individual components.

Such a common particle set is generated by the `ParticlesSuperset` function:

```
moving_bodies = ParticlesSuperset([gravity.code.particles, hydro.gas_particles])
```

It can later be saved by

```
write_set_to_file(moving_bodies, filename)
```

or subsequently read from a file:

```
moving_bodies = read_set_from_file(filename)
```

Here, `filename` is the name of the input or output file. In this example we write the data in the default `amuse` format (which uses `hdf5`), but several of the alternatives listed in Table A.7 could be used.

Another way to write composite data files is by simply storing the various particles sets successively, subsequently recovering them in the same order. In this example, we

would create two files (using the default, we omitted specifying the data format):

```
write_set_to_file(gravity_particles, gravity_filename)
write_set_to_file(hydro_particles, hydro_filename)
```

and in reverse read both files:

```
gravity_particles = read_set_from_file(gravity_filename)
hydro_particles = read_set_write_from_file(hydro_filename)
```

A.3 Notes on specific codes

A.3.1 Test particles in N -body codes

Some codes can handle test masses, objects with zero mass. They can be quite handy for following the gravitational potential. However, if you have to pay the N^2 -price for integrating them, they do not make a lot of sense. Some of the N -body codes in AMUSE are therefore equipped to treat zero-mass objects as test masses. This feature is enabled in *Huayno* and *ph4*, and maybe more codes will be enabled with this handy feature as this book goes to print. To enable test-mass integration, simply set the mass of a particular particle to 0 (or to some negative value).

```
particle.mass = 0 | nbody_system.mass
```

Once set to zero, other nonzero-mass particles accelerate the test mass, but the test mass does not affect any other particles.

A.3.2 Post-Newtonian integration with relativistic cross terms

Between 1907 and 1915 Einstein developed general relativity (see [Weinstein, 2012](#); [Poisson & Will, 2014](#); for an interesting read on the history of this development), and viewed gravity as the result of the curvature of space-time ([Einstein & Grossmann, 1914](#); [Einstein, 1915b,a](#); [Hilbert, 1915](#)). The Einstein field equations describe the gravitational interaction between particles, but these equations are nonlinear and notoriously hard to solve. [Schwarzschild \(1916\)](#) found the first nontrivial solution to the Einstein field equations. The Schwarzschild metric describes a point-like particle. A rotating black hole was first described analytically as a solution to the field equations by [Kerr \(1963\)](#). Today, there are standard software implementations to solve general relativistic dynamical systems ([Mewes et al., 2018](#)), even including magnetic fields ([Mewes et al., 2020](#)). However, simulating multiple black holes in a relativistic context is very expensive in terms of computer time.

The N -body code *Hermite-GRX* includes relativistic effects in the post-Newtonian approximation. It was introduced in [Portegies Zwart et al. \(2022\)](#). Apart from the usual parameters of any direct N -body code, *Hermite-GRX* has four extra settings, listed in Table [A.10](#).

The parameters `integrator` and `perturbation` control the integration method and the perturbation. Integrators include various forms of symmetrization and regular-

Table A.10: Overview of the most common tunable and non-tunable parameters for general relativity calculations in the gravitational N -body problem.

Tunable parameters			Gravitational dynamics	
parameter name	value	unit	explanation	
<code>integrator</code>	Hermite		Integrtror type (see Table A.11).	
<code>light_speed</code>	1	length/time	Speed of light.	
<code>num_threads</code>	1		Nuner of threads on the processing core.	
<code>perturbation</code>	None		Perturbation mode (see Table A.11).	

ization. Both help in speeding up the code and reducing energy errors. The perturbation is mainly used to include various post-Newtonian effects, the 1st post-Newtonian and 2.5 post-Newtonian orders of the Einstein Infeld Hoffman equation, and the 1st post-Newtonian equations of motions that include the relativistic cross terms (useful for multiple black holes).

Table A.11: Integrators available in `Hermite-GRX`. These integrators can be selected by providing the appropriate term in the `mode` keyword as an argument for `Hermite-GRX`.

parameter rame	keyword
<code>integrator</code>	Hermite (default) SymmetrizedHermite RegularizedHermite SymmetrizedRegularizedHermite.
<code>perturbation</code>	None (default), 1PN_Pairwise, 1PN_EIH 2.5PN_EIH.

In addition, `Hermite-GRX` has two additional particle types, `large_particles` and `small_particles`. The former is used for particles that are relativistically affected, whereas the others remain unaffected by general relativity. Normally, assigning new particles to the general particle set (`particles`) will cause all objects be affected by general relativistic effects. But for performance considerations, sometimes one might decide to assign certain minor bodies to the `small_particles` set, excluding them from relativistic effects.

A.3.3 The Huayno multi-component symplectic N -body code

Most codes use specific parameters to control their internal solvers. An example of a code with a quite extensive parameter set is `Huayno` (Pelupessy *et al.*, 2012), which uses a recursive Hamiltonian splitting strategy (much like `bridge`, see § Chapter 9) to generate multiple-timestep integrators that conserve momentum to machine precision. A wide variety of different integrators is available within `Huayno`. They are listed in Table A.12, but for more details we refer to Pelupessy *et al.* (2012). The code was developed within the AMUSE environment, and can be compiled with GPU support using `OpenCL` (for which the mode "`OpenCL`" should be specified in the run-time environment). Another gravity code with many options is `Rebound` (see Appendix Appendix A.8).

`Huayno` has two types of shared but variable time step codes: with and without a treatment of collisions. Both types come in all even orders, from 2nd to 10th. Such an

Table A.12: Integrators available in `Huayno`. These integrators can be selected by providing the appropriate term in the `mode` keyword as an argument for `Huayno`.

brief description	keyword
Shared time step	SHARED2, ..., SHARED10,
Collision-enabled	SHARED2_COLLISIONS, ..., SHARED10_COLLISION
Kepler solvers	KEPLER, CC_KEPLER, CCC_KEPLER, OK
Connected components	CC, CCC_BS, CC_BSA, CCC_BSA
Bulirsch-Stoer	SHAREDBS, BS_CC, BS_CC_KEPLER
Kick-drift-kick	PASS_KDK, HOLD_KDK, BRIDGE_KDK
Drift-kick-drift	PASS_DKD, HOLD_DKD, PPASS_DKD, BRIDGE_DKD

integrator is initialized as follows

```
gravity = Huayno(mode="SHARED10_COLLISION")
```

or equivalently by

```
gravity = Huayno()
gravity.set_integrator("SHARED10_COLLISION")
```

The Kepler solvers evolve the two-body problem, with a variety of alternative approaches. The OK solver (see [Pelupessy et al., 2012](#)), for example, adopts the optimal-kick Hamiltonian splitting method to evaluate every interaction in the system at the optimal-split time step, thereby using the least number of force evaluations to evolve the system.

The designation **CC** in Table A.12 stands for connected components. In the **CC** method we split the Hamiltonian to find isolated subsystems which are integrated separately for better accuracy compared to the rest system (see also § Section 10.1.2). The **CC** method is applicable to Kepler solvers, but also to the Bulirsch-Stoer integrator ([Bulirsch & Stoer, 1964](#)). This latter integrator can adopt shared time steps and run with connected components or with a Kepler solver. Each of these leads to better accuracy, particularly for hierarchical systems with complex configurations.

Finally, `Huayno` supports various types of kick-drift-kick and drift-kick-drift schemes, much as in `bridge` (see § Section 9.1).

A.3.4 Symple: The simplest possible symplectic N -body integrator

An example of a code that was written specifically for AMUSE is the direct symplectic N -body code **Symple**. Since there is no documentation on this code, we use it here as an example of a code that was written as an AMUSE module. **Symple** is simple, and symplectic, hence its name. The code can operate as a symplectic N -body integration module of orders 1, 2, 3, 4, 5, 6, 8, and 10, with fixed, shared time steps. Alternatively, setting the time step parameter $\eta > 0$ enables variable and shared time steps, as in, say **Hermite** (but the integrators then lose their symplectic quality). The default is

$\eta = 0.05$.

Initially, we developed **Symple** to be able to effectively integrate the orbit of the triple θ Muscae (see Section 8.4.5). As we saw, this triple comprises three Wolf-Rayet stars, each with a hefty stellar wind. The combination of tight orbits, massive stars, and copious stellar winds makes this system extremely challenging to integrate. It turns out that the linear coupling strategy adopted in Section 8.4.5 is slow and prone to numerical errors. Using a bridge for this system makes the integration more robust, but at an enormous cost in computing time. This made us decide to write **Symple**, a simple symplectic N -body integrator that included a mass-loss term for each star.

Although symplectic integrators do conserve energy on the long run much better than the usually adopted explicit Hermite-type integrators, they can still lead to precession in an elliptic 2-body orbit. Orbital phase and the argument of pericenter are not conserved quantities and therefore are very hard to preserve in a numerical simulation. Although generally this hardly has only minor consequences for the integration, in some cases, such as when integrating von Zeipel-Lidov-Kozai (Lidov, 1962; Kozai, 1962) affected triples, it is of crucial importance.

Even in symplectic integrators, non-commensurate steps lead to long-period modulations in the energy error, without a long-term drift. The amplitude of the modulation can be reduced by making the step an integral fraction of the period, and reduced more by making it an integral fraction of the actual (numerical) period.

Symple is not as simple as the name suggests: it facilitates an implicit Euler (Euler, 1768), second-order predictor-corrector, and higher-order symplectic integrators. The choice of integrators is made in your script through the code-related parameter. In Table A.13 we present the various integration possibilities. But let us first discuss the advantages and disadvantages of each of the integrators. It is important to bear in mind that a simple symplectic integrator, like **Symple**, works best if the time step is a power of 2 times the orbital period. Not obeying this requirement leads to a rather large drift in orbital phase. For higher order integrators (eighth- and tenth-order schemes) this time step requirement is not necessary.

The first order scheme is really an implicit Euler integrator. It has no appreciable long-term energy drift. Instead it will exhibit large orbital variations. For longer time steps the orbit will modulate with small amplitude. The second-order predictor-corrector scheme in **Symple**, however, is probably your best choice for most problems. Long-term orbital integration hardly shows any net drift of energy, exhibits long-term periodic modulation, and the integration is not appreciably affected by commensurate/non-commensurate steps. We recommend making the step commensurate with the numerical period, which reduces the modulation amplitude.

The order of integrator in **Symple** to 10th order, see Table A.13. The first column gives the order of the integrator, followed by the number of evaluations per step, which quickly grows beyond 6th order. The last column gives the number of actual evaluations of the integrator per orbital period of a standard binary system (in N -body units $m_1 = m_2 = 0.5$, $a = 0.57$, $e = 0.75$, resulting in an orbital period of $P = 2.71$.) for constant error in the energy. Considering these values, it may be hard to see why anybody would ever use a 6th-order symplectic integrator for solving a regular N -body problem, but for multi-object integrations these numbers will be different and there

Table A.13: Symple integration order and number of evaluation steps.

Integrator/order	function evals/step	evals/period (const. dE)
1	1	—
2	1	32k
3	3	3k
4	3	3k
5	7	1792
6	7	1792
8	15	3840
10	33	4224

may be a sweet spot in parameter space for optimal use of a 10th-order symplectic integrator.

A.3.4.1 Radiative transfer in SPH codes with SPHray

SPHray ([Altay et al., 2008](#)) is a smoothed particle hydrodynamics (SPH) ray tracer designed to solve the 3D, time-dependent, radiative transfer equation. The method relies on a Monte Carlo ray-tracing scheme that does not interpolate the SPH particles on to a grid but instead integrates directly through the SPH kernels. An Axis-Aligned Bounding Box (AABB) ([Zhao et al., 2021](#)) test taken from computer graphics applications allows for the acceleration of the ray-tracing component. The AABB is defined as a rectangular box (in 3D) with edges parallel to the coordinate axes.

SPHray is capable of handling multiple ionizing sources and complex geometries, making it suitable for studying complex astronomical structures such as giant molecular clouds. Its treatments for time-dependent photoionization, recombination, heating processes, and temperatures makes is excellently suited for coupling with other applications to study time-dependent processes.

The atomic rates are stored in a number of text files for various chemical species, including Hydrogen, Helium and Deuterium in various degrees of ionization. SPHray also supports a wide range of ionization spectra, which can be incorporated using

```
radiative = Sphray(convert=converter, mode="openmp")
radiative.set_spectra_file("spectra/thermal1e5.cdf")
```

alternative spectra include `spectra/thermal6e4.cdf`, `spectra/power2.0.cdf`, and `spectra/power4.0.cdf`.

A.3.5 External galactic potential models

In many cases we want to simulate a self-gravitating system in some background Galactic potential. In order to accommodate such studies, we have incorporated several potential models for the Milky Way Galaxy and other point-symmetric potentials.

The `galpy` (Bovy, 2015, 2014) package includes a bulge modeled as a power-law density profile with an exponential cut-off, a Miyamoto & Nagai (1975) disk, and an NFW (Navarro *et al.*, 1996) dark-matter halo. The parameters of each of the individual components are based on fits to observational data, with the additional overall constraint that the Sun’s distance to the Galactic center is $R = 8.0$ kpc and the solar circular velocity is $v = 220$ km/s Bovy (2014, 2015). We already presented an example of how to initialize such a model in Section 9.4.1.1.

The asymmetric disk of Miyamoto & Nagai (1975) can also be run separately, in which case a different bulge and halo model can be added. For example, it can be complemented using the spherically symmetric Plummer (1911) profile. These profiles can be found in `amuse.ext.galactic_potentials`. The Galactic disk model is initialized using the following lines:

```
disk_mass = 1.0e+10 | units.MSun
disk_size = 10 | units.kpc
disk_height = 100 | units.parsec
from amuse.ext.galactic_potentials import MiyamotoNagai_profile
galactic_disk_model = MiyamotoNagai_profile(disk_mass, disk_size, disk_height)
```

A point-symmetric Plummer potential model can be initiated in a similar fashion using

```
total_mass = 1.0e+6 | units.MSun characteristic_size = 3 | units.parsec
from amuse.ext.galactic_potentials import Plummer_profile
globular_cluster_model = Plummer_profile(total_mass, characteristic_size)
```

In Table A.14 we present a brief overview of the semi-analytic potentials that are directly available within AMUSE.

Table A.14: Short description of the in AMUSE available potential models.

model	Brief desription
NFW_profile	Navarro, Frenk, & White (Navarro <i>et al.</i> , 1996) dark-matter profile. Free parameters include the scale radius and central density.
MiyamotoNagai_profile	semi-analytic disk model (Miyamoto & Nagai, 1975). Free parameters include the total mass and the disk scale sizeand height.
Plummer_profile	Plummer density profile (Plummer, 1911). Free parameters include the total mass and the scale radius.
PowerLawCutoff_profile	Spherically symmetric power-law density profile with exponential cut-off. Free parameters include the central density, power-law scaling radius, the power-law index and the cut-off radius.
MWpotentialBovy2015	Jo Bovy’s <code>galpy</code> density profile model (Bovy, 2015).
Galaxia	Galaxy potential with bulge, bar, disk, spiral arms and halo (Sharma <i>et al.</i> , 2011).

The most versatile and complete semi-analytic potential model for the Milky Way

Galaxy is `Galaxia` (Allen & Santillan, 1991; Cox & Gómez, 2002). This parameterized galaxy-evolution model includes multiple potentials to describe the Milky Way or another galaxy. To make a representation of the current Milky Way a number of parameters are set by default. They are listed in Table A.15.

An alternative would be to use an external package for either generating complex initial conditions based on some pre-defined distribution function, or use a specific background potential. The AGAMA package, discussed in Appendix B.2.2 introduced in Vasiliev (2019), will prove to be quite useful for such studies. It would require the separate compilation of AGAMA, though, which is explained in Appendix B.2.2.

You can call `Galaxia` and set the mass of the disk to $1.0 \times 10^{11} M_{\odot}$ as follows.

```
from amuse.community.galaxia.interface import BarAndSpirals3D
galaxy = BarAndSpirals3D()
galaxy.parameters.mass_disk = 1.0e+11 | units.MSun
```

All the parameters listed in Table A.15 can be set separately to construct original background-potential models.

A.4 Higher-order bridges

Standard bridge is second order, and here we derive the fundamental method. As a result the numerical error (in particular in energy) scales with the square of the timestep dt^2 . Sometimes, higher order integration methods are desirable. For this reason we present some higher-order bridge schemes.

A.4.1 Fundamentals of high-order bridges

In some cases, the Bridge scheme may encounter numerical difficulties because the method is only second-order accurate in the time step dt . This can become a limiting factor when coupling two higher-order methods (for example, sixth- and fourth-order Hermite schemes), or when coupling a fourth-order scheme with a background potential expressed as an analytic function or look-up table. For these cases, a higher-order bridging method is desirable, especially if the symplectic quality can be preserved. Here, we show how Bridge can be extended to an arbitrarily high (even) order. There exist symplectic integrators of odd order, for example the implicit Euler, but they are not very suitable for constructing bridges (Jiao *et al.*, 2007).

We can derive higher order bridges formally as follows. This starts by assuming a system of particles composed of some number Q of subsystems k . In this case, the total Hamiltonian of the system,

$$H = \sum_{i \in S} \frac{\mathbf{p}_i^2}{2m_i} - \sum_{\substack{i,j \in S \\ i \neq j}} \frac{Gm_i m_j}{r_{ij}}, \quad (\text{A.3})$$

can be split to obtain

$$H = \sum_{k=1}^Q H_k + \sum_{\substack{k,l=1 \\ k \neq l}}^Q H_{kl}^{int}. \quad (\text{A.4})$$

Table A.15: Default parameters for the Milky Way Galaxy in `galaxia` (see `amuse.examples.solar_cluster_in_galaxy_potential` discussed in Section 9.4.1.1). These are many free parameters in this model. Here we have rounded off some of the numbers; in the package the pitch angle is given to 9 decimal places and the scale height of the disk to 12 digits.

parameter name	value	unit
phase	-0.34906	
a_disk	5.3178	kpc
a_halo	12.0	kpc
aaxis_bar	3.12	kpc
amplitude	1300	kms ² / kpc
amplitude2	680	kms ² / kpc
axis_ratio_bar	0.37	
b_bulge	0.3873	kpc
b_disk	0.25	kpc
bar_phase	-0.349	
caxis_bar	1.0×10^{-6}	kpc
fiducial_radius	8.0	kpc
initial_phase	-0.34906	
mass_bar	1.4000×10^{10}	M _⊙
mass_bulge	1.4059×10^{10}	M _⊙
mass_disk	8.5608×10^{10}	M _⊙
mass_halo	10.7068×10^{10}	M _⊙
nbt	0.0	
omega_bar	40	km / (s kpc)
omega_spiral	20	kms / kpc
omega_spiral2	20	km / (s kpc)
omega_system	40	km / (s kpc)
phi21_spiral	-3.49	
r_sigma	2.5	kpc
rsp	3.12	kpc
scale_height	0.18	kpc
sigma_s	100	Myr
spiral_density_amplitude	5.80×10^{14}	M _⊙ /(kpc ³)
spiral_model	0.0	
spiral_phase	-0.349	
t_sim	-34.132	Myr
tan_pitch_angle	0.227	

The terms in Equation (A.4) are given by:

$$\begin{aligned} H_k &= \sum_{i \in k} \frac{\mathbf{p}_i^2}{2m_i} - \sum_{\substack{i,j \in k \\ i \neq j}} \frac{Gm_i m_j}{r_{ij}} \\ H_{kl}^{int} &= - \sum_{\substack{i \in k \\ j \in l}} \frac{Gm_i m_j}{r_{ij}}. \end{aligned} \quad (\text{A.5})$$

Note that this is a slightly different notation than in the original paper by [Fujii *et al.* \(2007\)](#), but the end result is the same.

Based on this splitting, a general, multi-subsystem, second-order time evolution operator can be constructed as follows:

$$\text{Bridge}_2(\tau) = \prod_{k=1}^Q e^{\frac{1}{2}\tau D_{H_k}} \prod_{\substack{k,l=1 \\ k \neq l}}^Q e^{\tau D_{H_{kl}^{int}}} \prod_{k=1}^Q e^{\frac{1}{2}\tau D_{H_k}}. \quad (\text{A.6})$$

As in the classic Bridge, the operators $e^{\tau D_{H_k}}$ independently evolve each of the subsystems k in isolation. The operators $e^{\tau D_{H_{kl}^{int}}}$ represent pure momentum kicks due to interactions between subsystems k and l . In the case of the Hamiltonian in Equation (A.4), the interaction terms do not depend on velocity, so the operators $e^{\tau D_{H_{kl}^{int}}}$ commute. We note, however, that this is not the case for velocity-dependent forces, so special treatment is required in that case. Because each of the operators $e^{\tau D_{H_k}}$ and $e^{\tau D_{H_{kl}^{int}}}$ can, in principle, be associated with different solvers running concurrently, a highly efficient parallel implementation can be achieved for the generalized Bridge scheme.

A higher-order symplectic Bridge scheme can be constructed in a similar fashion. By defining the drift and kick operators

$$\mathcal{D}(\tau) = \prod_{k=1}^Q e^{\tau D_{H_k}}, \quad (\text{A.7})$$

and

$$\mathcal{K}(\tau) = \prod_{\substack{k,l=1 \\ k \neq l}}^Q e^{\tau D_{H_{kl}^{int}}}, \quad (\text{A.8})$$

Appendix A.4.1 can be extended to higher order by a suitable combination of $\mathcal{D}(\tau)$ and $\mathcal{K}(\tau)$ operators ([Hairer *et al.*, 2006](#)). For a fourth-order symmetric composition with four stages, the high-order Bridge takes the form:

$$\begin{aligned} \text{Bridge}_4(\tau) &= \mathcal{D}(u_0\tau) \mathcal{K}(u_0\tau) \mathcal{D}(u_1\tau) \mathcal{K}(v_1\tau) \\ &\quad \mathcal{D}(u_2\tau) \mathcal{K}(v_1\tau) \mathcal{D}(u_1\tau) \mathcal{K}(v_0\tau) \mathcal{D}(u_0\tau), \end{aligned} \quad (\text{A.9})$$

(see [Yoshida, 1990](#); [Hairer *et al.*, 2006](#); for the coefficients u_i and v_i). The sixth-order method becomes

$$\text{Bridge}_6(\tau) = \mathcal{D}[\tfrac{1}{2}w_0\tau] \mathcal{K}[w_0\tau] \mathcal{D}[\tfrac{1}{2}(w_0 + w_1)\tau] \dots \mathcal{D}[\tfrac{1}{2}(w_{s-1} + w_s)\tau] \mathcal{K}[w_s\tau] \mathcal{D}[\tfrac{1}{2}w_s\tau], \quad (\text{A.10})$$

where the coefficients w_i are given by [Sofroniou & Spaletta \(2005\)](#) and [Hairer *et al.* \(2006\)](#). The higher-order bridges are identified by order: Leapfrog, fourth, sixth, etc., and by the number of force evaluations: $m = 5$ or $m = 6$ for the fourth order, $m = 11$ or $m = 13$ for the sixth order. AMUSE includes implementations up to and including 10th order (with $m = 35$), which you will probably never need. A complete list of methods for bridge can be found in [Appendix A.4.2](#) (see [Table A.16](#)).

The above formulation provides a fully symplectic time evolution if the codes being bridged are themselves symplectic. Using this scheme, codes/integrators of different orders can be coupled to construct a formally high-order scheme by matching the order of the bridge used during the coupling. For example, when coupling a sixth order to a fourth-order method, it is appropriate to choose `Bridge4( $\tau$ )` to create a fourth-order compound method. If we chose `Bridge6( $\tau$ )`, the method would still be only fourth-order, because of the fourth-order sub-integrator. In what follows, we refer to the second-order (original) bridge as the classic bridge, and use the term higher-order bridge for the generic case.

The logic of selecting an appropriate bridge order may not always be straightforward. For example, if second-, fourth-, and sixth-order schemes were coupled using the classic bridge [`Bridge2( $\tau$ )`], the result would be a second-order compound method. However, we could improve this by adopting a *hierarchical coupling strategy*: bridge the fourth- and sixth-order codes using `Bridge4( $\tau$ )`, then bridge this compound method with the remaining second-order code using `Bridge2( $\tau$ )`. Although the overall compound solver is still formally second-order, the subsystems evolved with the more precise codes are coupled at a higher order than the rest of the system. This can be advantageous if such a subsystem dominates the overall error or requires higher precision—for example, because of the requirement to resolve close encounters. Finally, we note that, while the methods presented here are very flexible, the proper choice of integrators is highly problem-dependent.

A.4.2 Tuning the higher-order bridge

In [Chapter 9](#) (see [Section 9.3.1](#)) we discussed coupling codes using `bridge`, and in [Appendix A.4](#) we explained how higher-order bridges can be constructed by repeated steps with a bridge operator. Several higher-order bridge methods are incorporated into AMUSE; some of them worked out in `composition_methods`, which can be incorporated in your script via

```
from amuse.ext.composition_methods import {method_name}
```

where `method_name` is the name of the method you want to use. [Table A.16](#) presents a brief overview of the available methods.

An example of the working of a higher-order bridge can be found in the code example `amuse.examples.three_body_bridge_order4m4`, which we discuss in [Section 9.3.1](#). There we declare a 4th order bridge with 4 evaluations per bridge step, using

```
gravity = bridge.Bridge(use_threading=False, method=SPLIT_4TH_S_M4)
```

Table A.16: Bridge methods identified by the order of their integrator. The columns give the order of the integrator, the splitting strategy (S for symmetric composition and SS for symmetric composition of symmetric maps), and the number of evaluations per bridge step (M).

name	order	S	M
LEAPFROG			
SPLIT_4TH_S_M4	4	S	4
SPLIT_4TH_S_M5	4	S	5
SPLIT_4TH_S_M6	4	S	6
SPLIT_6TH_SS_M11	6	SS	11
SPLIT_6TH_SS_M13	6	SS	13
SPLIT_8TH_SS_M21	8	SS	21
SPLIT_10TH_SS_M35	10	SS	35

A.5 Stopping and boundary conditions

Any code has a domain of operation. This limitation can be controlled in terms of stopping conditions (at what point should a code hand back control to the parent script), and boundary conditions (when should a code hand back data that does not belong to its domain).

A.5.1 Hydrodynamical boundary conditions

In many circumstances, smoothed-particle hydrodynamics solvers don't have to worry much about boundary conditions. Simulations of star formation or stellar collisions, for example, simply remove SPH particles that stray too far from the region of interest. In other circumstances, however, boundary conditions are important, and these are implemented and accessible through the AMUSE interface (see Table 6.2). For example, periodic boundary conditions in an SPH code can be declared by setting the parameter `periodic_boundaries_flag` to `True` and giving a dimension to `periodic_box_size`.

On the other hand, grid-based hydrodynamics codes, as discussed in Section 6.2.3, always require some sort of boundary condition, and the number of options is more extensive. For these codes, the boundary conditions are an attribute of the code that can be set at run time. For example, to apply periodic boundary conditions along the Cartesian coordinates you could write:

```
hydro_code.parameters.x_boundary_conditions = ("periodic", "periodic")
hydro_code.parameters.y_boundary_conditions = ("periodic", "periodic")
hydro_code.parameters.z_boundary_conditions = ("periodic", "periodic")
```

Alternatives to periodic boundary conditions are "cont", "inflow", "interface", "outflow", and "reflective", although not all of these may be defined for each of the available hydrodynamics solvers. These terms may sound a bit arcane, but if you perform hydrodynamical simulations they are probably quite familiar.

A.5.2 Stopping conditions

Throughout this book we have stressed the importance of being able to stop a code cleanly when certain conditions are met. These conditions are called stopping conditions. We currently maintain a finite set of such conditions, but if more are needed we encourage you to implement them, because they make the interaction between the modules in AMUSE more flexible and versatile.

Table A.17 lists the most commonly used stopping conditions in the AMUSE community modules. Not all codes support all of the stopping conditions for that particular domain. The main reasons that some codes do not support some or all possible stopping conditions are either that the stopping conditions were never implemented (or tested, and therefore not advertised), or that a particular code does not behave well under externally initiated interrupts. Adding a new stopping condition is not hard for most codes, but it does require modifying the source code, and it tends to adversely affect performance.

A.6 Other handy routines

There are quite a few handy routines in AMUSE. If ever you find yourself in a position to write a new routine because you want to manipulate a specific particle set, chances are that we already wrote such a routine. This book, however, is too short to list all possibilities and refer explicitly to each of the handy routines we wrote and tested. If it exists, it is likely that our routine also turns out to be effectively optimized. On the other hand, there are many ways in which one can manipulate specific data. The best place to look for our own implementation, and hopefully in the near future also including your contributions, is in the actual source code. Particularly handy routines to handle data can be found in the following directories: `couple data`, `datamodel`, `ext`, `ic`, `io`, `plot`, `rfi`, `support`, `test`, and `units`.

An example of such a handy routine is the method to find a subset of particles that belong to a larger particle set:

```
L4_trojans = planetesimals.get_intersecting_subset_in(L4_trojans_rot)
```

here the particle set `planetesimals` is a set that supposedly contains members of another particle set, called `asteroids`. This routine returns the particular particles in the particle set `planetesimals` that are identical to those in the other particle set `asteroids`.

It may seem that this is a rather contrived operation, but in practice we need this more often than one might naively expect. There are a total of 5000 routines in the collective AMUSE directories. Some are more useful than others, but usually there is a good reason why we wrote them. Before you start writing your own data-handling routine we strongly recommend that you first search for an existing routine that does the job.

Table A.17: Stopping condition keywords for gravitational dynamics, stellar evolution, and hydrodynamics. There are no dedicated stopping conditions for radiative transfer codes, simply because up to now they have not been needed. The lack of stopping conditions for radiative-transfer codes does not mean that these are not needed, but rather that these codes have not been used as frequently yet in production simulations.

STOPPING CONDITION KEYWORDS	
Gravitational dynamics	
<code>collision_detection</code>	Detect collision between two particles.
<code>stopping_conditions_timeout</code>	Detect if the pre-determined end-time is reached.
<code>out_of_box_detection</code>	Check if particles have left some pre-defined box size.
<code>number_of_steps_detection</code>	Stop after pre-determined number of integration steps are realized.
<code>escape_detection</code>	check if a particles has escaped.
<code>pair_detection</code>	check if two particles have become mutually bound.
Stellar Evolution	
<code>supernova_detection</code>	Check if a supernova has occurred.
<code>max_age_stop_condition</code>	Check if some maximum age has been reached.
<code>max_iter_stop_condition</code>	Check for maximum number of internal code iteration-steps.
<code>min_timestep_stop_condition</code>	Guarantee the code to stop when the time step becomes too small.
Hydrodynamics	
<code>minimum_density_parameter</code>	Check for a minimum density threshold.
<code>maximum_density_parameter</code>	Check for a maximum density threshold.
<code>minimum_internal_energy_parameter</code>	Check for the minimum energy of a particle.
<code>maximum_internal_energy_parameter</code>	Check for the maximum energy of a particle.
<code>number_of_steps_parameter</code>	parameter to set the maximum number of steps.
<code>out_of_box_parameter</code>	parameter to set the size of the box.
<code>out_of_box_use_center_of_mass_parameter</code>	Assure that the out-of-box detection is done from the center of mass.
<code>timeout_parameter</code>	Set the maximum simulation time.

A.6.1 Particles and particle sets

Particles form a fundamental part of the AMUSE framework. They are a form of containers with attributes, member functions, manipulators, and communicators. They form the heart of the AMUSE data representation.

Table A.18: Overview of the minimum set of attributes for the gravity domain in AMUSE.

attribute name		Gravitational dynamics	
attribute	value	unit	explanation
mass	0.0	kg	particle the mass.
radius	0.0	m	particle radius.
position	[0.0, 0.0, 0.0]	m	position vector of a particle.
velocity	[0.0, 0.0, 0.0]	m/s	velocity vector of a particle.
acceleration	[0.0, 0.0, 0.0]	m/s ²	acceleraration vector.

Table A.19: Overview of the minimum set of attributes for the stellar evolution domain in AMUSE.

attribute name		Stellar evolution	
attribute	value	unit	explanation
mass	0.0	kg	particle mass.
radius	0.0	m	particle radius.
age	0.0	s	particle age.

A.6.2 Particle attributes

Each domain has a standard selection of attributes. Some codes have additional sets. The main attributes are listed in `amuse.datamodel.attributes`.

In Tables A.18 to A.21 we list the most fundamental attributes and their default values for the various particles sets for that specific domain.

A.6.3 Particle queries and functions

Apart from a simple storage container, there are numerous functions associated with particles and particle sets. Here we list the most useful ones. Many of these have already been presented in various forms, but here we give a small overview of the few of the most useful. Note that we have both singular (`Particle()`) and plural `Particles()`. The latter, we call a set. The difference is similar to an integer and

Table A.20: Overview of the minimum set of attributes for the hydrodynamics domain in AMUSE.

attribute name		Hydrodynamics	
attribute	value	unit	explanation
pressure	0.0	Pa	the pressure in a region of space
density	0.0	kg/m ³	the density of molecules or solid matter
temperature	0.0	K	the temperature of the gas
magnetic_field	0.0	T	magnetic field created by gas and stars
velocity_field	0.0	m/s	velocity of the gas
gravitational_potential	0.0	m ² /s ²	gravity forces from stars and gas
viscosity	0.0	kg/m/s	viscosity of the gas cloud

Table A.21: Overview of the minimum set of attributes for the radiative transfer domain in AMUSE.

attribute name			Radiative transfer
attribute	value	unit	explanation
temperature_gas	0.0	K	the temperature of the gas
temperature_dust	0.0	K	the temperature of the dust
temperature_background	0.0	K	the temperature of the background
density	0.0	mol/m ³	molecular density
magnetic_field	0.0	tesla	magnetic field created by gas and stars
velocity_field	0.0	m/s	velocity of the gas

an array of integers. Some functions will work on a single particle, others on a set of particles. Converting a single particle to a set can be realized using `as_set()` (see below).

- `get_timeline_of_attribute_as_vector()` returns a array of a specific attribute from the time-line of a particle set.
- `copy_to_new_particles` copies all a particle attribute data to a new particle.
- `new_channel_to(self, other, attributes=None, target_names=None)` makes a channel between two particle sets.
- `copy_values_of_attributes_to(self, attribute_names, particles)` copies the attribute values of one particle to another particle.
- `add_particles(self, particles)` adds a particle to a particle set.
- `ensure_presence_of(self, x)` ensures that a particle has a specific attribute.
- `remove_particles(self, particles)` removes an existing particle.
- `synchronize_to(self, other_particles)` synchronizes one particle set to another. This routine does not copy data, but makes sure that particles missing in one set will be synchronized to the other, or that particles not present in the one set are removed from the other.
- `as_set(self)` converts a single particle to a particle set.
- `select_array(self, selection_function, attributes=())` returns a subset view on a particle set, containing all particles for which the selection function returns True.
- `difference(self, other)` returns a new subset containing the difference between two particle sets.

```

particles = Particles(3)
particles.x = [1.0, 2.0, 3.0] | units.pc
subset = particles.select(lambda pos : pos < 2.5 | units.pc,
["x"])
inner_stars = particles.difference(subset)
print(inner_stars)

```

- `has_duplicates(self)` returns True when a set contains a particle with the same key more than once. Particles with the same key are interpreted as the same particles. This small example:

```

particles = Particles()
p1 = particles.add_particle(Particle(1))
p2 = particles.add_particle(Particle(2))
particles.has_duplicates()

```

returns False.

- `reversed(self)` returns a subset with the same particles, but with reversed sequential order (the first particle will become last)
- `sorted_by_attributes(self, *attributes)` sorts particles by a specific attribute. The last attribute name in the call is used for the primary sort order, the second-to-last attribute name for the secondary sort order, and so forth.

```

particles = Particles(4)
particles.mass = [2.0, 3.0, 1.0, 4.0] | units.kg
particles.radius = [3.0, 2.0, 1.0, 2.0] | units.m
sorted = particles.sorted_by_attributes('mass', 'radius')
print(sorted.radius)

```

- `random_sample(self, number_of_particles)` randomly selects a subset of `number_of_particles` particles from a parent set.
- `remove_attribute_from_store(self, name)` removes a specific attribute from a particle.

```

particles = Particles(3)
particles.temporary_attribute = 42
particles.remove_attribute_from_store("temporary_attribute")

```

Particles can be organized in supersets using the `ParticlesSuperset` particle set, which is a derived class from the `AbstractParticleSet`. In a superset of particles,

attribute values are not stored by the superset, but kept locally in the subset. The superset provides a view on two or more sets of particles.

Superset objects are not supposed to be created directly, but are always derived from two or more particle sets. An simple example is

```
p1 = Particles(3)
p1.mass = [10.0, 20.0, 30.0] | units.kg
p2 = Particles(3)
p2.mass = [40.0, 50.0, 60.0] | units.kg
p = ParticlesSuperset([p1, p2])
print(len(p))
6
print(p.mass)
[10.0, 20.0, 30.0, 40.0, 50.0, 60.0] kg
p[4].mass = 70 | units.kg
print(p.mass)
[10.0, 20.0, 30.0, 40.0, 70.0, 60.0] kg
p2[1].mass
quantity<70.0 kg>
cp = p.copy()
print(len(cp))
6
print(cp.mass)
[10.0, 20.0, 30.0, 40.0, 70.0, 60.0] kg
```

A.6.4 Grid-based data

Similarly to `Particle` and `Particles` sets AMUSE recognizes grids. The grid class is based on the `AbstractGrid` class. This abstract class has an endless range of member functions for grid manipulation. Best is to look in the module `amuse.datamodel.grids`. Derived classes from the `AbstractGrid` include `UnstructuredGrid`, `StructuredGrid`, `RectilinearGrid`, `RegularGrid`, and `CartesianGrid`. The functions to generate those grids include `new_cartesian_grid()`, `new_regular_grid()`, `new_rectilinear_grid()`, `new_structured_grid()`, and `new_unstructured_grid()`. And then we did not even mention the entire repository of `SubGrid` structures.

Apart from those grid-generating functions, there are numerous grid manipulation routines, query functions, attribute setters, and methods to sample the grid, similarly to the enormous range of functions associated for particles and particle sets. The number of routines is so large that even for this appendix it would go too far to mention them all. The best advice we have is to read the source code in the `amuse.datamodel` module.

A.7 Plotting

Making figures and plotting in general is an art. A good figure has a message, and it is important to have the right tools to communicate it. Here we discuss a few Python tools that we like and have used to make the figures in this book.

A.7.1 Basic plotting in Python

The most basic plotting can be done with `gnuplot` (Williams *et al.*, 2013; Vaught, 1996), which can be downloaded from <http://www.gnuplot.info>. If you have installed `gnuplot` on your computer, you can use the `gnuplot-py` package to run it directly from your script. To install `gnuplot-py`, open a shell and type

```
>% easy_install gnuplot-py
```

An extensive manual for `gnuplot` is available at: http://www.gnuplot.info/docs_5.0/gnuplot.pdf.

Sometimes more control is needed to create a plot exactly in accord with your demanding co-author's wishes. For more elaborate plotting, we have opted for `pyplot`, which is part of the `matplotlib` package (Hunter, 2007). `Matplotlib` is a Python plotting library which can produce publication-quality figures. Many of the AMUSE example scripts use this library to produce graphical output, and we refer you to `amuse.examples` for many examples, from simple to complex. If you would like to take advantage of `matplotlib`, get it from <https://matplotlib.org> and install it as a Python package. For your own work, it is of course also possible to print the required data to a file and use your favorite plotting tool to make the figures.

A.7.2 Plotting with units

Most of the figures in this book are made using `matplotlib`. In most cases, we used the standard `matplotlib` and set the units by hand. However, this is not really necessary in most cases, because we have overloaded the most important functions to allow them to handle arrays with units. The AMUSE enabled `matplotlib` functions can be imported via

```
from amuse.plot import scatter, plot, hist
```

The majority of the `matplotlib` routines can be imported in this way, but we have also created a few new ones that are specific to particular codes used in AMUSE. They are listed below in Appendix A.7.3.

An interesting tool for plotting that supports AMUSE particle sets is `vaex` at <https://vaex.io/> (Breddels, 2017). This package is particularly interesting if the particle sets become very large and hard to handle, because `vaex` can process a billion objects per second on a regular workstation. This enables direct exploration of large datasets, making slices and selections using boolean expressions. `Vaex` is easily installed from the git server using

```
%> pip install git+https://github.com/vaexio/vaex/
```

It is also possible to use the `jupyter` (Kluyver *et al.*, 2016) version of `vaex`, which is called `ipyvolume`. This enables you to construct a portable `html` compliant version of your plotted data which can easily be incorporated in a web interface. This package can be downloaded from the `github` repository.¹

¹See <https://github.com/widgetti/ipyvolume>.

A.7.3 Additional plotting utilities

We have created supporting routines for a variety of plotting applications. Some of these routines have found their way into AMUSE. Particle-based plots are generally rather easy to make, including animations, as we demonstrated in Chapter 4. Plotting results from radiative transfer and hydrodynamical simulations are more complicated, and for these purposes we provide a few additional plotting routines. Most can be found in the `amuse.plot` module.

- The results from SPH simulations are often easiest to interpret by simply plotting the individual particles. For this purpose we provide the function

```
sph_particles_plot()
```

which is a very simple and fast way to make a plot of the hydrodynamical state of a set of SPH particles. It takes a particle set as argument, with several additional arguments to control the energy range, the size of the viewport, and the transparency of the particles. The particles must have `position`, internal energy `e`, and a smoothing length `h_smooth`, otherwise the routine will not know how to calculate the density maps. Figure A.2 shows the 2010 Leiden Observatory Christmas card which we made using this routine.

- For more flexible plotting we can convert a particle set in AMUSE to a `pynbody` data set by calling²

```
convert_particles_to_pynbody_data()
```

Here, in addition to providing a particle set, the arguments `length_unit` and `pynbody_unit` should be set to the desired units, for example `length_unit=units.kpc` and `pynbody_unit="kpc"`.

- Since `pynbody` is a convenient package, we also provide a column density plotting routine for it:

```
pynbody_column_density_plot()
```

There are a number of limitations to these plotting packages, and they are not as well developed as the rest of AMUSE, but they may at least serve as a basis for more complete routines.

- Finally, for plotting equipotential surfaces we provide

```
effective_iso_potential_plot(gravity_code, ...)
```

which has a large number of possible arguments. This routine creates a contour plot of the effective potential of particles in a gravity code. The code must support `get_potential_at_point`, which is used to map the iso-contour. The routine is practical because it is almost completely independent of the underlying

²A concise manual for `pynbody` is available inline at <https://pynbody.github.io/pynbody/>.

code, and as a result, it is possible to make iso-density plots or equipotential-surface plots from any potential, even if the underlying potential is provided by a complicated bridging of multiple codes. It requests the gravity code from which the data are read, and some additional parameters, each of which is listed in the original source code, which can be found in the `amuse.plot` module.

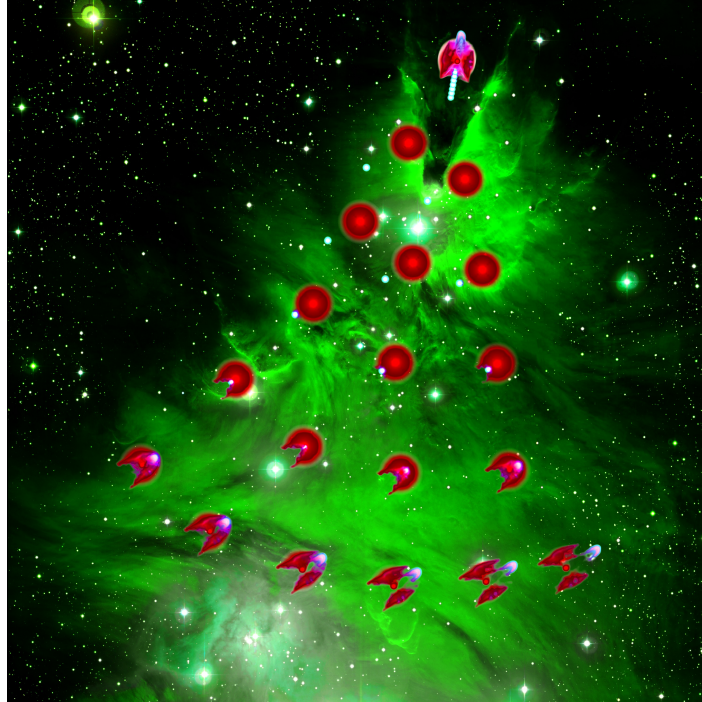


Figure A.2: False color image of the HII region NGC 2264 in the constellation Monoceros (the Unicorn). Overplotted are snapshots of a hydrodynamical simulation (using `Fi` of two stars of masses 0.3 and $3.0 M_{\odot}$ that collide head-on with a relative velocity of 3000 km/s at an age of 310 Myr , when the primary is a giant. The Christmas card made with AMUSE and used by Leiden Observatory in 2010. The figure was made using the script `amuse.examples.christmas_card_2010`; it takes a couple of hours to run on a laptop.

A.7.4 Multiprocessing Codes

We can implement multiprocessing in this example by slightly rewriting the main evolve function in Code example 5.4. In Code example A.1 we adopt a similar approach as in Code example 5.4, but we can enable multiprocessing by introducing the specific stellar evolution code as an argument. This routine takes `code` as an argument and uses it to evolve a $10 M_{\odot}$ star to an age of 1 Myr . We could expand its functionality by introducing more arguments, such as mass, end time, and metallicity, but that would make the code example longer and not necessarily easier to understand.

The driving sequence for the main evolution routine in Code example A.1 is presented in Code example A.2. Here, we use the Python packages `queue` and `threading` for synchronizing queues and enabling threading, respectively, along with the process-based threading interface `multiprocessing`. We recognize that this example uses several fairly sophisticated multiprocessing concepts that the novice reader may find intimidating. If that is the case, you can safely skip to the next subsection.

```

58 def evolve_single_star(code):
59     stars = Particles(mass=10 | units.MSun)
60     stellar = code()
61     stellar.particles.add_particles(stars)
62     channel = stellar.particles.new_channel_to(stars)
63
64     stellar.evolve_model(1 | units.Myr)
65     channel.copy()
66     print(
67         f"Star evolved to time= {stellar.model_time} "
68         f"M= {stars.mass} R= {stars.radius}"
69     )
70     stellar.stop()

```

Code example A.1: Rewritten routine (from Code example 5.4) for evolving a single $10 M_{\odot}$ star to an age of 1 Myr with the stellar evolution code `code` as an argument. Code fragment from `amuse.examples.multiple_stellar_threaded`.

```

33 code_queue = queue.Queue()
34
35
36 def remote_worker_code():
37     code = code_queue.get()
38     evolve_single_star(code)
39     code_queue.task_done()
40
41
42 def evolve_with_different_stellar_model(codes):
43     for code in codes:
44         if code is not None:
45             code_queue.put(code)
46     n_cpu = multiprocessing.cpu_count()
47     for i in range(n_cpu):
48         thread = threading.Thread(target=remote_worker_code)
49         thread.daemon = True
50         thread.start()
51     code_queue.join() # block until all tasks are done

```

Code example A.2: Multiprocess version of the main loop for evolving stars, again from `amuse.examples.multiple_stellar_threaded`.

We construct the task queue as a global variable `code_queue`, which in our case contains the list of codes used in our experiment. The actual work is done in the function `remote_worker_code`, which reads the next task from the queue, calls the evolution function `evolve_single_star`, and notifies the queue when the task has finished. The routine `evolve_with_different_stellar_model` forms the heart of the multiprocessing task. It fills the queue with the various codes, checks how many cores are available (to prevent overloading your system), and starts the threads. It ends by blocking each thread until all items in the queue have finished. The entire routine to launch multiple stellar evolution codes is given in `amuse.examples.stellar.multiple_stellar_threaded`.

Now we can evolve the same ($10 M_{\odot}$) star to the same age (1 Myr), using four different stellar evolution codes in parallel. These codes are listed in an array and passed as an argument to the driving routine:

```
codes = [SeBa, SSE, MESA, EVTwin]
evolve_with_different_stellar_model(codes)
```

Note that simulating the evolution of a large number of stars may run into fundamental limitations of the operating system. Modern operating systems are limited, by their standard settings, to $2^{15} = 32,768$ concurrent jobs (even on large supercomputers), and it often requires root access to change this.

A.8 AMUSE community modules

AMUSE contains a collection of some 50 different community modules, each of which can be used separately, or in conjunction with one or more other codes. Not all combinations make sense, even though technically the codes can be combined. Here we give a brief overview of all current community modules. This list is bound to change with 2892 time³

We recognize three classes of modules, the core selection, envelope, and the rest. Originally we designed AMUSE following a “Noah’s Ark” philosophy (Portegies Zwart, 2018). According to this paradigm, each domain should have two working implementations. In practice, we sometimes deviate from this, but two modules are the minimum necessary to be able to develop a generalized interface.

A.8.1 Modules by domain and functionality

Table A.23 lists the modules in AMUSE by domain and functionality. Note that, in the gravitational domain, we have expanded the code type category to include “dedicated” codes, which simply solve Kepler’s equations, and “limited” codes, which make such rigorous assumptions about the gravitational force that they should be considered potential solvers, rather than N -body codes.

Because the efficiency of a code depends on the algorithm, time-stepping, data transport, initial conditions, available hardware, parameter settings, etc., it is hard to give an absolute ranking, and in addition speed and efficiency are not always the same thing. Generally, but not always, slower codes tend to produce more accurate results, and therefore should not necessarily be avoided; the difference in speed may translate to higher accuracy or a better precision (see Portegies Zwart & Boekholt, 2018; for a discussion of the difference between accuracy and precision).

A.8.2 Modules by name

Finally, we list all current modules alphabetically, for reference. For each code, we (1) provide an indication of its main working domain, (2) point to a script in which the code’s usage is demonstrated, (3) list extra requirements or features regarding software or hardware, and give a short list of (4) equivalent, and (5) comparable codes. Equivalent codes use the same or a similar algorithm to solve the same underlying equations,

³For updates, see `{AMUSE_DIR}/examples/textbook/updates/AMUSE_updates.pdf`.

Table A.22: Overview of the core selection of codes in AMUSE.

Module	Gravitational dynamics
code name	application domain
ph4	Direct N -body
SmallN	Direct N -body
BHTree	Tree-code
Petar	Hybrid direct-tree N -body
Mercury	Planetary systems
SecularMultiple	Hierarchical multiples
Module	Stellar evolution
code name	application domain
MESA	Heney stellar evolution
SeBa	Parametrized stellar evolution
Module	Hydrodynamics
code name	application domain
Fi	Smoothed-particles Hydrodynamics
Athena	Grid-based hydrodynamics
Module	Radiative transfer dynamics
code name	application domain
Mocassin	Monte-Carlo radiative transfer
SPHray	Monte-Carlo radiative transfer
Module	Chemical dynamics
code name	application domain
Krome	Equilibrium chemistry
UCLChem	Non-equilibrium chemistry
Module	Coupling and initial conditions
code name	application domain
Kepler	2-body solver
Kepler_orbiters	N -body planetary solvers
mmams	Stellar collision routines
nameclot	Initial conditions generator
Fastkick	GPU-kick routine for bridge
FractalCluster	Initial conditions generator
GalactICs	Initial conditions generator
HaloGen	Initial conditions generator
hop	Clump finder

Table A.23: Community modules in AMUSE, listed by domain and algorithm.

GRAVITATIONAL DYNAMICS	
Pure	AarsethZare, Brutus, Hermite, ph4, SmallN.
Direct	Nbody6++, Mercury, HiGPU, Huayno, Rebound,
Approximate	BHTree, Bonsai, Petar.
Dedicated	Kepler, kepler_orbiters, Sakura, TwoBody.
Limited	SecularMultiple, TidyMess.
	Etics.
STELLAR EVOLUTION	
Henyey	EVTwin, MESA.
Parameterized	BSE, SeBa, SSE, CashedSE.
HYDRODYNAMICS	
SPH	Fi, Gadget2
Grid	Athena, Capreole, Flash, MPI-AMRVAC.
Moving mesh	Arepo.
RADIATIVE TRANSFER	
Monte Carlo	Mocassin, SPHray.
Photon transport	SimpleX2.
Interstellar CHEMISTRY	
Chemistry	Krome. UCLChem.
MISCELLANEOUS	
Analysis	Hop.
Initial conditions	GalactICs, FractalCluster, mameclot.
Potentials	Galactics, HaloGen, Galpy.
Visualization	Vaex, Fresco.
Stellar evolution	Evtwin2Sse.
Disk evolution	Vader.
Planet formation	PACE.
Specialized	ImpactCrater, PPD.
Coupling	Bridge, Venice

while comparable codes solve the same physics but using a different numerical method. Here *the same* and *different* are used rather loosely; the terms are meant to indicate what alternative code might be used to address a particular problem. Specifically, it should be possible to substitute an equivalent code into a script designed for any given code, with minimal changes, so if a code has no script listed, try one of its equivalents.

Core selection: these are codes we support and continue to support. We either update the interface, incorporate bug fixes or even new version of the code if they

become available.

- **Athena** (core)

Grid-based astrophysical hydrodynamics solver with support for magnetic fields (Stone *et al.*, 2010). It was originally developed for studying the interstellar medium, star formation, and accretion flows, but can address many other hydrodynamics problems (Gardiner & Stone, 2005, 2008; Stone *et al.*, 2008).

Work domain: hydrodynamics/radiative transfer

Working example: `amuse.examples.test_jeans_instability`

Extra requirements/features: MPI parallel

Equivalent codes: **Flash**, **Capreole**

Comparable codes: **Fi**, **Gadget2**

- **BHTree** (core)

Second-order softened gravitational N -body code with shared time steps and approximate forces. This is a parallel implementation of the original Barnes-Hut tree code (Barnes & Hut, 1986).

Work domain: gravitational dynamics, in particular galaxy dynamics

Working example: `amuse.examples.particles_and_gas_in_cluster`

Extra requirements/features: MPI parallel, GPU

Equivalent codes: **Bonsai**, **Octgrav**

Comparable codes: **MI6**

- **EVTwin2SSE** (core)

Version of **EVTwin** modified to handle exceptions and hand them to **SSE** when **EVTwin** crashes. The handover algorithm performs a root-mean-square search on the relative differences in mass, radius, and luminosity between the last known **EVTwin** state and the entire stellar history of **SSE** until the first state of a stellar remnant type. A brief description was presented in Section 8.4.3.

Note that although the code supports running several stars simultaneously, the point where **EVTwin** crashes is strongly dependent on the initial mass of the stars. It is advisable to run the **EVTwin2SSE** code for one initial mass at a time. Note that although the code allows running several stars simultaneously the timepoint where **EVTwin** crashes is strongly dependant on the initial mass of the stars. It is advisable to run the **EVTwin2SSE** code for single initial masses at a time.

Work domain: stellar evolution

Working example: `amuse.examples.hr_diagram_cluster`

Extra requirements/features: **EVTwin**, **SSE**

Equivalent codes: None

Comparable codes: None.

- **Fi** (core)

MPI-parallel smoothed-particle hydrodynamics (SPH) code ([Hernquist & Katz, 1989](#); [Pelupessy *et al.*, 2004](#); [Gerritsen & Icke, 1997](#)).

Work domain: gravitational dynamics/hydrodynamics

Working example: `amuse.examples.gas_in_cluster`

Extra requirements/features: OpenMP parallel

Equivalent codes: `Gadget2`

Comparable codes: `Flash`, `Athena`

- **Flash** (core)

Grid-based radiative magnetohydrodynamics solver originally written to simulate nova explosions on white dwarfs ([Fryxell *et al.*, 2010](#)). Flash is modular at the compile level, with fundamental structure not dissimilar to AMUSE, using modular codes and providing an environment to combine them.

Work domain: hydrodynamics, radiative transport

Working example: None

Extra requirements/features: OpenMP parallel

Equivalent codes: `Athena`

Comparable codes: `Fi`, `GadGet2`

- **Hermite_GRX** (core)

MPI parallel shared block time step regularized 4th-order Hermite direct N -body code with post-Newtonian expansions.

Work domain: collisional relativistic gravitational dynamics, in particular for Galactic nuclei.

Working example: `amuse.examples.montgomery`

Equivalent codes: `hacs64`, `AarsethZare`, `ph4`, `Hermite`

Comparable codes: `Bonsai`, `MI6`, `BHTree`

- **Huayno** (core)

Class containing a large variety of N -body codes based on kick-drift-kick algorithms using coupled second-order Hamiltonian splitting strategies of tunable order ([Pelupessy *et al.*, 2012](#)). Developed for integrating planetary systems using shared or individual time steps.

Work domain: collisional gravitational dynamics, in particular for planetary systems and star clusters

Working example: `amuse.examples.pythagorean`

Extra requirements/features: MPI parallel, GPU, GRAPE

Equivalent codes: `Hermite`, `Kepler`

Comparable codes: `HiGPUs`, `ph4`, `Hermite`

- **Kepler** (core)

Kepler orbit manipulation functions, imported from Starlab ([Portegies Zwart *et al.*, 1999](#); [Hut *et al.*, 2010](#)).

Work domain: gravitational dynamics/data processing and analysis

Working example: `amuse.examples.kira`

Extra requirements/features: MPI parallel, GPU, GRAPE

Equivalent codes: `Hermite`, `Huayno`, `Kepler`

Comparable codes: `HiGPUs`, `ph4`, `Hermite`

- `kepler_orbiters` (core)

Solve Kepler's equations for some number of two-body problems with one shared body, using universal variables ([Bate *et al.*, 1971](#)), with a choice between `Sakura` ([Gonçalves Ferrari *et al.*, 2014](#)) or `Huayno` ([Pelupessy *et al.*, 2012](#)).

Work domain: gravitational dynamics/data processing and analysis

Working example: `amuse.examples.kepler_orbiters_planets_around_sun`

Extra requirements/features: MPI parallel, GPU, GRAPE

Equivalent codes: `Hermite`, `Kepler`

Comparable codes: `HiGPUs`, `ph4`, `Hermite`

- `MESA` (core)

Heney stellar evolution solver with adaptive mesh refinement ([Paxton *et al.*, 2010, 2011](#)).

Work domain: stellar evolution

Working example: `amuse.examples.helium_star` see Code example [6.7](#)

Extra requirements/features: OpenMP parallel

Equivalent codes: `EVTwin`

Comparable codes: `SeBa`, `SSE`

- `MWpotentialBovy2015` (aka `galpy`) (core)

Static galactic potential model with characteristics similar to the Milky Way Galaxy ([Bovy, 2015, 2014](#)). The model includes semi-analytic prescriptions for the bulge, disk ([Miyamoto & Nagai, 1975](#)), and a dark-matter halo ([Navarro *et al.*, 1996](#)).

Work domain: gravitational dynamics, for galactic background potentials

Working example: `None`

Extra requirements/features: `None`

Equivalent codes: `Galaxia`

Comparable codes: `GalactICs`, `FractalCluster`, `HaloGen`

- `ph4` (core)

MPI-parallel individual block time step 4th-order Hermite direct N -body code with GPU support.

Work domain: collisional gravitational dynamics, in particular star clusters

Extra requirements/features: MPI parallel, GPU

Working example: `amuse.examples.evolve_orbit_in_potential`

Equivalent codes: `hacs64`, `phiGRAPE`

Comparable codes: `Bonsai`, `MI6`

- **Phantom** (core)

Smoothed particles hydrodynamics solver

MPI-parallel smoothed-particle hydrodynamics (SPH) code written to perform cosmological structure-formation simulations ([Price *et al.*, 2018](#)).

Work domain: gravitational dynamics/hydrodynamics.

Working example: see **Ekster**. Extra requirements/features: MPI parallel

Equivalent codes: **Fi**, **Gadget2**.

Comparable codes: **Athena**, **Flash**

- **SeBa** (core)

Parameterized stellar evolution code, with heuristic recipes for the evolution of binary stars of arbitrary metallicity ([Portegies Zwart & Verbunt, 2012](#)). The code includes treatments of wind accretion, mass transfer through Roche-lobe overflow, and the effect of supernova explosions on binary orbital parameters. The code was developed for studying the formation and evolution of high-mass X-ray binaries and compact object mergers ([Portegies Zwart & Verbunt, 1996](#); [Portegies Zwart & Yungelson, 1998](#)), but later expanded to include the prescription of the evolution of white dwarf binaries ([Toonen *et al.*, 2012](#)). In a new version it also provides support for triple evolution ([Toonen *et al.*, 2016](#)).

Work domain: stellar, binary, and triple evolution

Working example: `amuse.examples.evolve_binary`

Extra requirements/features: None

Equivalent codes: **BSE**

Comparable codes: **MESA**, **EVTwin**

- **SecularMultiple** (core)

Secular (orbit-averaged) gravitational dynamics of hierarchical multiple systems composed of nested binary orbits ([Hamers & Portegies Zwart, 2016](#)); see also Section 4.4.4. Tidal interactions (companion as a single body) and relativistic corrections are incorporated.

Work domain: gravitational dynamics, in particular planetary systems

Working example: `amuse.examples.hierarchical_quadruple`, and see Section 4.4.4

Extra requirements/features: None

Equivalent codes: **Sakura**

Comparable codes: **Mikkola**

- **SmallN** (core)

Shared time step direct N body integrator using a fourth-order time-symmetrized Hermite scheme, and including a modified unperturbed treatment of close approaches. **SmallN** was first introduced in Starlab ([Portegies Zwart *et al.*, 1999](#); [Hut *et al.*, 2010](#)) to handle close encounters and resonant multiparticle interactions.

Work domain: few-body gravitational dynamics, in particular democratic multiple systems

Working example: `amuse.examples.particles_and_multiples`

Extra requirements/features: `amuse.examples.kira`, `amuse.examples.basic_multiples_si` (see Section 8.7.1)

Equivalent codes: `Kepler`, `Kepler_orbiter`, `Sakura`

Comparable codes: `Hermite`, `HiGPUs`, `ph4`, `Hermite`

- **SPHray** (core)

Time-dependent Monte-Carlo radiative transfer code ([Altay *et al.*, 2011](#)). The code was developed to couple a smoothed-particle hydrodynamics (SPH) code with radiative transfer to study the non-equilibrium ionization state (HI, HII, HeI, HeII, HeIII, e) and temperature (internal energy/entropy) of arbitrary density fields ([Altay *et al.*, 2008](#)). Table 7.1 presents an overview of the code's application domain.

Work domain: radiative transfer

Working example: `amuse.examples.iliev_test5`

Extra requirements/features: OpenMP parallel

Equivalent codes: `Mocassin`, `SimpleX`

Comparable codes: `MPIAMRVAC`, `Flash`.

- **Tres** (core)

The Triple-star Evolution Code ([Toonen *et al.*, 2012, 2016](#)). More a coupled AMUSE script than a separate code, combining `SeBa` and `Secularmultiples` by including tidal effects, spin evolution and mass transfer in triples.

Work domain: triple-star evolution Extra requirements/features: none.

Equivalent codes: none

Comparable codes: none.

Envelope selection: these are codes we support, but not explicitly. It is largely up to the user community to help maintain these codes. If you need to use one of these codes, it may be easiest to first find an equivalent alternative from the core selection of codes. If the demand for some of these codes grows, we will consider making them part of the core selection. At this point, few research projects have requested or used these codes, and therefore we cannot spare the effort to maintaining them. Our recommendation is to test these codes before usage.

- **AarsethZare** (envelope)

Regularized direct N -body code with a Bulirsch-Stoer ([Bulirsch & Stoer, 1964](#)) integrator with support for post-Newtonian relativistic corrections to 2.5th order ([Aarseth & Zare, 1974a,b](#)).

Work domain: gravitational dynamics, in particular few-body interactions

Working example: None

Extra requirements/features: None

Equivalent codes: **Brutus**, **Hermite_GRX**, **Huayno**

Comparable codes: **MI6**, **SmallN**

- **Bonsai** (envelope)

Second-order softened gravitational N -body code with shared time steps and approximate tree-code forces controlled by an opening angle θ . This code is massively parallel and designed to run concurrently on a large number (tested to up to 18,600) of graphical processing units ([Bédorf *et al.*, 2014](#)).

Work domain: gravitational dynamics, in particular galaxy dynamics

Working example: None

Extra requirements/features: MPI parallel, GPU, parallel-GPU

Equivalent codes from core: **BHTree**, **Octgrav**

Comparable codes: **MI6**

- **BSE** (envelope)

Parameterized stellar evolution code expanded with heuristic recipes for the evolution of binary stars ([Hurley *et al.*, 2013a](#)). The code includes treatments for wind accretion, mass transfer through Roche-lobe overflow, and the effect of supernova explosions on binary orbital parameters.

Work domain: stellar and binary evolution

Working example: None

Extra requirements/features: None

Equivalent codes: **SeBa**

Comparable codes: **MESA**, **EVTwin**

- **Brutus** (envelope)

Direct N -body code using Bulirsch-Stoer integration ([Bulirsch & Stoer, 1964](#)), with support for arbitrary-precision arithmetic to calculate the orbits of multiple self-gravitating objects ([Boekholt & Portegies Zwart, 2015](#)). An older version was called **adaptb**.

Work domain: collisionless gravitational dynamics, in particular few-body systems to high precision and accuracy.

Working example: None

Extra requirements/features: MPI parallel, GPU

Equivalent codes: None

Comparable codes: **AarsethZare**

- **EVTwin** (envelope)

Henry stellar evolution code for all ranges of mass and composition ([Eggleton *et al.*, 2011](#)).

Work domain: stellar and binary evolution

Working example: `amuse.examples.hr_diagram_cluster`

Extra requirements/features: None

Equivalent codes: **MESA**

Comparable codes: **SeBa**, **SSE**, **BSE**

- **GadGet2** (envelope)

MPI-parallel smoothed-particle hydrodynamics (SPH) code written to perform cosmological structure-formation simulations, but implemented for more general applications ([Springel, 2000](#)).

Work domain: gravitational dynamics/hydrodynamics, in particular cosmology

Working example: `amuse.examples.gas_in_cluster`

Extra requirements/features: MPI parallel

Equivalent codes: **Fi**

Comparable codes: **Athena**, **Flash**

- **Hermite** (envelope)

MPI parallel shared block time step 4th-order Hermite direct N -body code with GPU support ([McMillan, 2014](#)).

Work domain: collisional gravitational dynamics, in particular for star clusters

Working example: `amuse.examples.pythagorean`

Extra requirements/features: MPI parallel, GPU

Equivalent codes: **hacs64**, **phiGRAPE**, **ph4**

Comparable codes: **Bonsai**, **MI6**, **BHTree**

- **Krome** (envelope)

Chemical network for solving the chemodynamical kinetics of a system modeled by coupled ordinary differential equations ([Grassi *et al.*, 2014](#)). Incorporated processes include photochemistry, cooling, heating, dust treatment, and reverse kinetics. It was designed to be used for studying primordial chemistry in low-metallicity environments, molecular cloud chemistry, and planetary atmospheres and dust grain populations.

Work domain: chemical evolution/hydrodynamics

Working example: `amuse.examples.molecular_cloud_chemistry`

Extra requirements/features: None

Equivalent codes: None

Comparable codes: **UCLChem**

- **MI6** (envelope)

MPI-parallel block-time step 6th-order Hermite direct N -body code with support for GPU and GRAPE ([Iwasawa *et al.*, 2015](#)). It was designed for integrating stellar orbits around a supermassive black hole, and has special treatment for this. It includes post-Newtonian expansions to general relativity up to 3.5th order (gravitational wave kick).

Work domain: collisional gravitational dynamics, in particular for galactic nuclei

Working example: `amuse.examples.plot_nbody_performance`

Extra requirements/features: MPI parallel, GPU, GRAPE

Equivalent codes: `Mikkola`, `Pikachu` ([Fujii *et al.*, 2012](#))

Comparable codes: `HiGPUs`, `ph4`, `Hermite`

- `Mikkola` (envelope)

Direct N -body code for self-gravitating systems, using algorithmic regularization and incorporating post-Newtonian effects ([Mikkola, 1983](#); [Mikkola & Merritt, 2008](#)). The code was designed for integrating stellar orbits around a supermassive black hole. It includes post-Newtonian expansions to general relativity to 2.5th order (gravitational wave radiation).

Work domain: collisional gravitational dynamics, in particular galactic nuclei

Extra requirements/features: None

Equivalent codes: `MI6`

Comparable codes: `HiGPUs`, `ph4`, `Hermite`, `SmallN`

- `Mercury` (envelope)

Symplectic N -body solver dedicated to studying the long-term dynamical evolution of planetary systems ([Chambers & Migliorini, 1997](#); [Chambers, 2012](#)). The equations of motion are solved using a variety of integrators, including mixed-variable symplectic methods (which are in fact quite similar to the `Bridge` method discussed in § Chapter 9), the Bulirsch-Stoer method, and a hybridization for studying planetary coagulation.

Work domain: gravitational dynamics, in particular planetary systems

Working example: `amuse.examples.mercury_example`

Extra requirements/features: None

Equivalent codes: `Huayno`, `Sakura`

Comparable codes: `HiGPUs`, `ph4`, `Hermite`, `Kepler`

- `Mocassin` (envelope)

Three-dimensional Monte Carlo photoionization code with radiative transfer through arbitrary geometry and density distributions ([Ercolano, 2011](#)). The code was originally developed to model photoionized regions such as HII regions and planetary nebulae ([Ritzerveld & Icke, 2006](#)). Table 7.1 presents an overview of the code's application domain.

Work domain: radiative transfer

Working example: `amuse.examples.h2region`

Extra requirements/features: None

Equivalent codes: `SimpleX2`, `SPHray`

Comparable codes: `MPIAMRVAC`, `Flash`

- `Nbody6++` (envelope)

Direct N -body code using a 4th order Hermite integrator with individual block time steps and regularized treatment of multiple interactions ([Aarseth, 2011](#)).

Work domain: collisional gravitational dynamics, in particular star clusters

Working example: None

Extra requirements/features: MPI parallel, GPU, GRAPE

Equivalent codes: `hacs64`, `MI6`

Comparable codes: `HiGPUs`, `ph4`, `Hermite`

- **Octgrav** (envelope)

MPI-parallel and GPU-enabled tree code up to octupole moments ([Gaburov et al., 2010b](#)).

Work domain: collisionless gravitational dynamics, in particular star clusters

Working example: None

Extra requirements/features: MPI parallel, GPU

Equivalent codes: `BHTree`, `Bonsai`

Comparable codes: `HiGPUs`, `ph4`, `Hermite`

- **phiGRAPE** (envelope)

Block time step 4th-order Hermite direct N -body code with support for GRAPE and/or GPU ([Harfst et al., 2007](#)).

Work domain: collisional gravitational dynamics, in particular star clusters

Extra requirements/features: MPI parallel, GRAPE-4, GRAPE-6, GPU

Equivalent codes: `hacs64`, `ph4`

Comparable codes: `Bonsai`, `MI6`

- **PPD** (envelope)

The FRIED interpolator object. Performs linear interpolation on the FRIED grid ([Haworth et al., 2020](#)).

Work domain: proto planetary disks in stellar clusters. Extra requirements/features: none.

Equivalent codes: none

Comparable codes: `Mocassin`, `SPHray`.

- **Rebound** (envelope)

Symplectic direct N -body integrator designed to study planetary rings, but of more general applicability ([Rein & Liu, 2012](#); [Rein & Tamayo, 2018](#); [Rein & Liu, 2011](#)).

Work domain: gravitational dynamics, in particular planetary systems

Working example: None

Extra requirements/features: MPI parallel

Equivalent codes: `Huayno`, `Mercury`, `Sakura`

Comparable codes: `ph4`, `Hermite`

- **Sakura** (envelope)

Kepler-based Hamiltonian splitting method for solving the gravitational N -body problem ([Gonçalves Ferrari et al., 2014](#)). This is a general N -body code, but due

to the imposed data structure it is ideally suited for integrating systems with large binary fractions.

Work domain: collisional gravitational dynamics, in particular planetary systems and star clusters

Extra requirements/features: MPI parallel

Working example: None

Equivalent codes: `SecularMultiple`, `Huayno`, `Kepler_orbiter`

Comparable codes: `hacs64`, `phiGRAPE`

- **SimpleX** (envelope)

Radiative transfer solver for an optically thin medium using unstructured Delaunay grid (Paardekooper *et al.*, 2010). Simplex was originally developed for cosmological radiative transfer (Altay *et al.*, 2008). Table 7.1 presents an overview of the code’s application domain.

Work domain: radiative transfer

Working example: `amuse.examples.iliev_test5`

Extra requirements/features: OpenMP parallel

Equivalent codes: `Mocassin`, `SPHray`

Comparable codes: `MPIAMRVAC`, `Flash`

- **SSE** (envelope)

Parameterized stellar evolution code with heuristic recipes for the evolution of single stars with arbitrary metallicity (Hurley *et al.*, 2013b). The code was originally used for studying stellar populations in star clusters (Hurley *et al.*, 2000).

Work domain: single stellar evolution

Working example: None

Extra requirements/features: None

Equivalent codes: `BSE`, `SeBa`

Comparable codes: `MESA`, `EVTwin`

- **TPI** (envelope)

CUDA code for test-particle integration.

Work domain: gravitational dynamics, in particular planetesimal or test-particle dynamics.

Working example: none. Extra requirements/features: CUDA

Equivalent codes: `SmallN`, `Kepler`

Comparable codes: `Huayno`, `Sakura`

- **twobody** (envelope)

Python module to solve Kepler’s equations for two-body problems using universal variables (Bate *et al.*, 1971).

Work domain: gravitational dynamics, in particular two-body dynamics

Working example: None

Extra requirements/features: None
 Equivalent codes: **SmallN**, **Kepler**
 Comparable codes: **Huayno**, **Sakura**

- **Vader** (envelope)

Viscous disk evolution code ([Krumholz & Forbes, 2015](#)). We extended the code to include internal photoevaporation (IPE) with mass loss profiles. It is build to work with the FRIED grid through the PPD class in AMUSE.

Work domain: proto planetary disks in stellar clusters. Extra requirements/features: none.

Equivalent codes: none

Comparable codes: **Mocassin**, **SPHray**.

Incomplete interface: these codes have been incorporated at some time into AMUSE, but are not longer (or not yet) explicitly supported. We consider them interesting, but they are not often used and we do not have the resources to maintain them. If you plan on using one of these, contact us to see if other users are interested in joining in to improve and complete the interface.

- **Arepo**

Vorenoi mesh hydrodynamical solver for simulationg interstellar gas and solids ([Springel, 2010a](#)).

Work domain: collisional gravitational dynamics with tides

Working example: None

Extra requirements/features: MPI parallel

Equivalent codes: None.

Comparable codes: **Fi**, **Gadget2**, **Athena**, **Flash**

- **Capreole**

MPI-parallel grid-based astrophysical hydrodynamics code ([Mellema *et al.*, 1991](#)). Works in one, two, or three spatial dimensions. For the hydrodynamics, it relies on the Roe-Eulderink-Mellema solver, which is an approximate Riemann solver for arbitrary metrics. It can solve different hydrodynamical problems. Capreole has run on single processors, but also on massively parallel systems (up to 512 processors on BlueGene/L).

Work domain: hydrodynamics, in particular cosmology

Working example: `amuse.examples.linear_wave`

Extra requirements/features: OpenMP parallel

Equivalent codes: **Athena**, **Flash**,

Comparable codes: **Fi**, **GadGet2**

- **Etics**

Gravity solver for collisionless stellar dynamical simulations using two fast force calculation methods based on series expansions of the Poisson equation ([Meiron *et al.*, 2014](#)). One method is the self-consistent field model, a Fourier-like expansion of the density field in some basis set. The other method is a multipole expansion method, a Taylor-like expansion of the Green's function.

Work domain: collisionless gravitational dynamics, in particular elliptical galaxies

Working example: None

Extra requirements/features: MPI parallel, GPU

Equivalent codes: None

Comparable codes: **Bonsai**, **Octgrav**

- **HiGPUs**

Block-timestep 6th-order Hermite direct gravitational N -body code ([Spera *et al.*, 2012](#)).

Work domain: collisional gravitational dynamics, in particular star clusters or planetary systems

Working example: None

Extra requirements/features: MPI parallel, GPU

Equivalent codes: **MI6**

Comparable codes: **ph4**, **Hermite**

- **hacs64**

Block time step Hermite direct N -body code with Ahmad-Cohen neighbor scheme and GPU support. The code uses a 6th-order Hermite integrator for the irregular step and a 4th-order Hermite for the regular step.

Work domain: collisional gravitational dynamics

Working example: None

Extra requirements/features: GPU

Equivalent codes: **MI6**

Comparable codes: **HiGPUs**, **ph4**, **Hermite**

- **ImpactCrater**

Impact crater size calculator. Simulates the effect of an impact of a minor body on a rocky major body.

Work domain: collisional dynamics.

Working example: None

Extra requirements/features: none

Equivalent codes: none

Comparable codes: **Fi**, **Gadget2**, **Phantom**, **Arepo**

- **MPI-AMRVAC**

Three-dimensional magnetohydrodynamics Adaptive Mesh Refinement (AMR) code with radiative transfer ([van der Holst et al., 2012](#); [Xia et al., 2018](#)),⁴ using the versatile advection method ([Tóth & Keppens, 2012](#)). The code was developed for simulating shock-dominated problems ([Meliani et al., 2007](#); [Keppens et al., 2012](#)), but handles smooth flows as well ([Porth et al., 2014](#)).

Work domain: hydrodynamics

Working example: `amuse.examples.linear_wave_boundaries`

Extra requirements/features: MPI parallel

Equivalent codes: **Flash**

Comparable codes: **Athena**

- **PACE**

Planet population synthesis code for simulating the formation of planetary systems from photoevaporated (PPD) viscous (vader) disks [Huang et al. \(2024\)](#).

Work domain: Planet formation.

Working example: None

Extra requirements/features: **Venice, Vader, PPD**

Equivalent codes: none

Comparable codes: none

- **Ramses** (deprecated)

Adaptive Mesh Refinement hydrodynamics solver with a tree-based data structure allowing recursive grid refinements on a cell-by-cell basis ([Teyssier, 2010](#)). Based on the method of [Kravtsov et al. \(1997\)](#) and designed for studying structure formation in the universe ([Teyssier, 2002](#)).

Work domain: hydrodynamics/radiative transfer, in particular cosmology

Working example: `amuse.examples.linear_wave_boundaries`

Extra requirements/features: None

Equivalent codes: **Flash, Athena, Capreole, MPI-AMRVAC**

Comparable codes: **Fi, Gadget2**

- **Tupan**

Direct N -body code for collisional stellar dynamics. Like **Huayno** it contains a wide range of different integrators. The code was originally written by Guilherme Goncalves, but regrettably never published.

Work domain: collisional gravitational dynamics

Working example: None

Extra requirements/features: MPI parallel, GPU

Equivalent codes: **Huayno, Sakura, Sei**

Comparable codes: **Kepler**

⁴See also <http://amrvac.org/> or the github version at <https://github.com/amrvac/amrvac>.

- **TidyMess**

Direct N -body code with tidal evolution solver. Excellently suited for chaotic N -body problems for which tides are relevant (Boekholt & Correia, 2023).

Work domain: collisional gravitational dynamics with tides

Working example: None

Extra requirements/features: MPI parallel

Equivalent codes: **Huayno**, **Sakura**, **Sei**

Comparable codes: **Kepler**

- **Venice**

General solver for matrices of differential equations. primarily used for bridging multiple codes (Wilhelm & Portegies Zwart, 2024)

Work domain: General multi-code bridging strategy.

Working example: see Section 10.1.3

Extra requirements/features: none

Equivalent codes: none

Comparable codes: **PACE**

Support codes: these are codes for

- **FractalCluster**

Generator for producing fractal stellar distributions by defining a cube of side 2 which is subsequently divided into sub-cubes, on which the same star addition algorithm is used. Eventually this procedure leads to a 3-dimensional, scale-free distribution of equal-mass particles (Goodwin & Whitworth, 2004).

Work domain: initial conditions

Working example: `amuse.examples.plot_fractal_clumpy_cluster`

Extra requirements/features: None

Equivalent codes: None

Comparable codes: **GalactICs** **HaloGen**

- **Fresco**

Visualization package specific for making “observational” images of simulations, focused on simulations in which stellar evolution, stellar dynamics, and possibly gas dynamics are incorporated. **Fresco** is not in the **amuse** github page, but can be downloaded from github at <https://github.com/rieder/Fresco>.

Work domain: visualization

Working example: None

Extra requirements/features: None

Equivalent codes: None

Comparable codes: **vaex**

- **GalactICs**

Generator for an N -body particle realization of an axisymmetric galaxy that consists of a disk, bulge, and halo ([Kuijken & Dubinski, 2011](#)), based on the original work by [Kuijken & Dubinski \(1995\)](#).

Work domain: initial conditions

Working example: None

Extra requirements/features: None

Equivalent codes: `FractalCluster`, `HaloGen`

Comparable codes: `Galaxia`

- **Galaxia**

Dynamic potential of a galactic model with characteristics similar to the Milky Way Galaxy ([Sharma *et al.*, 2011](#)). The model includes semi-analytic prescriptions for the axisymmetric parts (bulge, disk, halo ([Allen & Santillan, 1991](#))), a parameterization for the central (Ferrers) bar ([Romero-Gómez *et al.*, 2007, 2011](#)) and transient 2-D and 3-D spiral arms ([Cox & Gómez, 2002](#)).

Work domain: gravitational dynamics, in particular for background potentials.

Working example: None

Extra requirements/features: None

Equivalent codes: `MWpotentialBovy2015`

Comparable codes: `GalactICs`, `FractalCluster`, `HaloGen`

- **HaloGen**

Generator for an N -body particle realization of the Galactic halo ([Zemp *et al.*, 2008](#); [Zemp, 2014](#); [Avila Perez & Murray, 2015](#)) by generating cosmological tracer distributions (halos) for a range of different cosmologies.

Work domain: initial conditions, in particular for galaxies

Working example: `amuse.examples.solar_cluster_in_live_galaxy`

Extra requirements/features: parallel

Equivalent codes: `GalactICs`, `FractalCluster`.

Comparable codes: `Galaxia`

- **hop**

Coordinate-free group-finding algorithm for particle distributions in three dimensions ([Eisenstein & Hut, 2011](#)). Original use was finding groups of particles in N -body realizations in Cartesian coordinates.

Work domain: data processing and analysis

Working example: `amuse.examples.clump_finding`

Extra requirements/features: None

Equivalent codes: None

Comparable codes: None

- **mameclot**

Generator for an N -body particle realization of one or two star clusters in orbit around each other (see <https://github.com/mgieles/mameclot>). The velocities can be isotropic or radially anisotropic (Osipkov, 1979; Merritt, 1985).

Work domain: initial conditions

Working example: None

Extra requirements/features: None

Equivalent codes: None

Comparable codes: `FractalCluster`, `GalactICs`

- **mmams**

Fast stellar collision method using Archimedes' principle to merge stars, including rudimentary treatments for shock heating, hydrodynamic mixing, mass ejection, and angular momentum transfer (Gaburov *et al.*, 2008; Lombardi & Sawyer Warren, 2014).

Work domain: resolving stellar collisions

Working example: see Code example 8.10

Extra requirements/features: None

Equivalent codes: None

Comparable codes: `Gadget2`, `Fi`

- **MMC**

Monte-Carlo star cluster dynamics simulation code (Askar *et al.*, 2017). The code was developed to compare simulation results with observed globular clusters (Giersz, 1998, 2001).

Work domain: gravitational dynamics

Working example: None

Extra requirements/features: None

Equivalent codes: None

Comparable codes: None.

- **UCLChem**

Chemical network for solving the chemodynamical kinetics of a system modeled by coupled ordinary differential equations (Holdship *et al.*, 2017; Keil *et al.*, 2022). Incorporated processes include photochemistry, cooling, heating, dust treatment, and reverse kinetics. It was designed to be used for studying primordial chemistry in low-metallicity environments, molecular cloud chemistry, and planetary atmospheres and dust grain populations.

Work domain: chemical evolution/hydrodynamics

Working example: None

Extra requirements/features: None

Equivalent codes: None

Comparable codes: `Krome`

- **Vaex**

Visualization package written in Python for analyzing large data volumes. It was written by Maarten Breddels as a project at Groningen University (Breddels, 2017), see also <https://vaex.io>.

Work domain: visualization and analysis

Working example: None

Extra requirements/features: does not require but can use the GPU.

Equivalent codes: None

Comparable codes: `pyplot`, `gnuplot`

We are sorry if your code is not among the listed modules, and we will gladly work with you to include it. Writing a custom interface to a new code is relatively easy, but regrettably, this is not the right place to explain how to develop a new AMUSE interface. However, we can teach you how to do this, or you may want to visit the amusecode.org web-site for a brief tutorial.

A.8.3 Packages removed since the 2018 version of the book

Regrettably, we cannot muster the support we would love to provide for all packages to which AMUSE has an interface. Some of these packages have similar functionality as others, and some are simply not used a lot. The result is that bit-rot sets in, compilers change, and it turns out that modules are not always written in a sustainable way. Most FORTRAN code remains compilable, but for many C and C++ packages this is not the case. Also, some of these codes have become obsolete, because new versions or better implementations of the same processes have become available. So long as no new bugs are found, or if the original authors remain accessible, this does not necessarily pose a problem. However, not using a code for a while does not stop compilers and computers from changing, and therefore the codes may stop working eventually.

An example is the code `Adaptb`, which basically is an early version of `Brutus` (Boekholt & Portegies Zwart, 2015). The code `mmc` (Leveque *et al.*, 2022), was never properly incorporated into the AMUSE framework. We will miss the GPU-supported treecode in `Octgrav` (Gaburov *et al.*, 2010b,a), but this was simply replaced by `Bonsai` (Bédorf *et al.*, 2014). The `pikachu` code (Iwasawa *et al.*, 2015) was replaced by `PeTar` (Wang *et al.*, 2015). Although `hacs64` has a 6th-order predictor-corrector Hermite integrator for direct N -body with an Ahmad-Cohen Scheme (Ahmad & Cohen, 1973) running on GPU, a combination of method nowhere else available in AMUSE, there are high order integrators in `Symple`, and we can relatively easily expand `ph4` to include higher orders. Eventually, `sei` (Rein & Tremaine, 2011) also had to go, even though it is a nice and effective 183-line direct N -body code, it was replaced with `Rebound` (Rein & Spiegel, 2015) which utilizes the faster IAS15 integrator.

Bibliography

Aarseth, S. J., & Zare, K. 1974a. A Regularization of the Three-Body Problem. *Celestial Mechanics*, **10**(2), 185–205.

- Aarseth, S. J., & Zare, K. 1974b. Errata: “A regularization of the three-body problem” [Celestial Mech., Vol. 10, p. 185 - 205 (1974)]. *Celestial Mechanics*, **10**(Dec.), 516.
- Aarseth, Sverre J. 2011 (Feb.). *NBODY Codes: Numerical Simulations of Many-body (N-body) Gravitational Interactions*. Astrophysics Source Code Library, record ascl:1102.006.
- Ahmad, A., & Cohen, L. 1973. Random Force in Gravitational Systems. *ApJ*, **179**(Feb.), 885–896.
- Allen, Christine, & Santillan, Alfredo. 1991. An improved model of the galactic mass distribution for orbit computations. *Revista Mexicana de Astronomia y Astrofisica*, **22**(Oct.), 255.
- Altay, Gabriel, Croft, Rupert A. C., & Pelupessy, Inti. 2008. SPHRAY: a smoothed particle hydrodynamics ray tracer for radiative transfer. *MNRAS*, **386**(4), 1931–1946.
- Altay, Gabriel, Croft, Rupert A. C., & Pelupessy, I. 2011 (Mar.). *SPHRAY: A Smoothed Particle Hydrodynamics Ray Tracer for Radiative Transfer*. Astrophysics Source Code Library, record ascl:1103.009.
- Askar, Abbas, Giersz, Mirek, Pych, Wojciech, & Dalessandro, Emanuele. 2017 (Mar.). *COCOA: Simulating Observations of Star Cluster Simulations*. Astrophysics Source Code Library, record ascl:1703.002.
- Avila Perez, Santiago, & Murray, Steven. 2015 (May). *HALOGEN: Approximate synthetic halo catalog generator*. Astrophysics Source Code Library, record ascl:1505.017.
- Barnes, Josh, & Hut, Piet. 1986. A hierarchical $O(N \log N)$ force-calculation algorithm. *Nat*, **324**(6096), 446–449.
- Bate, R. R., Mueller, D. D., & White, J. E. 1971. *Fundamentals of astrodynamics*.
- Bédorf, Jeroen, Gaburov, Evghenii, Fujii, Michiko S., Nitadori, Keigo, Ishiyama, Tomoaki, & Portegies Zwart, Simon. 2014. 24.77 Pflops on a Gravitational Tree-code to Simulate the Milky Way Galaxy with 18600 GPUs. *Pages 54–65 of: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. SC '14. Piscataway, NJ, USA: IEEE Press.
- Boekholt, Tjarda, & Portegies Zwart, Simon. 2015. On the reliability of N-body simulations. *Computational Astrophysics and Cosmology*, **2**(Mar.), 2.
- Boekholt, Tjarda C. N., & Correia, Alexandre C. M. 2023. A direct N-body integrator for modelling the chaotic, tidal dynamics of multibody extrasolar systems: TIDYMESS. *MNRAS*, **522**(2), 2885–2900.
- Bovy, Jo. 2014 (Nov.). *galpy: Galactic dynamics package*. Astrophysics Source Code Library, record ascl:1411.008.
- Bovy, Jo. 2015. galpy: A python Library for Galactic Dynamics. *ApJS*, **216**(2), 29.
- Breddels, M. A. 2017 (June). Interactive (statistical) visualisation and exploration of a billion objects with vaex. *Pages 299–304 of: Brescia, Massimo, Djorgovski, S. G., Feigelson, Eric D., Longo, Giuseppe, & Cavuoti, Stefano (eds), Astroinformatics*, vol. 325.
- Bulirsch, Roland, & Stoer, Josef. 1964. Fehlerabschätzungen und Extrapolation mit rationalen Funktionen bei Verfahren vom Richardson-Typus. *Numerische Mathematik*, **6**, 413–427. 10.1007/BF01386092.
- Chambers, J. E., & Migliorini, F. 1997 (July). Mercury - A New Software Package for

- Orbital Integrations. *Page 27.06 of: AAS/Division for Planetary Sciences Meeting Abstracts #29*. AAS/Division for Planetary Sciences Meeting Abstracts, vol. 29.
- Chambers, John E. 2012 (Jan.). *Mercury: A software package for orbital dynamics*. Astrophysics Source Code Library, record ascl:1201.008.
- Cox, Donald P., & Gómez, Gilberto C. 2002. Analytical Expressions for Spiral Arm Gravitational Potential and Density. *ApJS*, **142**(2), 261–267.
- Eggleton, P. P., Tout, Christopher, Pols, Onno, Izzard, Rob, Eldridge, John, Lesaffre, Pierre, Stancliffe, Richard, Church, Ross, & Lau, Herbert. 2011 (July). *STARS: A Stellar Evolution Code*. Astrophysics Source Code Library, record ascl:1107.008.
- Einstein, A. 1915a. Zur allgemeinen Relativitätstheorie. *Sitzungsberichte der Königlich Preußischen Akademie der Wissenschaften (Berlin)*, Seite 778–786., 778–786.
- Einstein, A. 1915b. Zur allgemeinen Relativitätstheorie (Nachtrag). *Sitzungsberichte der Königlich Preußischen Akademie der Wissenschaften (Berlin)*, Seite 799–801., 799–801.
- Einstein, Albert, & Grossmann, Marcel. 1914. Kovarianzeigenschaften der Feldgleichungen der auf die verallgemeinerte Relativitätstheorie gegründeten Gravitationstheorie. *Zeitschrift für Mathematik und Physik*, **63**(Jan.), 215–225.
- Eisenstein, Daniel, & Hut, Piet. 2011 (Feb.). *HOP: A Group-finding Algorithm for N-body Simulations*. Astrophysics Source Code Library, record ascl:1102.019.
- Emden, R. 1907. *Gaskugeln: Anwendungen der mechanischen Wärmetheorie auf kosmologische und meteorologische Probleme*. Leipzig: B. G. Teubner.
- Ercolano, Barbara. 2011 (Oct.). *MOCASSIN: MONte CARlo SimulationS of Ionized Nebulae*. Astrophysics Source Code Library, record ascl:1110.010.
- Euler, Leonhard. 1768. *Institutionum calculi integralis*. St. Petersburg: Academy of Sciences. Vols. 1–3.
- Fryxell, B., Olson, K., Ricker, P., Timmes, F. X., Zingale, M., Lamb, D. Q., MacNeice, P., Rosner, R., Truran, J. W., & Tufo, H. 2010 (Oct.). *FLASH: Adaptive Mesh Hydrodynamics Code for Modeling Astrophysical Thermonuclear Flashes*. Astrophysics Source Code Library, record ascl:1010.082.
- Fujii, M. S., Iwasawa, M., Saitoh, T. R., & Portegies Zwart, S. 2012 (July). N-body Simulations of Star Clusters, Black Holes, and Their Host Systems using BRIDGE. *Page 211 of: Capuzzo-Dolcetta, R., Limongi, M., & Tornambè, A. (eds), Advances in Computational Astrophysics: Methods, Tools, and Outcome*. Astronomical Society of the Pacific Conference Series, vol. 453.
- Fujii, Michiko, Iwasawa, Masaki, Funato, Yoko, & Makino, Junichiro. 2007. BRIDGE: A Direct-Tree Hybrid N-Body Algorithm for Fully Self-Consistent Simulations of Star Clusters and Their Parent Galaxies. *Publ. Astr. Soc. Japan*, **59**(Dec.), 1095.
- Gaburov, E., Lombardi, J. C., & Portegies Zwart, S. 2008. Mixing in massive stellar mergers. *MNRAS*, **383**(1), L5–L9.
- Gaburov, E., Bédorf, J., & Portegies Zwart, S. 2010a. Gravitational tree-code on graphics processing units: implementation in CUDA. *Procedia Computer Science*, volume 1, p. 1119–1127, **1**(May), 1119–1127.
- Gaburov, Evghenii, Bédorf, Jeroen, & Portegies Zwart, Simon. 2010b (Oct.). *OCT-GRAV: Sparse Octree Gravitational N-body Code on Graphics Processing Units*. Astrophysics Source Code Library, record ascl:1010.048.

- Gaburov, Evghenii, Lombardi, James C. Jr., Portegies Zwart, Simon, & Rasio, F. A. 2018 (May). *StarSmasher: Smoothed Particle Hydrodynamics code for smashing stars and planets*. Astrophysics Source Code Library, record ascl:1805.010.
- Gardiner, Thomas A., & Stone, James M. 2005. An unsplit Godunov method for ideal MHD via constrained transport. *Journal of Computational Physics*, **205**(2), 509–539.
- Gardiner, Thomas A., & Stone, James M. 2008. An unsplit Godunov method for ideal MHD via constrained transport in three dimensions. *Journal of Computational Physics*, **227**(8), 4123–4141.
- Gear, C. W. 1967. The Numerical Integration of Ordinary Differential Equations. *Mathematics of Computation*, **21**(98), 146–156.
- Gerritsen, J. P. E., & Icke, V. 1997. Star formation in N-body simulations. I. The impact of the stellar ultraviolet radiation on star formation. *A&A*, **325**(Sept.), 972–986.
- Giersz, Mirek. 1998. Monte Carlo simulations of star clusters - I. First Results. *MNRAS*, **298**(4), 1239–1248.
- Giersz, Mirek. 2001. Monte Carlo simulations of star clusters - II. Tidally limited, multimass systems with stellar evolution. *MNRAS*, **324**(1), 218–230.
- Gonçalves Ferrari, G., Boekholt, T., & Portegies Zwart, S. F. 2014. A Keplerian-based Hamiltonian splitting for gravitational N-body simulations. *MNRAS*, **440**(1), 719–730.
- Goodwin, S. P., & Whitworth, A. P. 2004. The dynamical evolution of fractal star clusters: The survival of substructure. *A&A*, **413**(Jan.), 929–937.
- Grassi, Tommaso, Bovino, Stefano, Prieto, Joaquín, Seifried, Daniel, Simoncini, Eugenio, Gianturco, Francesco, & Schleicher, Dominik. 2014 (Feb.). *KROME: Chemistry package for astrophysical simulations*. Astrophysics Source Code Library, record ascl:1402.011.
- Hairer, Ernst, Lubich, Christian, & Wanner, Gerhard. 2006. *Geometric Numerical Integration: Structure-Preserving Algorithms for Ordinary Differential Equations; 2nd ed.* Dordrecht: Springer.
- Hamers, Adrian S., & Portegies Zwart, Simon F. 2016. Secular dynamics of hierarchical multiple systems composed of nested binaries, with an arbitrary number of bodies and arbitrary hierarchical structure. First applications to multiplanet and multistar systems. *MNRAS*, **459**(3), 2827–2874.
- Harfst, Stefan, Gualandris, Alessia, Merritt, David, Spurzem, Rainer, Portegies Zwart, Simon, & Berczik, Peter. 2007. Performance analysis of direct N-body algorithms on special-purpose supercomputers. *New Astron.*, **12**(5), 357–377.
- Haworth, Thomas J., Cadman, James, Meru, Farzana, Hall, Cassandra, Albertini, Emma, Forgan, Duncan, Rice, Ken, & Owen, James E. 2020. Massive discs around low-mass stars. *MNRAS*, **494**(3), 4130–4148.
- Hernquist, Lars, & Katz, Neal. 1989. TREESPH: A Unification of SPH with the Hierarchical Tree Method. *ApJS*, **70**(June), 419.
- Hilbert, David. 1915. Die Grundlagen der Physik. 1. *Gott. Nachr.*, **27**, 395–407.
- Holdship, J., Viti, S., Jiménez-Serra, I., Makrymallis, A., & Priestley, F. 2017. UCLCHEM: A Gas-grain Chemical Code for Clouds, Cores, and C-Shocks. *AJ*

- , **154**(1), 38.
- Huang, Shuo, Portegies Zwart, Simon, & Wilhelm, Maite J. C. 2024. Suppression of giant planet formation around low-mass stars in clustered environments. *A&A*, **689**(Sept.), A338.
- Hunter, John D. 2007. Matplotlib: A 2D Graphics Environment. *Computing in Science and Engineering*, **9**(3), 90–95.
- Hurley, Jarrod R., Pols, Onno R., & Tout, Christopher A. 2000. Comprehensive analytic formulae for stellar evolution as a function of mass and metallicity. *MNRAS*, **315**(3), 543–569.
- Hurley, Jarrod R., Tout, Christopher A., & Pols, Onno R. 2013a (Mar.). *BSE: Binary Star Evolution*. Astrophysics Source Code Library, record ascl:1303.014.
- Hurley, Jarrod R., Pols, Onno R., & Tout, Christopher A. 2013b (Mar.). *SSE: Single Star Evolution*. Astrophysics Source Code Library, record ascl:1303.015.
- Hut, Piet, McMillan, Steve, Makino, Jun, & Portegies Zwart, Simon. 2010 (Oct.). *Starlab: A Software Environment for Collisional Stellar Dynamics*. Astrophysics Source Code Library, record ascl:1010.076.
- Iwasawa, Masaki, Portegies Zwart, Simon, & Makino, Junichiro. 2015. GPU-enabled particle-particle particle-tree scheme for simulating dense stellar cluster system. *Computational Astrophysics and Cosmology*, **2**(July), 6.
- Jiao, Yandong, Dai, Guidong, Feng, Quandong, & Tang, Yifa. 2007. NON-EXISTENCE OF CONJUGATE-SYMPLECTIC MULTI-STEP METHODS OF ODD ORDER. *Journal of Computational Mathematics*, **25**(6), 690–696.
- Keil, Marcus, Viti, Serena, & Holdship, Jonathan. 2022. UCLCHEMCMC: An MCMC Inference Tool for Physical Parameters of Molecular Clouds. *ApJ*, **927**(2), 203.
- Keppens, R., Meliani, Z., van Marle, A. J., Delmont, P., Vlasis, A., & van der Holst, B. 2012. Parallel, grid-adaptive approaches for relativistic hydro and magnetohydrodynamics. *Journal of Computational Physics*, **231**(3), 718–744.
- Kerr, Roy P. 1963. Gravitational Field of a Spinning Mass as an Example of Algebraically Special Metrics. *Phys. Rev. Lett.*, **11**(5), 237–238.
- Kluyver, Thomas, Ragan-Kelley, Benjamin, Perez, Fernando, Granger, Brian, Bussonier, Matthias, Frederic, Jonathan, Kelley, Kyle, Hamrick, Jessica, Grout, Jason, Corlay, Sylvain, Ivanov, Paul, Avila, Damián, Abdallah, Safia, Willing, Carol, & Jupyter, Project. 2016. Jupyter Notebooks: a publishing format for reproducible computational workflows. IOS Press.
- Kozai, Yoshihide. 1962. Secular perturbations of asteroids with high inclination and eccentricity. *AJ*, **67**(Nov.), 591–598.
- Kravtsov, Andrey V., Klypin, Anatoly A., & Khokhlov, Alexei M. 1997. Adaptive Refinement Tree: A New High-Resolution N-Body Code for Cosmological Simulations. *ApJS*, **111**(1), 73–94.
- Krumholz, M. R., & Forbes, J. C. 2015. VADER: A flexible, robust, open-source code for simulating viscous thin accretion disks. *Astronomy and Computing*, **11**(June), 1–17.
- Kuijken, K., & Dubinski, J. 1995. Nearly Self-Consistent Disc / Bulge / Halo Models for Galaxies. *MNRAS*, **277**(Dec.), 1341.
- Kuijken, Konrad, & Dubinski, John. 2011 (Sept.). *GalactICS: Galaxy Model Building*

- Package*. Astrophysics Source Code Library, record ascl:1109.011.
- Lane, J. Homer. 1870. On the theoretical temperature of the sun under the hypothesis of a gaseous mass maintaining its volume by its internal heat and depending on the laws of gases as known to terrestrial experiment. *American Journal of Science and Arts, Second Series*, **50**, 57–74.
- Leveque, A., Giersz, M., Banerjee, S., Vesperini, E., Hong, J., & Portegies Zwart, S. 2022. A Monte Carlo study of early gas expulsion and evolution of star clusters: new simulations with the MOCCA code in the AMUSE framework. *MNRAS*, **514**(4), 5739–5750.
- Lidov, M. L. 1962. The evolution of orbits of artificial satellites of planets under the action of gravitational perturbations of external bodies. *Planet. Space Sci.*, **9**(10), 719–759.
- Lombardi, James, & Sawyer Warren, Jessica. 2014 (Dec.). *MMAS: Make Me A Star*. Astrophysics Source Code Library, record ascl:1412.010.
- McMillan, Stephen L. W. 2014 (May). Modeling the Formation and Evolution of Dense Stellar Systems. *Page 303.01 of: AAS/Division of Dynamical Astronomy Meeting #45*. AAS/Division of Dynamical Astronomy Meeting, vol. 45.
- Meiron, Yohai, Li, Baile, Holley-Bockelmann, Kelly, & Spurzem, Rainer. 2014. Expansion Techniques for Collisionless Stellar Dynamical Simulations. *ApJ*, **792**(2), 98.
- Meliani, Zakaria, Keppens, Rony, Casse, Fabien, & Giannios, Dimitrios. 2007. AM-RVAC and relativistic hydrodynamic simulations for gamma-ray burst afterglow phases. *MNRAS*, **376**(3), 1189–1200.
- Mellema, G., Eulderink, F., & Icke, V. 1991. Hydrodynamical models of aspherical planetary nebulae. *A&A*, **252**(Dec.), 718–732.
- Merritt, D. 1985. Spherical stellar systems with spheroidal velocity distributions. *AJ*, **90**(June), 1027–1037.
- Mewes, Vassilios, Zlochower, Yosef, Campanelli, Manuela, Ruchlin, Ian, Etienne, Zachariah B., & Baumgarte, Thomas W. 2018. Numerical relativity in spherical coordinates with the Einstein Toolkit. *Phys. Rev. D.*, **97**(8), 084059.
- Mewes, Vassilios, Zlochower, Yosef, Campanelli, Manuela, Baumgarte, Thomas W., Etienne, Zachariah B., Armengol, Federico G. Lopez, & Ciolletta, Federico. 2020. Numerical relativity in spherical coordinates: A new dynamical spacetime and general relativistic MHD evolution framework for the Einstein Toolkit. *Phys. Rev. D.*, **101**(10), 104007.
- Mikkola, S. 1983. Encounters of binaries. I - Equal energies. *MNRAS*, **203**(June), 1107–1121.
- Mikkola, Seppo, & Merritt, David. 2008. Implementing Few-Body Algorithmic Regularization with Post-Newtonian Terms. *AJ*, **135**(6), 2398–2405.
- Miyamoto, M., & Nagai, R. 1975. Three-dimensional models for the distribution of mass in galaxies. *Publ. Astr. Soc. Japan*, **27**(Jan.), 533–543.
- Navarro, Julio F., Frenk, Carlos S., & White, Simon D. M. 1996. The Structure of Cold Dark Matter Halos. *ApJ*, **462**(May), 563.
- Osipkov, L. P. 1979. Spherical systems of gravitating bodies with ellipsoidal velocity distribution. *Pisma v Astronomicheskii Zhurnal*, **5**(Feb.), 77–80.

- Paardekooper, J. P., Kruip, C. J. H., & Icke, V. 2010. SimpleX2: radiative transfer on an unstructured, dynamic grid. *A&A*, **515**(June), A79.
- Paxton, Bill, Bildsten, Lars, Dotter, Aaron, Herwig, Falk, Lesaffre, Pierre, & Timmes, Frank. 2010 (Oct.). *MESA: Modules for Experiments in Stellar Astrophysics*. Astrophysics Source Code Library, record ascl:1010.083.
- Paxton, Bill, Bildsten, Lars, Dotter, Aaron, Herwig, Falk, Lesaffre, Pierre, & Timmes, Frank. 2011. Modules for Experiments in Stellar Astrophysics (MESA). *ApJS*, **192**(1), 3.
- Paxton, Bill, Cantiello, Matteo, Arras, Phil, Bildsten, Lars, Brown, Edward F., Dotter, Aaron, Mankovich, Christopher, Montgomery, M. H., Stello, Dennis, Timmes, F. X., & Townsend, Richard. 2013. Modules for Experiments in Stellar Astrophysics (MESA): Planets, Oscillations, Rotation, and Massive Stars. *ApJS*, **208**(1), 4.
- Paxton, Bill, Marchant, Pablo, Schwab, Josiah, Bauer, Evan B., Bildsten, Lars, Cantiello, Matteo, Dessart, Luc, Farmer, R., Hu, H., Langer, N., Townsend, R. H. D., Townsley, Dean M., & Timmes, F. X. 2015. Modules for Experiments in Stellar Astrophysics (MESA): Binaries, Pulsations, and Explosions. *ApJS*, **220**(1), 15.
- Paxton, Bill, Schwab, Josiah, Bauer, Evan B., Bildsten, Lars, Blinnikov, Sergei, Duffell, Paul, Farmer, R., Goldberg, Jared A., Marchant, Pablo, Sorokina, Elena, Thoul, Anne, Townsend, Richard H. D., & Timmes, F. X. 2018. Modules for Experiments in Stellar Astrophysics (MESA): Convective Boundaries, Element Diffusion, and Massive Star Explosions. *ApJS*, **234**(2), 34.
- Pelupessy, F. I., van der Werf, P. P., & Icke, V. 2004. Periodic bursts of star formation in irregular galaxies. *A&A*, **422**(July), 55–64.
- Pelupessy, Federico I., Jänes, Jürgen, & Portegies Zwart, Simon. 2012. N-body integrators with individual time steps from Hierarchical splitting. *New Astron.*, **17**(8), 711–719.
- Plummer, H. C. 1911. On the problem of distribution in globular star clusters. *MNRAS*, **71**(Mar.), 460–470.
- Poisson, Eric, & Will, Clifford M. 2014. *Gravity*.
- Portegies Zwart, S. F., & Verbunt, F. 1996. Population synthesis of high-mass binaries. *A&A*, **309**(May), 179–196.
- Portegies Zwart, S. F., & Verbunt, F. 2012 (Jan.). *SeBa: Stellar and binary evolution*. Astrophysics Source Code Library, record ascl:1201.003.
- Portegies Zwart, S. F., & Yungelson, L. R. 1998. Formation and evolution of binary neutron stars. *A&A*, **332**(Apr.), 173–188.
- Portegies Zwart, S. F., Makino, J., McMillan, S. L. W., & Hut, P. 1999. Star cluster ecology. III. Runaway collisions in young compact star clusters. *A&A*, **348**(Aug.), 117–126.
- Portegies Zwart, S. F., Boekholt, T. C. N., Por, E. H., Hamers, A. S., & McMillan, S. L. W. 2022. Chaos in self-gravitating many-body systems. Lyapunov time dependence of N and the influence of general relativity. *A&A*, **659**(Mar.), A86.
- Portegies Zwart, Simon. 2018. Computational astrophysics for the future. *Science*, **361**(6406), 979–980.
- Portegies Zwart, Simon F., & Boekholt, Tjarda C. N. 2018. Numerical verification of the microscopic time reversibility of Newton’s equations of motion: Fighting expo-

- nential divergence. *Communications in Nonlinear Science and Numerical Simulations*, **61**(Aug.), 160–166.
- Porth, O., Xia, C., Hendrix, T., Moschou, S. P., & Keppens, R. 2014. MPI-AMRVAC for Solar and Astrophysics. *ApJS*, **214**(1), 4.
- Price, Daniel J., Wurster, James, Tricco, Terrence S., Nixon, Chris, Toupin, Stéven, Pettitt, Alex, Chan, Conrad, Mentiplay, Daniel, Laibe, Guillaume, Glover, Simon, Dobbs, Clare, Nealon, Rebecca, Liptai, David, Worpel, Hauke, Bonnerot, Clément, Dipierro, Giovanni, Ballabio, Giulia, Ragusa, Enrico, Federrath, Christoph, Iaconi, Roberto, Reichardt, Thomas, Forgan, Duncan, Hutchison, Mark, Constantino, Thomas, Ayliffe, Ben, Hirsh, Kieran, & Lodato, Giuseppe. 2018. Phantom: A Smoothed Particle Hydrodynamics and Magnetohydrodynamics Code for Astrophysics. *Proc. Astron. Soc. Aust.*, **35**(Sept.), e031.
- Rein, H., & Liu, S. F. 2012. REBOUND: an open-source multi-purpose N-body code for collisional dynamics. *A&A*, **537**(Jan.), A128.
- Rein, Hanno, & Liu, Shang-Fei. 2011 (Oct.). *REBOUND: Multi-purpose N-body code for collisional dynamics*. Astrophysics Source Code Library, record ascl:1110.016.
- Rein, Hanno, & Spiegel, David S. 2015. IAS15: a fast, adaptive, high-order integrator for gravitational dynamics, accurate to machine precision over a billion orbits. *MNRAS*, **446**(2), 1424–1437.
- Rein, Hanno, & Tamayo, Daniel. 2018. JANUS: a bit-wise reversible integrator for N-body dynamics. *MNRAS*, **473**(3), 3351–3357.
- Rein, Hanno, & Tremaine, Scott. 2011. Symplectic integrators in the shearing sheet. *MNRAS*, **415**(4), 3168–3176.
- Ritzerveld, Jelle, & Icke, Vincent. 2006. Transport on adaptive random lattices. *Phys. Rev. E*, **74**(2), 026704.
- Romero-Gómez, M., Athanassoula, E., Masdemont, J. J., & García-Gómez, C. 2007. The formation of spiral arms and rings in barred galaxies. *A&A*, **472**(1), 63–75.
- Romero-Gómez, M., Athanassoula, E., Antoja, T., & Figueras, F. 2011. Modelling the inner disc of the Milky Way with manifolds - I. A first step. *MNRAS*, **418**(2), 1176–1193.
- Schwarzschild, K. 1916. On the Gravitational Field of a Mass Point According to Einstein's Theory. *Abh. Konigl. Preuss. Akad. Wissenschaften Jahre 1906,92, Berlin, 1907*, **1916**, 189–196.
- Sharma, Sanjib, Bland-Hawthorn, Joss, Johnston, Kathryn V., & Binney, James. 2011 (Jan.). *Galaxia: A Code to Generate a Synthetic Survey of the Milky Way*. Astrophysics Source Code Library, record ascl:1101.007.
- Sobol, I.M. 1967. Distribution of Points in a Cube and Approximate Evaluation of Integrals. *Zh. Vych. Mat. Mat. Fiz.*, **7**, 784–802.
- Sofroniou, M., & Spaletta, G. 2005. Derivation of symmetric composition constants for symmetric integrators. *Optimization Methods and Software*, **20**(4-5), 597–613.
- Spera, M., Capuzzo Dolcetta, R., & Punzo, D. 2012 (July). *HiGPUs: Hermite's N-body integrator running on Graphic Processing Units*. Astrophysics Source Code Library, record ascl:1207.002.
- Springel, Volker. 2000 (Mar.). *GADGET-2: A Code for Cosmological Simulations of Structure Formation*. Astrophysics Source Code Library, record ascl:0003.001.

- Springel, Volker. 2010a. E pur si muove: Galilean-invariant cosmological hydrodynamical simulations on a moving mesh. *MNRAS*, **401**(2), 791–851.
- Springel, Volker. 2010b. Smoothed Particle Hydrodynamics in Astrophysics. *ARA&A*, **48**(Sept.), 391–430.
- Stone, James M., Gardiner, Thomas A., Teuben, Peter, Hawley, John F., & Simon, Jacob B. 2008. Athena: A New Code for Astrophysical MHD. *ApJS*, **178**(1), 137–177.
- Stone, James M., Gardiner, Thomas A., Teuben, Peter, Hawley, John F., & Simon, Jacob B. 2010 (Oct.). *Athena: Grid-based code for astrophysical magnetohydrodynamics (MHD)*. Astrophysics Source Code Library, record ascl:1010.014.
- Teyssier, R. 2002. Cosmological hydrodynamics with adaptive mesh refinement. A new high resolution code called RAMSES. *A&A*, **385**(Apr.), 337–364.
- Teyssier, Romain. 2010 (Nov.). *RAMSES: A new N-body and hydrodynamical code*. Astrophysics Source Code Library, record ascl:1011.007.
- Toonen, S., Nelemans, G., & Portegies Zwart, S. 2012. Supernova Type Ia progenitors from merging double white dwarfs. Using a new population synthesis model. *A&A*, **546**(Oct.), A70.
- Toonen, Silvia, Hamers, Adrian, & Portegies Zwart, Simon. 2016. The evolution of hierarchical triple star-systems. *Computational Astrophysics and Cosmology*, **3**(1), 6.
- Tóth, Gábor, & Keppens, Rony. 2012 (July). *VAC: Versatile Advection Code*. Astrophysics Source Code Library, record ascl:1207.003.
- van der Holst, Bar, Keppens, Rony, Meliani, Zakaria, Porth, Oliver, van Marle, Al-lard Jan, Delmont, Peter, & Xia, Chun. 2012 (Aug.). *MPI-AMRVAC: MPI-Adaptive Mesh Refinement-Versatile Advection Code*. Astrophysics Source Code Library, record ascl:1208.014.
- Vasiliev, Eugene. 2019. AGAMA: action-based galaxy modelling architecture. *MNRAS*, **482**(2), 1525–1544.
- Vaught, A. 1996. Graphing with Gnuplot and Xmgr: Two Graphing Packages Available Under Linux. *Linux J.*, **1996**(28es).
- Wang, Long, Spurzem, Rainer, Aarseth, Sverre, Nitadori, Keigo, Berczik, Peter, Kouwenhoven, M. B. N., & Naab, Thorsten. 2015. NBODY6++GPU: ready for the gravitational million-body problem. *MNRAS*, **450**(4), 4070–4080.
- Weinstein, Galina. 2012. Einstein’s 1912-1913 struggles with Gravitation Theory: Importance of Static Gravitational Fields Theory. *arXiv e-prints*, Feb., arXiv:1202.2791.
- Wilhelm, Maite J. C., & Portegies Zwart, S. 2024. VENICE: A multi-scale operator-splitting algorithm for multi-physics simulations. *A&A*, **691**(Nov.), A71.
- Williams, T., Kelley, C., & et al.,. 2013 (April). *Gnuplot 4.6: an interactive plotting program*. <http://gnuplot.sourceforge.net/>.
- Xia, C., Teunissen, J., El Mellah, I., Chané, E., & Keppens, R. 2018. MPI-AMRVAC 2.0 for Solar and Astrophysical Applications. *ApJS*, **234**(2), 30.
- Yoshida, Haruo. 1990. Construction of higher order symplectic integrators. *Physics Letters A*, **150**(5-7), 262–268.
- Zemp, Marcel. 2014 (July). *Halogen: Multimass spherical structure models for N-body*

- simulations*. Astrophysics Source Code Library, record ascl:1407.020.
- Zemp, Marcel, Moore, Ben, Stadel, Joachim, Carollo, C. Marcella, & Madau, Piero. 2008. Multimass spherical structure models for N-body simulations. *MNRAS*, **386**(3), 1543–1556.
- Zhao, Y., Zhang, Y., & Wang, J. 2021. Review of Bounding Box Algorithm Based on 3D Point Cloud. *International Journal of Advanced Network, Monitoring and Controls*, **6**(1), 20–27.

Appendix B

How to add a code to AMUSE

Jag har aldrig provat det förut, så jag tror absolut att jag borde kunna göra det.

I have never tried that before, so I think I should definitely be able to do that.

Pippi Långstrump (apocryphal)

The main part of this manuscript presents an extended description of how to use the AMUSE framework. Here we focus on the basic structure and design of AMUSE.

B.1 The AMUSE framework

At first sight, the approach of assimilating existing software into a larger framework would appear to be a difficult undertaking, particularly since community software is written in a wide variety of languages, such as FORTRAN, C, and C++, and exhibits an enormous diversity of internal units and data structures. On the other hand, such a framework, if properly designed, could be relatively easy to use, since learning one simulation package is much easier than mastering the idiosyncrasies of many separate programs. The use of standard calling procedures can enable even novice researchers to quickly become acquainted with the environment and to perform complicated simulations using it.

Within AMUSE, a user writes a relatively simple script with a standardized calling sequence to each of a set of underlying *community codes*. Instructions are communicated through a message-passing interface to any number of spawned community modules, which respond by performing operations and transferring data back through the interface to the user script. This Appendix is intended to help you write production quality (meaning working and clearly written) user scripts. There are numerous examples in the `amuse/examples` directory, and here we discuss several of them (see for example the code example in Chapters 4 and 5).

As illustrated in Figure B.1, the two top-level components of AMUSE are:

- The *user script*, which implements a specific physical problem or set of problems, in the form of system-provided or user-written scripts that serve as the

user interface to the AMUSE framework. Coupling between community codes is implemented at this level only, with the help of support classes provided by the *manager*.

- The *community module*, comprising three key elements:
 1. The *manager*, which provides an object-oriented interface onto the *communication* layer via a suite of system-provided utility functions. This layer handles unit conversion, as discussed below, and also contains the state engine and the associated data repository, both of which are needed to guarantee consistency of data across modules. The role of this layer is generic—it is not specific to any particular problem or to any single physical domain. In the rest of this manuscript we discuss an actual implementation of the *manager* for astrophysical problems and give some examples of coupling codes.
 2. The *communication* layer, which realizes the bi-directional communication between the *manager* and the community code layer. This is implemented via a proxy and an associated partner, which together provide the connection to the community code.
 3. The *community code* layer, which contains the actual community codes and implements control and data management operations on them. Each piece of code in this layer is domain-specific, although the code may be designed to be very general within its particular physical domain.

B.1.1 The community module

The community module consists of the actual community code and the bi-directional communication layer enabling low-level MPI (Gropp *et al.*, 1996) communication between the proxy and the partner. Each community module contains a Python class of functions dedicated to communication with the community code. Direct use of this low-level class from within a user script is possible but discouraged because it requires considerable replication of code dealing with data models, unit conversion, and exception handling. Instead, the AMUSE framework provides the manager layer on top of the community module layer to manage the bookkeeping needed to maintain the interface with the low-level class.

B.1.1.1 The community code

Community applications may be programmed in any computer language that supports smart sockets or bindings to the MPI protocol. In practice, these codes exhibit a wide variety of structural properties, model diverse physical domains, and span a broad range of temporal and spatial scales. In AMUSE this diversity is exemplified by a suite of community programs ranging from toy codes to production-quality high-precision dedicated solvers. A user script can be developed quickly using toy codes until the

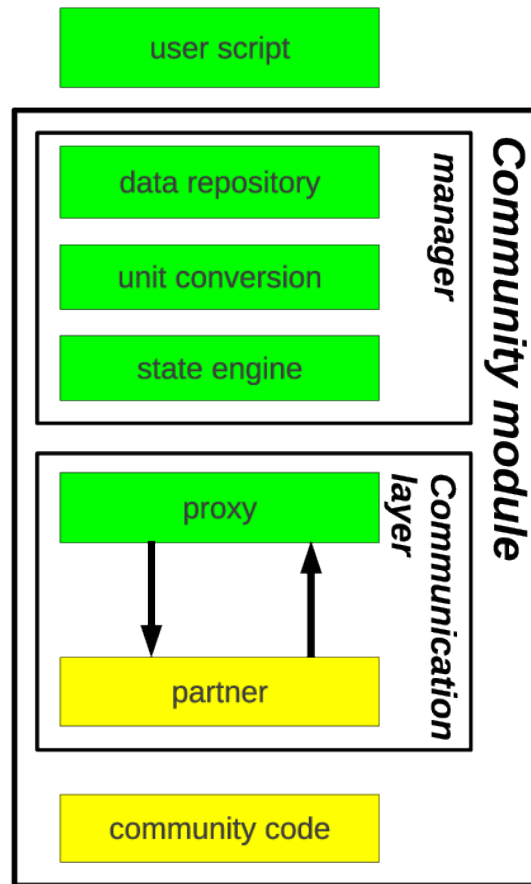


Figure B.1: The AMUSE framework architecture is based on a three-layered design. The top layer consists of a user script written by the end-user in Python (top box). The *manager* is part of the *community module* and consists of a data repository, unit conversion, and a state engine. The *community module* also includes the partner-proxy pair in the communication layer—a bi-directional message passing framework—and a community code (bottom box).

production and data analysis pipelines are fully developed, then easily switched to production quality implementations of the physical solvers for actual large-scale simulations.

In principle, each of the community modules can be used standalone, but the main strength of AMUSE is in the coupling between them. Many of the codes incorporated in our practical implementation were not written by us, but are publicly available and are amply described in the literature.

B.1.1.2 The communication layer

Bi-directional communication between a running community process and the manager could in principle be realized using any protocol for remote interprocess communication. Our main reasons for choosing MPI are similar to those for adopting Python—widespread acceptance in the computational science community and broad support by many programming languages and on most computing platforms.

The communication layer consists of two main parts, a Python proxy on the manager side of the framework and a partner on the community code side. The proxy converts Python commands into MPI messages, transmits them to the partner, then waits for a response. The partner waits for and decodes MPI messages from the proxy into community code commands, executes them, then sends a reply. The MPI interface allows the user script and community modules to operate asynchronously and independently. As a practical matter, the detailed coding, decoding, and communications operations in the proxy and the partner are never hand-coded. Rather, they are automatically generated as part of the AMUSE build process, from a high-level Python description of the community module interface.

We note that other solutions for linking high-level languages within Python exist—e.g. `swig` (see <http://www.swig.org>) and `f2py` (Pearu, 2009), and in fact both were used in earlier implementations of the AMUSE framework (Portegies Zwart *et al.*, 2009). However, despite their generality, these solutions cannot maintain namespace independence between community modules, and in addition are incapable of accommodating high-performance parallel community codes. For these reasons, we abandoned the standard solutions in favor of our custom, high-performance MPI alternative.

The use of MPI may seem like overkill in the case of serial operation on a single computing node (and this might well be the case), but it imposes negligible overhead, and even here it offers significant practical advantages. It rigorously separates all community processes in memory, guaranteeing that multiple instances of a community module are independent, and explicitly avoids namespace conflicts. It also ensures that the framework remains thread-safe when using older community codes that may not have been written with that concept in mind.

Despite the initial threshold for its implementation and use, the relative simplicity of the message-passing framework allows us to easily couple multiple independent community codes as community modules, or multiple copies of the same community module, if desired (see Fig. B.1). An example of the latter might be to simulate simultaneously two separate objects of the same type, or the same physics on different scales, with different boundary conditions, without having to modify the data structures in

the community code. Specifically, we might want to evolve two stars of the same mass but with different metallicities.

As a bonus, the framework naturally accommodates inherently parallel community modules and allows simultaneous execution of independent modules from the user script. Individual community modules can be offloaded to other processors, in a cluster or on the grid, and can be run concurrently without requiring any changes in the user script, which still looks like a simple serial code. This makes the AMUSE framework well suited for distributed computing with a wide diversity of hardware architectures.

B.1.1.3 The manager

Between the user script and the communication layer we introduce the manager, part of the community module. This part of the code is visible to the script-writing user and guarantees that all accessible data are up to date, have the proper format, and are expressed in the correct units.

The manager layer constructs data models (such as particle sets or tessellated grids) on top of all functions in the low-level communication class and checks the error codes they return. To allow different modules to work in their preferred systems of units, the manager also incorporates a unit conversion protocol (see Appendix B.1.3). It also includes a state engine to ensure that codes are not in the middle of other activities when they hand control back to the framework, as many community codes are written with specific assumptions about calling procedures. The state engine is important, for example in adding particle to a tree code: for performance, it is much better to delay rebuilding the tree until all particles have been added rather than rebuilding the tree every time a particle is added. This repository can be structured in one or more native formats, such as particles, grids, tessellation, etc, depending on the topology of the data adopted in the community code.

Each community code has its own internal data, needed for modeling operations but not routinely exposed to the user script. However, some of these data may be needed by other parts of the framework. To share data effectively and to minimize the bookkeeping for data coherency, the most fundamental parts of the community data are replicated as a structured repository in the manager. The internal data structures differ, but the repository imposes a standard format. It allows the user to access community module data from the user script without having to make individual (and often idiosyncratic) calls to individual community modules. The repository is updated from the community module on demand and is considered to be authoritative within the script.

The separation of the manager from the community module realizes a flexibility that allows the user to swap modules and recompute the same problem using different physics solvers, providing an independent check of the implementation, validation of the model, and verification of the results, simply by rerunning the same realization of the initial conditions using another community module. Potentially even more interesting is the possibility of solving some parts of a problem with one solver and others with a different solver of the same type, then combining the results at a later stage to complete the solution. While studying the general behavior of a simulation,

perhaps to test a hypothesis or guide our physical intuition, we may elect to use computationally cheaper physics solvers, then switch to more robust, but computationally more expensive, modules in a production run. Alternatively, we might adopt less accurate solvers for physical domains that are not deemed crucial for capturing the correct overall behavior.

A further advantage of separating the manager from the community module is the possibility of combining existing community modules into new community modules which can themselves be coupled via the manager, building in this way a hierarchical environment for simulating complex physical systems with multiple components.

B.1.2 The user script

The AMUSE framework is controlled by the user via Python scripts. The main tasks of these scripts are to identify and spawn community modules, control their calling sequences and manage the data flow among them. The user script and the spawned community code are connected by a communication channel embedded in the community module and maintained by the manager. Since the community modules do not communicate directly with one another, the only communication occurs between modules and the user script. The number of community modules controlled by a user script is effectively unlimited, and multiple instances of identical modules can be spawned.

The main objective of the user script is to read in or generate an input model, process these initial conditions through one or more simulation modules, and subsequently mine and analyze the resultant data. In practice, the user script will itself be composed of a series of scripts, each performing one specific functionality.

B.1.3 Inter-module data transfer and unit conversion

Computer scientists tend to attribute types to parameters and variables, and generally impose strict rules for the conversion from one type to another. In physics, data types are almost never important (everything is real or integer), but units are crucial for validating and checking the consistency of a calculation or simulation. The lack of coherent unit handling in programming environments is notorious, and can cause significant problems in multi-physics simulations, particularly for inexperienced researchers.¹

In many cases, a researcher can (and arguably should) define a set of dimensionless variables applicable to a specific simulation, within the confines of a set of rather strict assumptions. For a mono-physics simulation, the consistent use of such variables is generally clear to most users. However, as soon as an expert from another field attempts to interpret the results, or when the output of one simulation is used by another program, units must be restored and the data converted to another physical system for further processing and/or interpretation.

¹The Mars Climate Orbiter crashed on 11 December 1998 on the red planet due to a miscalculation during Trajectory Correction Maneuver 4 in the conversion of imperial unit pound-seconds adopted by NASA to the metric units of newton-seconds used by Lockheed.

Table B.1: Dependencies needed to compile and run AMUSE.

package name	minimum version number	note
Python	3.7	Scripting programming language.
HDF5	1.6.5	Little/big endian neutral binary formatted I/O.
MPI		Message passing interface.
FFTW	3.0	Subroutine library for discrete Fourier transforms.
GSL		Scientific Library for C and C++ programmers.
CMake	2.4	Cross-platform, open-source build system.
GMP	4.2.1	Library for arbitrary precision arithmetic.
MPFR	2.3.1	C Library for multiple-precision floating-point computations.
Numpy	1.18.0	Numerical toolbox for Python.
h5py	3.0	HDF5 interface for Python.
mpi4py	3.0	Python interface for MPI.
pytest	3.0	Extension of the Python testing framework.
docutils	0.18	Documentation support for Python packages.

The absence of units in scientific software raises two important problems. The first is the loss of an independent consistency check for theoretical calculations. The second is the expert knowledge and intuition required to manage dimensionless variables or otherwise unfamiliar units. Few would recognize 4.303×10^{43} as the radius of the Sun in units of the Planck length, even though it is not completely inconceivable that an astronomer might use such units in a cosmological simulation. To address these issues, AMUSE incorporates a unit-conversion module as part of the manager (see Appendix B.1.1.3) to guarantee that all communications within the top-level user script are performed in the proper units. In order to prevent unit checking and conversion from becoming a performance bottleneck, we adopt “lazy” evaluation, performing unit conversion only when explicitly required.

B.1.4 Installing prerequisites and debugging

The non-Python packages on which AMUSE depends are listed in Table B.1. These dependencies may differ in later releases, so to be sure that you install the correct packages it is best to consult the AMUSE web site <https://www.amusecode.org>. Many of these dependencies may already be installed in recent Linux and macOS releases, but some fine-tuning may be needed to complete the dependency installation.

The Message Passing Interface (or MPI for short) is an important package, although the same functionality can be realized using sockets (Gay, 2000). We recommend using either sockets, OpenMPI (Dagum & Menon, 1998) or MPICH2 (Gropp, 2002). The former has the advantage that the communication does not require a process when running, which saves computer time for other tasks, but the latter is more standard, usually faster and you will then have an MPI running for other parallel tasks.

When running on a supercomputer it is probably best to use the native `numpy`

(van der Walt *et al.*, 2011) and the MPI interface specific to the architecture rather than installing your own, but for all other packages this should not make a difference. When installing AMUSE on multiple machines in order to use distributed operation (using `ibis`—see Appendix B.1.6; Bal *et al.*, 2010; Drost *et al.*, 2012), it is important to ensure that each of the machines runs the same version of Java (Gosling *et al.*, 1996, 2014). Otherwise, the distributed interface will become confused and your scripts will fail with unpleasant distributed error messages, which are hard to debug.

B.1.5 Reproducibility with git

A nice feature of `git` is that older versions are guaranteed to exist, and can be downloaded and installed if needed. In this way, we can, for example, ensure that the version of AMUSE we are using is identical to the version used to write this book. To check the version number you can use the command

```
>% git rev-parse HEAD
```

which, on the day we submitted the manuscript to the publisher, gave

```
8d81c91a9fbd55b7510b3949700d3a594e5cc179
```

or version v2025.5.0.rc1 from May 30, 2025. This is the unique key for the particular version of AMUSE under which we tested many of the scripts in the book. If you wish to use exactly the same version of `amuse`, you can download and install it, using the bash script presented in Code example B.1. The script automatically downloads and starts the script that will guide you through installing the AMUSE package.

```
1 #!/bin/bash
2
3 GITHASH=d77c113afd71f8063652a783276807f2d629b396
4 temp=amuse-temp-$$
5 mkdir $temp; cd $temp
6
7 git clone https://github.com/amusecode/amuse.git
8 cd amuse; git reset --hard $GITHASH; cd ../..
9
10 export AMUSE_DIR="$PWD/amuse-$GITHASH"
11 mv $temp/amuse $AMUSE_DIR
12 rm -rf $temp
13
14 cd $AMUSE_DIR
15 ./configure && make framework
```

Code example B.1: Automated script to download a specific version of AMUSE. The script is available via `wget https://amusecode.org/get_amuse_and_install.sh`.

Note, by the way, that we test AMUSE frequently to ensure that our scripts and test suites run as expected, so there is no actual reason to use only this specific version. However, for purposes of code verification it is often important to be able to specify precisely the version of the software that was used to obtain a specific result, so this feature of `git` is key to writing reproducible code.

B.1.6 Tuning your AMUSE installation

The operation of AMUSE can be controlled and tuned using environment variables. These are generally stored in your home directory in the file `.amuserc`. These settings are superseded by the same named file in the directory from which you launch your AMUSE script.

The file `.amuserc` can contain a number of directives. These include the choice between `MPI`, `sockets`, or `ibis`. `MPI` is the default; to use `sockets` as your main communication channel with the following lines in your `.amuserc` file.

```
[channel]
channel_type=sockets
```

The `ibis` channel allows the user to run `amuse` in distributed mode using the `ibis` package.

Another option is to redirect the output of any of the AMUSE modules you may be running. The native output of the community codes is usually discarded, this would be reflected by

```
redirection=null
```

but by writing

```
redirection=none
```

it is redirected to `stdout`. An alternative would be to write `file` instead of `null` or `none`. This latter option may produce a number of files, depending on the codes you run. The two lines

```
redirect_stdout_file=code.out
redirect_stderr_file=code.out
```

will redirect the standard output and error channels `stdout` and `stderr`.

A debugger, in this example the GNU debugger `gdb` (see [Robbins, 2005](#)) may be enabled by adding the line

```
debugger=gdb
```

to the `.amuserc` file. The standard setting is `debugger=none`, indicating that no debugger is to be used. Alternatives are `xterm` for debugging with an `xterm` screen, `ddd` for a `ddd` session using a graphical debugger, or `valgrind` ([Nethercote & Seward, 2007](#)) to run the community codes under `valgrind` to check for memory leaks. We present more detail on debugging options in Appendix [B.1.7](#).

These options can also be provided as part of the argument list for the various modules at run time. For example, one could start a *N*-body code `ph4` with the following arguments

```
gravity = Ph4(redirection="file", number_of_workers=16, mode="gpu",
              debugger="gdb")
```

to redirect the code output to the standard files identified in the original source code, run the code concurrently on 16 cores, each equipped with a GPU, and run the debugger in order to track numerical problems.

Several more options can be specified in the `.amuserc` file, but these are less relevant for the day-to-day operation of the package. An example of an `.amuserc` file is provided in the installation directory in your AMUSE repository.

B.1.7 Debugging in AMUSE

Debugging a monophysics solver is sometimes difficult, but debugging an AMUSE script when the problem arises in one of the coupled codes can be much more challenging. With any luck, you will never have to do this, because we hope that the authors of the individual community codes will continue to maintain their packages, but you never know.

There are several ways to debug a community code. Most basic is to set the code's internal "verbosity" level to a high value. Almost all codes have something like this, but there is absolutely no consensus on what value that should be because every author has a private method to do this. Since the community codes in AMUSE are (almost) virgin, meaning that we have touch them as little as possible while integrating them into the framework, the private verbosity setting are unchanged. However, apparently due to some unwritten understanding, setting most codes' verbosity to a large positive integer value leads to an immense flow of information from the running code. This is illustrated in the example below.

```
self.hydro = Fi(converter, redirection="none")
self.hydro.parameters.verbosity = 99
```

Combining high verbosity with the `redirection` argument (indicating that the code's standard output should be written to your terminal) and running the code again generally produces enough (probably too much) information to debug obvious bugs.

A more systematic way is to start an online debugger. although this generally requires significant knowledge of the community code. In the following snippet we start the GNU debugger `gdb` by providing it as an argument in the code (see also Appendix B.1.6)

```
hydro = Fi(converter, debugger="gdb")
```

You can also specify these arguments in the `.amuserc` file in your root directory, or the directory from which you run the AMUSE script (see Appendix B.1.6). There are several options, as listed in Tab. B.2.

B.2 Other associated codes and packages

Many people to contributed AMUSE. They helped with development, maintenance, expansion, bug fixes, and provided interesting examples. Without the community, AMUSE would only be a shadow of its current form. Here we discuss a few of the

Table B.2: Debugging directives.

method	explanation
none	no debugger, normal run
gdb	start a gdb debugger in a separate xterm
xterm	start the code under xterm, ideal for console logging
ddd	start a ddd session (a graphical debugger)
valgrind	check for memory leaks at runtime (Nethercote & Seward, 2007)
gdb-remote	run gdb as a server that listens on a port
For remote debugging specify a port to listen to	
debugger	none
debugger	gdb-remote
debugger_port	4343

more prominent packages, not as big as **PeTar**, **Torch**, **Ekster**, or **Venice**, but still important. Many of these packages are still in development—honestly, all software is continuously in development. But they have been used in scientific production.

B.2.1 TRES: Triple-star Evolution

The Triple-star Evolution Code (**Tres**) ([Toonen *et al.*, 2012, 2016, 2020](#)) is a numerical simulation framework designed to model the evolution of hierarchical triple systems, including both stellar and planetary components. **Tres** combines stellar evolution (such as changes in mass, radius, and luminosity) using the **SeBa** package with secular three-body orbital dynamics ([Hamers & Portegies Zwart, 2016](#)), allowing for mutual influences between the stars’ evolution and their orbital interactions.

In **Tres** three stars with orbits are integrated synchronously. The stellar evolution is realized with **SeBa**, as well as the mass-transfer recipes between two stars. The tertiary orbit, and its effect on the inner binary is integrated using the secular dynamics code. In addition, **Tres** accounts for the tidal evolution of the three stars, which is important because of von Zeipel-Lidov-Kozai cycles ([von Zeipel, 1910](#); [Lidov, 1962](#); [Kozai, 1962](#)) can render the triple highly eccentric.

Tres is particularly useful for population synthesis studies, enabling researchers to explore the evolutionary pathways and interaction rates in large samples of triple systems, as as presented in [Toonen *et al.* \(2020\)](#).

Code example [B.2](#) shows a small example script on how to set-up a simulation using **Tres**. This is considerably more elaborate, mainly because now we have three stars and two orbits to initialize.

B.2.2 AGAMA: Action-based Galaxy Modelling Architecture

AGAMA (Action-based Galaxy Modelling Architecture) is an open source toolkit for generating realistic N -body representation of galaxies ([Vasiliev, 2019](#)); see also [Vasiliev \(2018\)](#). It offers a suite of computational methods for galactic modeling and analy-

sis. It was developed since 2015 and is documented in over 130 pages of technical specifications.

AGAMA is a comprehensive software library for stellar dynamics and galactic modeling. Its core functionality centers on the computation of gravitational potentials and forces, which can be derived from analytic density profiles or N-body simulations using spherical-harmonic expansions and adaptive solvers. AGAMA supports the numerical integration of stellar orbits in both static and time-dependent potentials, accommodating composite systems such as interacting galaxies.

```

9 from tres import run_tres, initialize_triple_class
10 from tres.setup import make_particle_sets
11
12
13 inner_primary_mass = 10.0 | units.MSun
14 inner_secondary_mass = 8.0 | units.MSun
15 outer_mass = 5 | units.MSun
16 inner_semimajor_axis = 1.0 | units.au
17 outer_semimajor_axis = 12.0 | units.au
18 inner_eccentricity = 0.5
19 outer_eccentricity = 0.5
20 relative_inclination = np.pi / 3
21 inner_argument_of_pericenter = 0.0
22 outer_argument_of_pericenter = 0.0
23 inner_longitude_of_ascending_node = 0.0
24
25 stars, bins, correct_params = make_particle_sets(
26     inner_primary_mass,
27     inner_secondary_mass,
28     outer_mass,
29     inner_semimajor_axis,
30     outer_semimajor_axis,
31     inner_eccentricity,
32     outer_eccentricity,
33     relative_inclination,
34     inner_argument_of_pericenter,
35     outer_argument_of_pericenter,
36     inner_longitude_of_ascending_node,
37 )
38
39 stellar_code = Seba()
40 secular_code = Seculartriple()
41
42
43 triple = initialize_triple_class(
44     stars, bins, correct_params, stellar_code, secular_code
45 )
46
47 triple.stellar_code.parameters.metallicity = 0.02
48 triple.evolve_model(10 | units.Myr)
49 triple.print_stellar_system()

```

Code example B.2: Minimal example to set up a simulation using Tres (see `amuse.examples.tres_minimal`.)

The library provides robust tools for coordinate transformations, enabling conversion between position-velocity and action-angle variables via the Stäckel approximation or torus mapping techniques. AGAMA implements a range of action-based distribution functions for disk and spheroidal systems, facilitating predictions of velocity distributions and the construction of self-consistent, multi-component galaxy models.

Scientifically, AGAMA is suited for galactic structure reconstruction using Schwarzschild orbit-superposition methods, dark matter halo profiling from stellar kinematics, equilibrium initial condition generation for N-body simulations, and the analysis of tidal streams and dynamical populations. The framework also supports parameter optimization against observational data.

Technically, AGAMA is written in C++ with performance optimizations using OpenMP parallelization. It offers a native Python API for interactive analysis and is interoperable with other astrophysical software ecosystems such as AMUSE, GALPY, and NEMO. The package is modular, allowing users to extend its capabilities with custom potential models and distribution functions. Included are advanced mathematical utilities for spline interpolation and multidimensional sampling, as well as extensive test suites and example workflows.

B.2.3 Vader: Viscous Accretion Disk Evolution Resource

We implemented an interface to the VADER viscous evolution code (Krumholz & Forbes, 2015) within the AMUSE framework. Our interface supports external photoevaporation using mass loss rates from the FRIED grid (Appendix B.2.4), and we extended the code to include internal photoevaporation (IPE) with mass loss profiles from Picogna *et al.* (2019), scaled by stellar mass using the relations in Owen *et al.* (2012). Stellar X-ray luminosities, which drive IPE rates, are based on Flaccomio *et al.* (2012) and decrease over time as given by Eq. B.4.

To model accretion onto the central star, we apply a time-dependent accretion rate following viscous disc theory (Lynden-Bell & Pringle, 1974):

$$\dot{M}_{\text{accr}}(t) = \frac{M_d}{2t_s} \left(1 + \frac{t}{t_s}\right)^{-3/2}, \quad (\text{B.1})$$

with viscous timescale

$$t_s = \frac{R_d^2}{3} \frac{\Omega}{\alpha c_s^2}, \quad (\text{B.2})$$

which we approximate using the models temperature profile and ideal gas law:

$$t_s = 11 \text{ Myr} \left(\frac{\alpha}{10^{-3}}\right)^{-1} \left(\frac{R_d}{200 \text{ au}}\right) \left(\frac{M_*}{M_\odot}\right)^{1/4}. \quad (\text{B.3})$$

Rather than explicitly evaluating Eq. B.1, we scale the input accretion rate by $(1 + t/t_s)^{-3/2}$.

Boundary conditions are handled dynamically: when the inner mass flux is insufficient to sustain the prescribed accretion, we first attempt to reduce the accretion rate to empty the innermost cell; if this fails, we switch to a vanishing-torque condition.

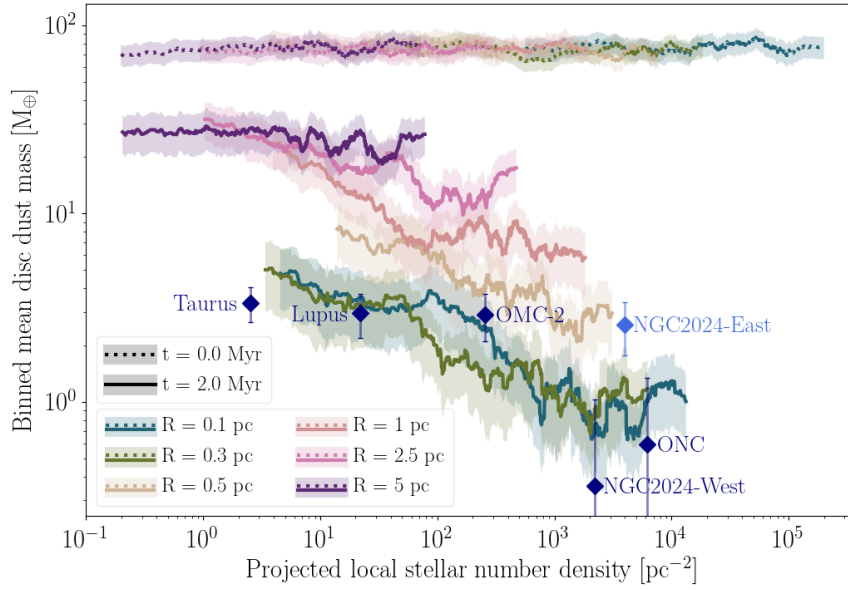


Figure B.2: Binned mean disc dust mass versus local stellar density, projected in 2D. Mean values use bins of 100 stars. Observational comparisons are included from Lupus, ONC, OMC-2, Taurus, and NGC 2024 (see [Concha-Ramírez et al., 2021](#); for details)

External photoevaporation (EPE) rates are fixed per model and scaled linearly with disc radius, with optional interpolation from the FRIED grid ([Haworth et al., 2018](#)) under a fixed 10 G_0 radiation field. Dust is evolved as a passive scalar, initially 1% of the gas mass, and subject to EPE following [Haworth et al. \(2018\)](#) with an exponential decay representing grain growth.

The implementation uses a logarithmic grid of 330 radial zones from 0.01 to 3000 au and assumes a turbulent viscosity parameter of $\alpha = 10^{-3}$.

B.2.4 FRIED: Far-ultraviolet Radiation Induced Evaporation of Discs

External photoevaporation (EPE) occurs when nearby stellar radiation heats disc material until it becomes unbound, driving a thermal wind. Far-ultraviolet radiation usually dominates, with extreme ultraviolet important near massive stars.

The FRIED grid ([Haworth et al., 2018](#)) provides EPE mass loss rates across host star mass, radiation field, disc radius, and mass. [Coleman & Haworth \(2020\)](#) used a simplified scaling due to the high minimum radiation field in FRIED, which yields mass loss rates too large for Peter Pan discs around M dwarfs.

Internal photoevaporation (IPE), driven by the host stars radiation, shows mass loss nearly independent of stellar mass but strongly dependent on X-ray luminosity ([Owen et al., 2012](#); [Picogna et al., 2019](#)). X-ray luminosity scales with stellar mass ([Flaccomio et al., 2012](#)), giving a mass loss scaling exponent $\beta \approx 1.87$. X-ray luminosity decreases by an order of magnitude from 1 to 110 Myr, approximately as $L_X \propto t^{-2/5}$ after

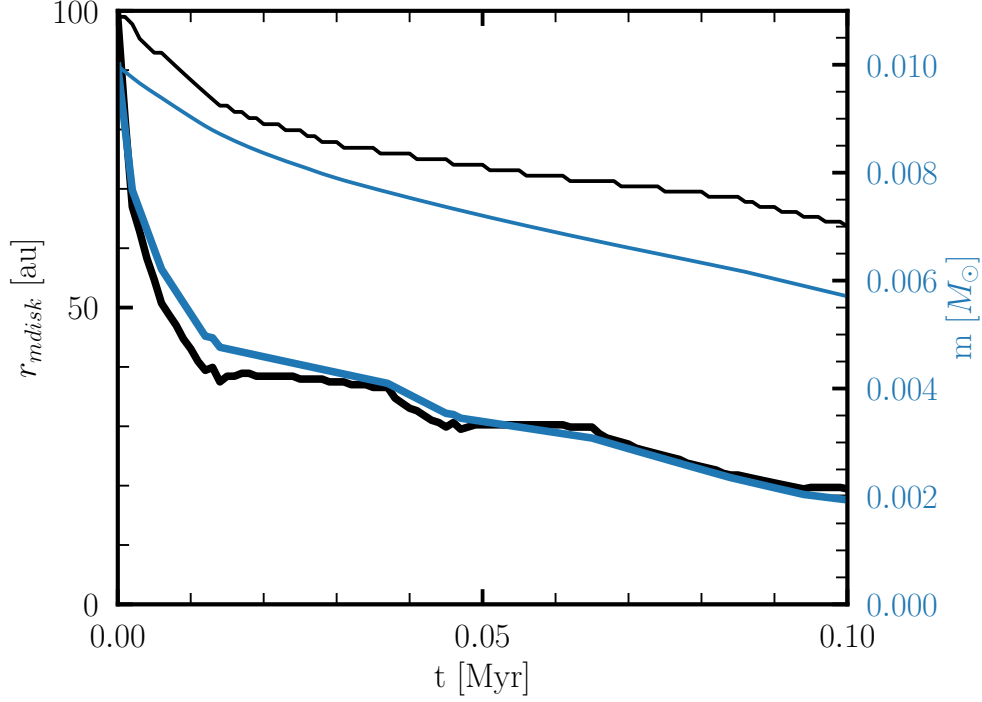


Figure B.3: Demonstration of how SeBa, Vader and the Fried grid can lead to interesting results in terms of the external photoevaporation of a circumstellar disk due to a nearby massive star. The blue curves adopt the disk mass as a function of time (right-hand axis scale), and the black curve gives the disk radius (left). A script to generate this figure can be found at `amuse.examples.ppd_minimal`.

radiative core formation (Johnstone *et al.*, 2020). We adopt:

$$L_X(t) = \begin{cases} L_{X,0} & t < 1 \text{ Myr} \\ L_{X,0} \left(\frac{t}{1 \text{ Myr}} \right)^{-2/5} & t > 1 \text{ Myr}. \end{cases} \quad (\text{B.4})$$

Here $L_{X,0}$ was adopted from Flaccomio *et al.* (2012).

In Figure B.3 we present an example of how the FRIED grid can be used in combination with SeBa and Vader. The thick curves are for a $16 M_\odot$ star irradiating a $0.01 M_\odot$ disk around a $1 M_\odot$ star at a distance of 0.1 pc. The thin curves are calculated for the same star but then at a distance of 1 pc.

B.2.5 PACE: Planet ACcretion and Evolution

Planet growth in our model proceeds via core accretion (Perri & Cameron, 1974), beginning with the formation of a solid core that grows by accreting pebbles. Pebble accretion (Ormel *et al.*, 2015) is efficient for Moon- to Ceres-sized embryos and is described by

$$\dot{M}_{\text{peb}} = \varepsilon 2\pi r u_{\text{peb}} \Sigma_{\text{peb}}, \quad (\text{B.5})$$

where ε is the pebble accretion efficiency and u_{peb} the pebble drift velocity, both of which depend on the Stokes number St . These are calculated in a mass-weighted

fashion from the two-population dust model:

$$\text{St} = (1 - f_m)\text{St}_0 + f_m\text{St}_1, \quad u_{\text{peb}} = (1 - f_m)v_0 + f_mv_1. \quad (\text{B.6})$$

Accretion ceases when the planet reaches the pebble isolation mass (Bitsch *et al.*, 2018), at which point it creates a pressure bump that halts pebble inflow:

$$M_{\text{iso}} = 25M_{\oplus} \left(\frac{h}{0.05} \right)^3 \left[0.34 \left(\frac{-3}{\log_{10} \alpha_t} \right)^4 + 0.66 \right] \times \left[1 - \frac{\partial \ln P / \partial \ln r + 2.5}{6} \right] \frac{M_{\star}}{M_{\odot}}. \quad (\text{B.7})$$

Gas accretion onto the core then follows, initially regulated by Kelvin–Helmholtz contraction (Ikoma *et al.*, 2000):

$$\dot{M}_{\text{KH}} = 10^{-5} \left(\frac{M_{\text{p}}}{M_{\oplus}} \right)^4 \left(\frac{\kappa}{0.1 \text{ m}^2 \text{ kg}^{-1}} \right)^{-1} M_{\oplus} \text{ yr}^{-1}, \quad (\text{B.8})$$

but may later be limited by gas inflow across the planetary Hill sphere (Tanigawa & Tanaka, 2016):

$$\dot{M}_{\text{Hill}} = \frac{0.29}{3\pi h^4} \left(\frac{M_{\text{p}}}{M_{\star}} \right)^{4/3} \frac{\dot{M}_{\text{g}}}{\alpha_{\text{acc}}} \frac{\Sigma_{\text{gap}}}{\Sigma_{\text{g}}}, \quad (\text{B.9})$$

with gap depth factor

$$K = \left(\frac{M_{\text{p}}}{M_{\star}} \right)^2 h^{-5} \alpha_t^{-1}. \quad (\text{B.10})$$

The final gas accretion rate is taken as the minimum of all limiting rates:

$$\dot{M}_{\text{GA}} = \min [\dot{M}_{\text{KH}}, \dot{M}_{\text{Hill}}, \dot{M}_{\text{g}}]. \quad (\text{B.11})$$

Figure B.4 illustrates the diversity observed in planetary systems from two runs using PACE, one without dynamics, and one in which we took the dynamical evolution of the stars and planetary systems into account. In the latter run, we also incorporated the FRIED grid Appendix B.2.4 to affect the evolution of circumstellar disks using *vader* Appendix B.2.3. In Figure B.4, we include the Solar system for comparison: the orbital parameters range over more than 4 orders of magnitude, provoking many questions about their formation, evolution, the role of birth environment and later isolated secular evolution. So far, research has focused on the isolated formation and evolution of planetary systems, but it has become clear that stars form clustered and that this clustering has a long-term effect on the evolution of planetary systems.

B.2.6 Packages not yet used in scientific production

Several packages are being developed as this book goes to print. Some of them have an operational interface, but are not yet tested in production simulations. The experience has taught us that software not used for producing scientific results is buggy, at best.

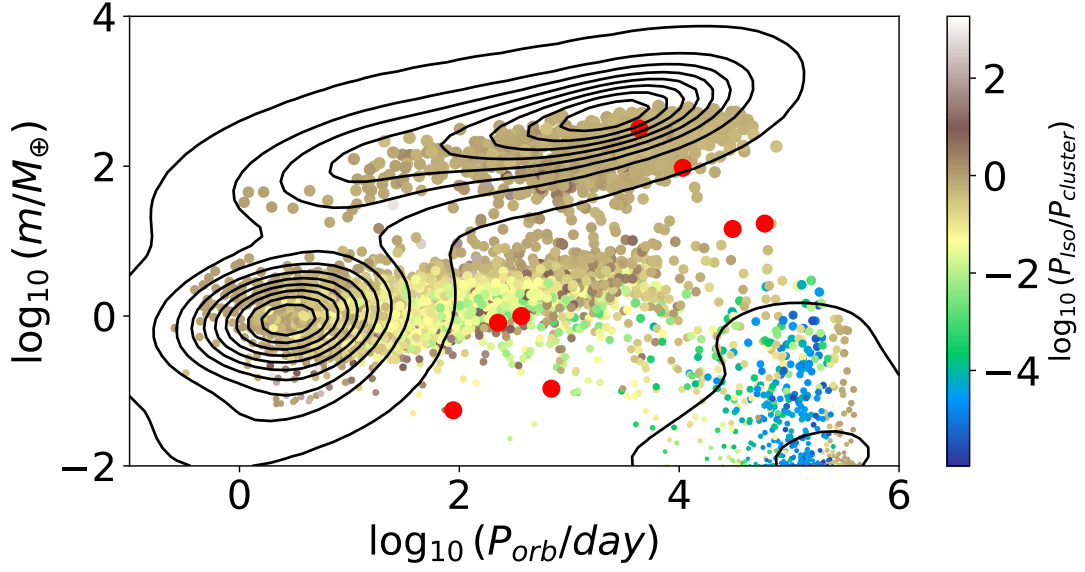


Figure B.4: Simulated planet population for isolated stars (contours) and for stars in a cluster (bullets) [Huang et al. \(2024\)](#). The initial conditions are identical in both cases, but the planetary systems in a clustered environment are subject to external photo-evaporation onto the planet-forming disk. The colors indicate the logarithmic change in the planetary system's orbital period from isolation to perturbed.

We therefore do not necessarily recommend using these packages, but by the time this book is available some of them may have a working operational interface and will have been tested in practice.

They may also inspire the reader to start another project, and contribute to the framework. The more people contribute, the better and more useful the software becomes.

B.2.7 Solar system initial conditions

As part of a Masters project, Nadia Kempers developed a collection of routines to generate a realization of the Solar system as we know it, based on available ephemerides from NASA and JPL. The scripts generates a current Solar system particle set, allowing us to integrate it in a larger environment, or to study the orbital evolution of specific objects.

It enables to selection of specific objects, planets, moons, planetesimals, comets, etc. Although, there are a number of hidden assumptions, such as the mass of objects, or their sizes, where observational data is unavailable, it is, so far, the most complete solar system orbital generator we have seen.

B.2.8 The interface to interstellar non-equilibrium chemistry

As part of a Masters project, Jelmer Stroo developed an interface to the UCLCHEM. UCLCHEM ([Holdship et al., 2017](#); [Keil et al., 2022](#)) is a gas-grain chemical code

designed for astrochemical modeling across diverse environments, such as molecular clouds, protostellar cores, and shocks. Unlike many codes that focus solely on gas-phase reactions, UCLCHEM incorporates comprehensive gas-grain processes, including freeze-out, non-thermal and thermal desorption, and grain surface reactions

UCLCHEM models both gas-phase and grain-surface reactions, treating molecules on grains as distinct species from their gas-phase counterparts. Non-thermal desorption processes include molecule release due to H₂ formation, cosmic ray interactions, and UV irradiation. Thermal desorption is uniquely modeled as multiple events, reflecting laboratory findings, rather than a single peak as in other codes. The code propagates species abundances through a user-defined network of reactions, adapting to the physical conditions of the modeled environment.

Written in Fortran 90, UCLCHEM utilizes the DVODE algorithm to solve the resulting system of rate equations. The code is optimized for high-density, low-temperature environments (e.g., protostellar cores and shocks), and is less suitable for hot, low-density conditions where gas-grain interactions are minimal.

When used within AMUSE, it is essential to ensure that simulations remain within UCLCHEM's intended parameter space to maintain accuracy.

Although not yet used in production, we envision the use of UCLCHEM for protostellar core chemistry, shock and jet-driven desorption, chemistry in Seyfert galaxies, prebiotic molecule formation, effects of cosmic rays in dense regions, and machine learning emulators for astrochemical networks.

While this proof-of-concept simulation has notable limitations, it demonstrates the significant potential of integrating UCLCHEM with AMUSE for advanced scientific studies. The current implementation offers several features that can support more sophisticated simulations.

In hydrodynamical simulations, sink particles are employed to represent high-density regions that cannot be resolved at the simulation's spatial resolution (Bate *et al.*, 1995). These are modeled as point masses that interact gravitationally but not hydrodynamically, and can accrete surrounding material. AMUSE facilitates the creation, management, and tracking of sink particles, including the chemical abundances of accreted material, which is valuable for core-scale modeling.

The choice of hydrodynamics code within AMUSE influences the physical processes included in a simulation. For example, the SPH code `Fi` is straightforward to use but lacks magnetic fields, turbulence, and molecular cooling. In contrast, codes such as Athena2 (AMR: Stone *et al.*, 2008, 2020) and Phantom (SPH: Price *et al.*, 2018) support magnetohydrodynamics, with Phantom also incorporating a basic chemical network for molecular cooling, an important process in molecular clouds.

AMUSE's multiphysics framework allows for the integration of additional physical processes. This includes *N*-body codes for simulating planetary embryos, coupling chemical codes with stellar evolution models to study stellar winds, or combining stellar evolution, hydrodynamics, and radiative transfer to investigate radiative feedback in the interstellar medium. While increasing the complexity and computational cost, AMUSE's modular design streamlines the coupling of diverse codes, now including astrochemistry, enabling comprehensive and flexible astrophysical simulations.

In Figure B.5 we show the result of a simulated molecular cloud on the verge of

collapsing and making the first stars. The setup for the on-the-fly simulation is similar to that in [Priestley et al. \(2023a\)](#): two spherical molecular clouds, each with mass $M = 10^4 M_\odot$, radius of $r = 19$ pc, and a uniform number density of $n_H = 10 \text{ cm}^{-3}$, are initialized on either side of the x-axis with their centers at $\pm r$, so that the clouds' edges touch at $x = 0$. The clouds are then each given a bulk motion of $v = \pm 7 \text{ km/s}$ to cause them to collide. The initial gas temperature is set at $T = 300 \text{ K}$. The SPH code `fi` ([Pelupessy et al., 2004](#)) is used to simulate the hydrodynamics of the molecular gas. Each molecular cloud consists of 100,000 particles, for a mass resolution of $0.1 M_\odot$ per particle. Time steps of $4.4 \times 10^3 \text{ yr}$ are used between steps of the hydrodynamical and chemical codes, and the simulation is stopped at a time of $t_{\text{end}} = 5 \text{ Myr}$.

As in [Priestley et al. \(2023a\)](#), the UV radiation field is set at 1.7 times the Habing (1968) field, and the cosmic ray ionization rate at $\xi = 10^{-16} \text{ s}^{-1}$. While the assumption of a constant cosmic ray ionization rate is common in similar works ([Priestley et al., 2024, 2023b](#)), recent work has shown that this rate varies drastically based on the environment (see [Luo et al., 2023](#)) for diffuse clouds, and for different locations within star formation regions ([Sabatini et al., 2023](#)). Though the AMUSE framework is capable of simulating a variable cosmic ray field, in this proof-of-concept simulation the cosmic ray ionization rate is kept constant. This is done to make comparison with earlier work easier, and also to reduce the complexity of this simulation. Due to this same argument of simplicity, turbulence and magnetic fields are not included either.

In the center of the cloud, substructures with higher densities are visible, while near the outside scattered particles that were likely expelled from the clouds during the collision can be seen. This distinction is also visible in Figure B.5, which shows the distribution of the gas temperature as a function of the density for the chemical particles at the end of the simulation. As expected, the bulk of the particles are at relatively high density and low temperature, while lower-density particles, near the outside of the cloud, have higher temperatures. The relatively low number of particles prevents a more comprehensive analysis of this distribution. The temperatures of the full set of hydrodynamical particles would allow such an analysis, but (due to an error) they are unfortunately not accessible. Regardless, this distribution is qualitatively in line with expectations.

Unexpectedly, the simulation shows relatively low values for the number density. [Priestley et al. \(2023a\)](#) reached number densities up to 10^6 cm^{-3} with a very similar setup, which is comparable to observed densities in dense cores. The fact that this simulation does not reproduce these values indicates it is missing some crucial components needed to form these dense structures. Unlike [Priestley et al. \(2023a\)](#), the simulation in this work does not contain turbulence, magnetic fields, or sink particles. Turbulence and magnetic field effects together cause an effect named reconnection diffusion, which is able to mediate cloud collapse. This simulation also does not contain sink particles to model the areas with densities beyond the simulations resolution. The mass resolution is also higher than in [Priestley et al. \(2023a\)](#): they used a mass resolution of $0.005 M_\odot$ per

The hydrodynamics of the simulation have a clear effect on the chemistry. The jump in density caused by the collision of the clouds translates to higher abundances for a wide variety of gas-phase molecules. The specific molecules in this figure were

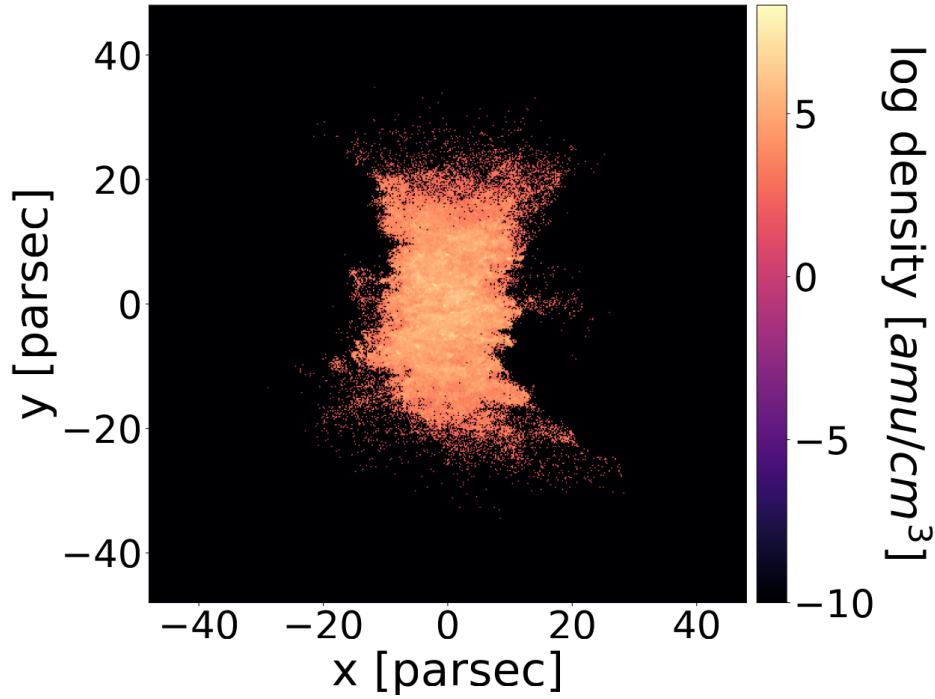
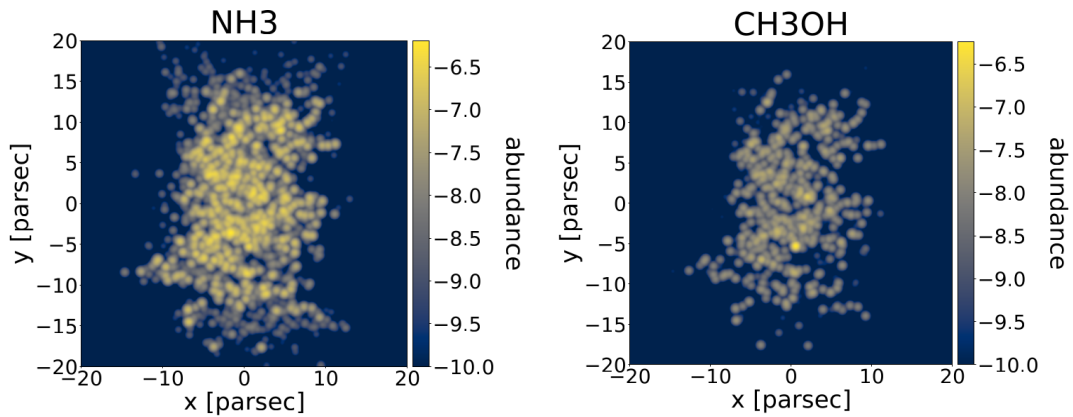


Figure B.5: Projected view of a simulated molecular cloud.

chosen to allow for comparison with [Priestley *et al.* \(2023b\)](#), who present their results for the same selection of molecules, all in the gas phase. It should be noted, however, that the abundance on the grain surface for each of these molecules is also available in this simulation, as well as every other molecule in the network. These two are chosen because they are commonly observed in the gas phase. This shows the successful coupling between the hydrodynamics and chemistry in this simulation. The abundances show a similar spatial distribution to the hydrogen density shown in Figure [B.6](#).

Figure B.6: Projected distribution of abundances for NH_3 and CH_3OH .

B.2.9 Tidymess: TIdal DYnamics of Multi-body ExtraSolar Systems

As part of a Masters project, Tunde Meyer developed an interface to the `Tidymess` code. `Tidymess` (Boekholt & Correia, 2023a,b) (TIdal DYnamics of Multi-body ExtraSolar Systems) is an open-source N -body code designed to model the complex dynamical evolution of systems where tidal forces and deformations play a critical role, such as in planetary systems, moons, stars, and compact objects. `Tidymess` enables detailed, efficient, and flexible modeling of tidal dynamics in multi-body astrophysical systems, filling a unique niche for studying chaotic and non-linear tidal evolution.

`Tidymess` implements the Creep deformation law (Norton, 1929; Bailey, 1935), allowing each body’s tidal response to be characterized by its fluid Love number and relaxation time. This approach is more realistic for long-term evolution and can handle both linear and non-linear tidal regimes. The code models bodies as ellipsoids, accounting for both tidal and centrifugal deformations. The gravitational field is calculated up to quadrupole order, enabling accurate computation of accelerations and torques. Orbits, spins, and deformations are integrated self-consistently using a fourth-order symplectic method. A novel aspect is that deformation integration uses a time-step based on orbital timescales, independent of spin or relaxation times, which improves efficiency and consistency across different tidal regimes. `Tidymess` is particularly suited for systems with many bodies, chaotic orbital evolution, and strong, non-linear tidal interactions.

Bibliography

- Bailey, R. W. 1935. Creep of Steel under Simple and Compound Stresses. *Proceedings of the Institution of Mechanical Engineers*, **131**(1), 1–16.
- Bal, H.E., Maassen, J., van Nieuwpoort, R.V., Drost, N., Kemp, R., Palmer, N., Wrzesinska, G., & Kielmann, T. 2010. Real-World Distributed Computing with Ibis. *IEEE Computer*, **43**(Aug.), 54–62.
- Bate, Matthew R., Bonnell, Ian A., & Price, Nigel M. 1995. Modelling accretion in protobinary systems. *MNRAS*, **277**(2), 362–376.
- Bitsch, Bertram, Morbidelli, Alessandro, Johansen, Anders, Lega, Elena, Lambrechts, Michiel, & Crida, Aurélien. 2018. Pebble-isolation mass: Scaling law and implications for the formation of super-Earths and gas giants. *A&A*, **612**(Apr.), A30.
- Boekholt, Tjarda C. N., & Correia, Alexandre C. M. 2023a. A direct N-body integrator for modelling the chaotic, tidal dynamics of multibody extrasolar systems: TIDYMESS. *MNRAS*, **522**(2), 2885–2900.
- Boekholt, Tjarda C. N., & Correia, Alexandre C. M. 2023b (June). *TIDYMESS: TIdal DYnamics of Multi-body ExtraSolar Systems*. Astrophysics Source Code Library, record ascl:2306.004.
- Coleman, Gavin A. L., & Haworth, Thomas J. 2020. Peter Pan discs: finding Neverland’s parameters. *MNRAS*, **496**(1), L111–L115.
- Concha-Ramírez, Francisca, Wilhelm, Martijn J. C., Portegies Zwart, Simon, van Ter-

- wisga, Sierk E., & Hacar, Alvaro. 2021. Effects of stellar density on the photoevaporation of circumstellar discs. *MNRAS*, **501**(2), 1782–1790.
- Dagum, L., & Menon, R. 1998. OpenMP: An Industry-Standard API for Shared-Memory Programming. *IEEE Comput. Sci. Eng.*, **5**(1), 46–55.
- Drost, Niels, Maassen, Jason, van Meersbergen, Maarten A. J., Bal, Henri E., Pelupessy, F. Inti, Zwart, Simon Portegies, Kliphuis, Michael, Dijkstra, Henk A., & Seinstra, Frank J. 2012. High-Performance Distributed Multi-Model / Multi-Kernel Simulations: A Case-Study in Jungle Computing. *Pages 150–162 of: IPDPS Workshops*.
- Flaccomio, E., Micela, G., & Sciortino, S. 2012. X-ray variability of pre-main-sequence stars. Toward an explanation of the different X-ray properties of classical and weak-line T Tauri stars. *A&A*, **548**(Dec.), A85.
- Gay, Warren W. 2000. *Linux Socket Programming: By Example*. Indianapolis, IN, USA: Que Corp.
- Gosling, J., Joy, B., & Steele, G.L. 1996. *The Java Language Specification*. 1st edn. Boston, MA, USA: Addison-Wesley Longman Publishing Co., Inc.
- Gosling, J., Joy, B., Steele, G. L., Bracha, G., & Buckley, A. 2014. *The Java Language Specification, Java SE 8 Edition*. 1st edn. Addison-Wesley Professional.
- Gropp, W. 2002. *MPICH2: A New Start for MPI Implementations*. Berlin, Heidelberg: Springer Berlin Heidelberg. Pages 7–7.
- Gropp, William, Lusk, Ewing, Doss, Nathan, & Skjellum, Anthony. 1996. A high-performance, portable implementation of the MPI message passing interface standard. *Parallel Computing*, **22**(6), 789–828.
- Hamers, Adrian S., & Portegies Zwart, Simon F. 2016. Secular dynamics of hierarchical multiple systems composed of nested binaries, with an arbitrary number of bodies and arbitrary hierarchical structure. First applications to multiplanet and multistar systems. *MNRAS*, **459**(3), 2827–2874.
- Haworth, Thomas J., Clarke, Cathie J., Rahman, Wahidur, Winter, Andrew J., & Facchini, Stefano. 2018. The FRIED grid of mass-loss rates for externally irradiated protoplanetary discs. *MNRAS*, **481**(1), 452–466.
- Holdship, J., Viti, S., Jiménez-Serra, I., Makrýmallis, A., & Priestley, F. 2017. UCLCHEM: A Gas-grain Chemical Code for Clouds, Cores, and C-Shocks. *AJ*, **154**(1), 38.
- Huang, Shuo, Portegies Zwart, Simon, & Wilhelm, Maite J. C. 2024. Suppression of giant planet formation around low-mass stars in clustered environments. *A&A*, **689**(Sept.), A338.
- Ikoma, Masahiro, Nakazawa, Kiyoshi, & Emori, Hiroyuki. 2000. Formation of Giant Planets: Dependences on Core Accretion Rate and Grain Opacity. *ApJ*, **537**(2), 1013–1025.
- Johnstone, Colin P., Bartel, Michael, & Güdel, Manuel. 2020. The active lives of stars: a complete description of rotation and XUV evolution of F, G, K, and M dwarfs. *arXiv e-prints*, Sept., arXiv:2009.07695.
- Keil, Marcus, Viti, Serena, & Holdship, Jonathan. 2022. UCLCHEMCMC: An MCMC Inference Tool for Physical Parameters of Molecular Clouds. *ApJ*, **927**(2), 203.
- Kozai, Yoshihide. 1962. Secular perturbations of asteroids with high inclination and

- eccentricity. *AJ* , **67**(Nov.), 591–598.
- Krumholz, M. R., & Forbes, J. C. 2015. VADER: A flexible, robust, open-source code for simulating viscous thin accretion disks. *Astronomy and Computing* , **11**(June), 1–17.
- Lidov, M. L. 1962. The evolution of orbits of artificial satellites of planets under the action of gravitational perturbations of external bodies. *Planet. Space Sci.* , **9**(10), 719–759.
- Luo, Gan, Zhang, Zhi-Yu, Bisbas, Thomas G., Li, Di, Zhou, Ping, Tang, Ningyu, Wang, Junzhi, Zuo, Pei, & Yue, Nannan. 2023. Abundance Ratios of OH/CO and HCO⁺/CO as Probes of the Cosmic-Ray Ionization Rate in Diffuse Clouds. *ApJ* , **946**(2), 91.
- Lynden-Bell, D., & Pringle, J. E. 1974. The evolution of viscous discs and the origin of the nebular variables. *MNRAS* , **168**(Sept.), 603–637.
- Nethercote, N., & Seward, J. 2007 (June). Valgrind: A Framework for Heavyweight Dynamic Binary Instrumentation. *Pages 89–100 of: McKinley, K.S., & Ferrante, J. (eds), Proceedings of ACM SIGPLAN 2007 Conference on Programming Language Design and Implementation (PLDI 2007)*.
- Norton, Frederick Harwood. 1929. *The Creep of Steel at High Temperatures*. New York: McGraw-Hill Book Company, Inc.
- Ormel, Chris W., Shi, Ji-Ming, & Kuiper, Rolf. 2015. Hydrodynamics of embedded planets’ first atmospheres - II. A rapid recycling of atmospheric gas. *MNRAS* , **447**(4), 3512–3525.
- Owen, James E., Clarke, Cathie J., & Ercolano, Barbara. 2012. On the theory of disc photoevaporation. *MNRAS* , **422**(3), 1880–1901.
- Pearu, P. 2009. F2PY: a tool for connecting Fortran and Python programs. *International Journal of Computational Science and Engineering* , **4**(4), 296–305.
- Pelupessy, F. I., van der Werf, P. P., & Icke, V. 2004. Periodic bursts of star formation in irregular galaxies. *A&A* , **422**(July), 55–64.
- Perri, F., & Cameron, A. G. W. 1974. Hydrodynamic Instability of the Solar Nebula in the Presence of a Planetary Core. *Icarus* , **22**(4), 416–425.
- Picogna, Giovanni, Ercolano, Barbara, Owen, James E., & Weber, Michael L. 2019. The dispersal of protoplanetary discs - I. A new generation of X-ray photoevaporation models. *MNRAS* , **487**(1), 691–701.
- Portegies Zwart, Simon, McMillan, Steve, Harfst, Stefan, Groen, Derek, Fujii, Michiko, Nualláin, Breannán Ó., Glebbeek, Evert, Heggie, Douglas, Lombardi, James, Hut, Piet, Angelou, Vangelis, Banerjee, Sambaran, Belkus, Houria, Fragos, Tassos, Fregeau, John, Gaburov, Evghenii, Izzard, Rob, Jurić, Mario, Justham, Stephen, Sottoriva, Andrea, Teuben, Peter, van Bever, Joris, Yaron, Ofer, & Zemp, Marcel. 2009. A multiphysics and multiscale software environment for modeling astrophysical systems. *New Astron.* , **14**(4), 369–378.
- Price, Daniel J., Wurster, James, Tricco, Terrence S., Nixon, Chris, Toupin, Stéven, Pettitt, Alex, Chan, Conrad, Mentiplay, Daniel, Laibe, Guillaume, Glover, Simon, Dobbs, Clare, Nealon, Rebecca, Liptai, David, Worpel, Hauke, Bonnerot, Clément, Dipierro, Giovanni, Ballabio, Giulia, Ragusa, Enrico, Federrath, Christoph, Iaconi, Roberto, Reichardt, Thomas, Forgan, Duncan, Hutchison, Mark, Constantino,

- Thomas, Ayliffe, Ben, Hirsh, Kieran, & Lodato, Giuseppe. 2018. Phantom: A Smoothed Particle Hydrodynamics and Magnetohydrodynamics Code for Astrophysics. *Proc. Astron. Soc. Aust.* , **35**(Sept.), e031.
- Priestley, F. D., Clark, P. C., Glover, S. C. O., Ragan, S. E., Fehér, O., Prole, L. R., & Klessen, R. S. 2023a. NEATH - II. N_2H^+ as a tracer of imminent star formation in quiescent high-density gas. *MNRAS* , **526**(4), 4952–4960.
- Priestley, F. D., Clark, P. C., Glover, S. C. O., Ragan, S. E., Fehér, O., Prole, L. R., & Klessen, R. S. 2023b. Non-Equilibrium Abundances Treated Holistically (NEATH): the molecular composition of star-forming clouds. *MNRAS* , **524**(4), 5971–5983.
- Priestley, F. D., Clark, P. C., Glover, S. C. O., Ragan, S. E., Fehér, O., Prole, L. R., & Klessen, R. S. 2024. NEATH - III. A molecular line survey of a simulated star-forming cloud. *MNRAS* , **531**(4), 4408–4421.
- Robbins, A. 2005. *GDB - pocket reference: debugging quickly and painlessly*. O'Reilly.
- Sabatini, Giovanni, Bovino, Stefano, & Redaelli, Elena. 2023. First ALMA Maps of Cosmic-Ray Ionization Rate in High-mass Star-forming Regions. *ApJL* , **947**(1), L18.
- Stone, James M., Gardiner, Thomas A., Teuben, Peter, Hawley, John F., & Simon, Jacob B. 2008. Athena: A New Code for Astrophysical MHD. *ApJS* , **178**(1), 137–177.
- Stone, James M., Tomida, Kengo, White, Christopher J., & Felker, Kyle G. 2020. The Athena++ Adaptive Mesh Refinement Framework: Design and Magnetohydrodynamic Solvers. *ApJS* , **249**(1), 4.
- Tanigawa, Takayuki, & Tanaka, Hidekazu. 2016. Final Masses of Giant Planets. II. Jupiter Formation in a Gas-depleted Disk. *ApJ* , **823**(1), 48.
- Toonen, S., Nelemans, G., & Portegies Zwart, S. 2012. Supernova Type Ia progenitors from merging double white dwarfs. Using a new population synthesis model. *A&A* , **546**(Oct.), A70.
- Toonen, S., Portegies Zwart, S., Hamers, A. S., & Bandopadhyay, D. 2020. The evolution of stellar triples. The most common evolutionary pathways. *A&A* , **640**(Aug.), A16.
- Toonen, Silvia, Hamers, Adrian, & Portegies Zwart, Simon. 2016. The evolution of hierarchical triple star-systems. *Computational Astrophysics and Cosmology*, **3**(1), 6.
- van der Walt, S., Colbert, S. C., & Varoquaux, G. 2011. The NumPy Array: A Structure for Efficient Numerical Computation. *Computing in Science Engineering*, **13**(2), 22–30.
- Vasiliev, Eugene. 2018 (May). *AGAMA: Action-based galaxy modeling framework*. Astrophysics Source Code Library, record ascl:1805.008.
- Vasiliev, Eugene. 2019. AGAMA: action-based galaxy modelling architecture. *MNRAS* , **482**(2), 1525–1544.
- von Zeipel, H. 1910. Sur l'application des séries de M. Lindstedt à l'étude du mouvement des comètes périodiques. *Astronomische Nachrichten*, **183**(22), 345.

Appendix C

Programming Primer

To be a programmer is to develop a carefully managed relationship with error.

Ellen Ullman

In this book, we assume that you are familiar with the Linux operating system, with the Python programming language and with the use of `git`. If this is not the case, we present here a very basic primer about them. **Linux** is a **UNIX**-based operating system designed in 1991 by Linus Torvalds. **Python** is a third-generation computer language designed in 1991 by Guido van Rossum. **Git** is a source code management system that began in 2005 as a development system for Linus Torvalds' Linux kernel.

C.1 Linux

The Linux operating system was originally designed for microcomputers.¹ It is composed of a number of functional components that together form the operating system. These components include (see <http://linux.com>):

The **bootloader**, the software that manages the startup process of your computer.

The **kernel**, the core of the system that manages the CPU, RAM memory, and peripheral devices.

Daemons, background services that manage printing, sound, scheduling, etc., that either start up during boot, or when you log in to the desktop.

The **shell**, which manages commands to control the computer. Commands are typically typed into a terminal with a text interface, or contained in scripts.

¹“Microcomputer” refers to the type of machine we nowadays call a workstation. Minicomputers used to be the mid-level machines that we now call cluster computers or Beowulfs ([Sterling, 2002](#)), not to be confused with the Raspberry Pi processor board, which is sometimes also referred to as a minicomputer.

The **graphical server** (or **X server**), the subsystem that displays graphics on your monitor.

The desktop environment, the customizable user interface. There are many choices, such as Gnome, KDE, Cinnamon, Enlightenment, etc.

Applications (apps), the programs that run on your computer. In this book, you write your own applications using AMUSE, but you can download applications or write them in other languages too. One application you will be using a lot is a text editor. This can be **emacs** (Ciccarelli, 1978), **Pico**, **vim**, or any of a number of others. Choice of editor is squarely in the category of religion and politics, and we make no recommendation. Another application you will be using a lot is the Python interpreter (van Rossum, 1995a,b,c,d).

You generally communicate with your computer via the desktop environment. Most modern operating systems have desktop environments, but the sophisticated shell and the programming style it enables make **Linux** (and its cousin **UNIX**) rather unique. Some other operating systems have command-line interfaces, but they are often undeveloped and almost completely useless. Note that there is not a single Linux, but many flavors, including Fedora, Ubuntu, RedHat, Suse, Debian, Gentoo, and much more (see <http://distrowatch.com/>). The practical differences are generally not so large, although true OS geeks swear by their preferred flavor. MacOS is not **Linux**, but is based on FreeBSD, a version of **UNIX**, and presents a programming environment essentially equivalent to that found in **Linux**.

The shell allows you to send commands directly to your computer, or launch scripts to automate this process. Some typical **Linux** commands, in no particular order, include:

```
%> man -t ascii | ps2pdf - > ascii.pdf makes a PDF file of a manual page.

%> apropos whatis combines two commands: apropos, which lists alternatives
on a specific command, and whatis, a rudimentary help function.

%> which python shows the full path name of the command Python.

%> mkdir amuse creates a directory with the name amuse.

%> ls amuse lists the contents of directory amuse.

%> cd amuse go to directory named amuse.

%> find . -name *.py finds all files with the extention .py in all subdirectories.

%> locate sun_venus_earth.py finds the file sun_venus_earth.py.

%> tar -x dir | gzip make a zip-compressed archive of the directory dir.

%> sort -u file1 file2 make a union of unsorted files.

%> gzip < /dev/cdrom > cdrom.iso.gz saves a copy of data from a CD-ROM.
```

```
%> ps -aux shows a list of all processes running on your computer.

%> top shows an interactive list of your current processes.

%> grep MemTotal /proc/meminfo shows the total random access memory in
the system. (Note that macOS does not have a /proc filesystem.)

%> cat /proc/cpuinfo displays the contents of the file named /proc/cpuinfo.

%> tail -lines=10 filename shows the last 10 lines of the file named filename.

%> less filename displays the content of the file filename, page by page.
```

Note the use of the operators “<” (read from a file or the keyboard), “>” (write to a file or the computer screen), and “|” (connect the output of one command to the input of another).

C.1.1 A little awking and grepping

The Linux shell includes a large number of handy utilities, which can help you work more efficiently with the operating system. The commands **awk** (see <http://www.gnu.org>) and **grep** (Kernighan, 1984) are particularly useful for rudimentary data processing.

- **awk** is a scripting language designed to to manipulate ASCII text documents and automate the extraction of data. It was developed in the 1970s by Alfred Aho, Peter Weinberger, and Brian Kernighan (Aho *et al.*, 1987).
- **grep** is a utility run from the Linux command line that searches for regular expressions in plain-text data sets.

Typically, **grep** is used to search and catch certain phrases, whereas **awk** is used to manipulate them. Knowing some basic **awk** and **grep** commands will save you valuable time, and we strongly recommend that you learn how to work with them. You might begin with

```
%> man awk
```

and

```
%> man grep
```

both of which will provide more information than you need right now. These utilities make for a powerful experience in the command shell.

Another handy command-line tool is the stream editor **sed**, which is very efficient at manipulating a data stream (from a file or another program) on the fly.

C.2 Git

While working through this book you are likely to write source code for several numerical experiments. `Git` is an excellent source code management system that lets you keep track of and manage the different versions of your code. Tutorials and complete documentation are readily available online at many locations, but we recommend <https://www.atlassian.com/git/tutorials/>.

Probably the simplest way to start using `git` is to create an account on one of the online `git` brokers, such as bitbucket.org, github.com, or sourceforge.net. These sites are widely used and free. There are many more sites, some providing better services and enhanced security, but at a cost. The details vary, but after you create an account you will generally be guided through the steps needed to create a new minimal project, which you can then clone to your own computer and populate with your own files, as described below.

Alternatively, you can make and manage your own `git` repository. First create a directory on the computer where you want the repository to reside. The following commands create a directory containing an empty `git` repository.

```
%> mkdir project-home
%> cd project-home
%> git init --bare
%> git update-server-info
```

Now you can update the repository with your project source code, copied from another computer or another directory on the same computer. We assume you have placed it in the local subdirectory `project`. You can add the code to the repository as follows.

```
%> cd project
%> git init
%> git add *
%> git commit -m "First version of my code"
%> git remote add origin spz@eucalypta.strw.leidenuniv.nl:project-home
%> git push -u origin master
```

Here `spz@eucalypta.strw.leidenuniv.nl:project-home` refers to your `git` server's URL and the particular directory on that server where the project resides. Of course, you don't want to store your data on the first author's computer at the Leiden Observatory in the Netherlands, so you'd better create your own!

Now suppose you want to access the code on another computer, maybe your laptop, make changes, and upload them to the master repository. To do this, you first have to make a local directory for your code and clone the master copy of the code:

```
%> git clone spz@eucalypta.strw.leidenuniv.nl:project-home project
```

Now you can make changes to the source code using your favorite editor and upload them:

```
%> cd project
%> ...(edit files)...
%> git add filename.cpp
%> git commit -m "Small change made in filename.cpp"
%> git push origin master
```

Your changes are now added to the master repository, but that does not mean that they have been propagated to other clones of the code. Suppose your friend's computer also has a working copy of the repository. She is still working with the version from before the changes you made, but can update her local version with the commands

```
%> cd project
%> git pull
```

Now she also has the latest version.

This all sounds very nice, although maybe a bit arcane, but believe us, it is a great improvement over exchanging source code by e-mail or using other version control systems. More information on `git` can be found at <http://rogerdudler.github.io/git-guide/>.

C.3 Python

If you learned to program computers in the dark ages, like two of the authors, Python ([van Rossum, 1995a,b,c,d](#)) is a breath of fresh air. Almost like air travel in the 1980s: few restrictions, and nothing to declare! Python is easy to learn and it is almost impossible to write obfuscated code in (in contrast to C: see <http://www.ioccc.org/>). In this section, we review a few basics you should know about Python in order to use this book. This section will be as short as possible because there are a great many excellent books written about the language. We recommend especially the many books about the language by Hans Petter Langtangen, and specifically *Python Scripting for Computational Science* ([Langtangen, 2009](#)).

Python is a procedural language, which means that it works with lists of instructions that tell the computer what to do with input and data. Other examples of procedural languages include C ([Kernighan & Ritchie, 1988](#)) and FORTRAN ([American National Standards Institute, 1966](#)). Some procedural languages incorporate abstract data types that allow objects can be created and manipulated. Examples of these object-oriented languages include C++ ([Stroustrup, 2013](#)) and Java ([Gosling *et al.*, 1996, 2014](#)). Python is also an object-oriented language, and includes some other interesting features drawn from C++ and Lisp.

Python is a typed language—variables have specific types that can only be combined in certain ways—but unlike the “scientific” programming languages listed above, types are not declared at compile time, but instead are inferred from context as needed. This makes Python both very flexible and much slower than C++ or FORTRAN. Indeed, the whole point of AMUSE is to combine the flexibility of Python with the speed of

compiled procedural languages. Python also supports reflection—the program structure can be modified during execution.

You generally start a Python session on the command line by invoking the interpreter

```
%> python
```

or, for the somewhat more practical interactive shell interpreter,

```
%> ipython
```

Both are basically the same language (Perez & Granger, 2007). If you wish you can try the Jupyter notebook environment (Kluyver *et al.*, 2016), which can be found at <http://jupyter.org/>. This is a browser-based support structure on top of the usual Python interpreter, and has a number of powerful and interesting features.

C.3.1 Python types and printing

As a procedural language, a Python script is composed of a list of instructions. One of the simplest is a scalar assignment:

```
>>> star = 5
```

(here and below >>> represents the interactive Python prompt), which here assigns the integer 5 to the variable `star`. If we had wanted to use a floating point number, we would have said

```
>>> star = 5.0
```

The instructions can contain arithmetic operations

```
>>> other_star = 4*star - 2
>>> print(other_star)
18
```

Note that the Python interpreter decides a variable's type at run time, and the type can change from line to line, e.g.

```
>>> star = 5
>>> a = star
>>> star = 5.0
>>> print(a, star)
5. 5.0
```

One “feature” of Python 2 was that dividing an integer by an integer returns an integer. For example, if you divided two integers and printed the result, you would get

```
>>> print 1/2
0
```

This is very familiar to C and FORTRAN programmers, and is even very useful once you understand that it happens, but it is a never-ending source of confusion in beginners.

Python 3 has eliminated this feature (to the chagrin of the old-timers), and the same code would produce the following:

```
>>> print(1/2)
0.5
```

while the former behavior can be reproduced with the `//` operator:

```
>>> print(1//2)
0
```

In our opinion it is generally better (meaning that it produces more predictable code) to write the decimal point explicitly:

```
>>> print(1./2.)
0.5
```

To be sure about the type of variable, you can always query its type explicitly:

```
>>> type(star)
<type 'int'>
```

We have used the function `print` several times above. In Python 2.x we could have eliminated the parentheses:

```
>>> print 1./2.
0.5
```

but in Python 3 they are required. Integer division and the `print` statement are the two most noticeable differences between Python 2 and Python 3. Because AMUSE was originally designed and implemented using Python 2.7, we usually assumed that integer division yields an integer and used `print` statements without parentheses in the AMUSE source for the first edition of this book. In this edition we have switched fully to Python 3.

C.3.2 Lists

Variables of different types can be combined into lists, e.g.

```
>>> stellar_list = [star, other_star, 1/2, 1./2.]
```

which will give

```
>>> print (stellar_list)
[5, 18, 0, 0.5]
```

Lists are nice. They can contain anything you like. Individual elements can be accessed by using their position in the list. Like C, but unlike FORTRAN, Python counts from zero, so the list here runs from element 0 to element 3.

The length of a list can be queried by the function `len()` with the list as an argument:

```
>>> len(stellar_list)
4
```

The last element of a list can also be accessed using by the index -1:

```
>>> print(stellar_list[-1])
0.5
```

You can check this by comparing the last element of a list referenced in two different ways:

```
>>> stellar_list[-1] == stellar_list[len(stellar_list)-1]
True
```

The value `True` indicates that the two expressions are indeed identical.

A list is really a class (see below). That means that a number of operations are defined to work with or on lists. For example to add a new element to a list

```
>>> stellar_list.append(100)
```

or sort a list

```
>>> stellar_list.sort()
>>> print(stellar_list)
[0, 0.5, 4, 14, 100]
```

or remove an element from a list

```
>>> stellar_list.pop(0)
>>> print(stellar_list)
[0.5, 4, 14, 100]
```

Lists also have operators, for example the `+` operator that enables you to concatenate several lists

```
>>> stellar_list + [3, 2, 1]
[0.5, 4, 14, 100, 3, 2, 1]
```

You can do a lot more with lists, but by now you probably get the idea.

C.3.3 Dictionaries

Making a hash table in Python is easily accomplished by using a dictionary. This works as follows

```
>>> stellar_mass = {'Betlegeuse': 8, 'Rigel': 21, 'Spica': 10.25}
>>> for name, mass in stellar_mass.iteritems():
...:     print('star: %s has mass: %s' % (name, mass))
...:
star: Rigel has mass: 21
star: Spica has mass: 10.25
star: Betlegeuse has mass: 8
```

Each of the entries `Betlegeuse`, `Spica` and `Rigel` are now associated with a value.

Just as in a list this value can be anything, a parameter, a function, or even a class.

C.3.4 Flow control

A simple python program is usually composed of conditionals and loops, just as in any other procedural language. The main flow control elements in Python are `if`, `for`, and `while`.

A typical `if` statement chooses between two or more alternatives:

```
>>> if star == 6:
...:     print(star)
...:else:
...:     print(other_star)
...:
18
```

Other comparison operators (e.g. `<`, `<=`, `>`, `>=`, etc.) can be used in place of `==`. One slight peculiarity of Python is that it also includes the operators `is` and `is not`. You might think that `==` and `is` should mean more or less the same, but actually `==` compares the value of two expressions, whereas `is` tests if the two objects are the same:

```
>>> 1 == True
True
>>> 1 is True
False
```

The `for` loop has a running parameter (here called `i`) that increases from a value 0 up to some value:

```
>>> for i in range(3):
...:     print(i)
...:
0
1
2
```

In this case, we simply print the variable `i`. The loop stops automatically when the objective is reached. Note that, in consistent Python usage, `range(3)` actually returns the list `[0, 1, 2]`. Python recognizes which instructions belong to the loop by indentation. In most AMUSE scripts we adopt 4 spaces of indentation, although sometimes, if the lines become long and the number of inner loops large, we use 2. Other languages use special symbols to indicate the start and the end of a loop, such as braces in `C` or the words `DO` and `END DO` in `FORTRAN`. (`C` and `FORTRAN` programmers generally hate Python's strict reliance on indentation to indicate structure.)

A `while` loop continues until some boolean expression is no longer true. The previous loop could have been written

```
>>> i = 0
>>> while (i < 3):
...:     print(i)
...:     i += 1
...:
0
1
2
```

To be honest (as is also the case in **C** and **FORTRAN**), we don't really need both **for** and **while** loop constructs—one can always be written in terms of the other, but tradition is a powerful thing. Unlike **C** or **C++**, Python has no **case** or **switch** statements. This is not a problem because again these functionalities can be constructed as needed using the available flow control operators.

C.3.5 Functions

Python allows the user to define functions. They can take and return any argument (including other functions). The purpose of a function is to prevent code duplication and to make your code as modular as possible. A simple function in Python might look like

```
def factorial(n):
    if n < 1:
        return 1
    else:
        return n*factorial(n-1)
```

(Note: you should never write the factorial function this way in a real script!)

A more useful function that calculates the radius of a star in solar radii, given its mass in solar masses, might be:

```
def stellar_radius(mass=1.0):
    radius = mass**0.8
    return radius
```

Note the use of a default value for the argument. You might want to save this handy function in a file called `calculate_stellar_radius.py` for later use. Then, from another script (or interactive Python session) you could load this function and use it.

```
from calculate_stellar_radius import stellar_radius
r = stellar_radius(2)
print(r)
```

We recommend trying to keep functions small, less than a page in length if possible, and focused on a single operation.

C.3.6 Importing packages

In an earlier example (in Appendix C.3.5), we imported a function from a file we had just created. Many functions and entire packages are already written, and you should always try to avoid rewriting anything that has already been written, except perhaps for pedagogical purposes. Just as we import the `amuse.lab` package, you can import other packages. Important packages include `numpy` (van der Walt *et al.*, 2011), `scipy` (see for example Rojas *et al.*, 2015), `math` for numerical, scientific, and mathematical extensions of the language. There are also packages for plotting, such as `pyplot`, machine learning `pyML`, Markov-Chain Monte Carlo (such as `emcee`; Foreman-Mackey *et al.*, 2013), `AstroPy` (The Astropy Collaboration *et al.*, 2013), and many more.

C.3.7 Classes

A class is a data container with associated operators and functions to manipulate the data it contains. An example of a widely used class in AMUSE is the `Particle` set. Here we illustrate the use of classes by creating a simple `Body` class that can form the basis for more complex data structures. Code example C.1 illustrates how to declare a `Body` base class. This class is not complicated—it requires only one initialization

```

2 class Body:
3     def __init__(self, mass):
4         self.type = "body"
5         self.mass = mass
6
7     def __str__(self):
8         return f"{self.type} mass = {self.mass}"

```

Code example C.1: A base class.

parameter, the mass of the object, contains two class variables, called `type` and `mass`, and two class methods (functions), one to initialize the class itself, and one to return its contents as a string. A more elaborate example of the use of classes in AMUSE can be found in Section 9.3.2.

A derived class is a class that inherits the properties of a base class and extends its functionality. Code example C.2 presents two examples, for a planet (called `Planet`), and for a star (called `Star`), both derived from `Body`. (Class declaration names in Python conventionally start with a capital letter.) They share the `Body` class variable `mass` and the `Body` member functions. Both, however, extend the class, `Planet` by adding the orbital parameters semi-major axis and eccentricity, and `Star` by adding a radius. In addition, they both override the `type` class variable, to `"planet"` and `"star"`, respectively, and modify the string returned by `__str__`. We can now manipulate these classes, make declarations, put them in arrays, give them to functions as arguments, etc.

An useful aspect of classes is the ability to make compound classes that themselves are classes. This is illustrated in Code example C.3. We could further develop this class structure by making it more object-oriented, but that would be beyond the scope

```

14 MassEarth = 3.0e-6
15 RadiusEarth = 0.01
16
17
18 class Planet(Body):
19     def __init__(self, mass, semimajor_axis, eccentricity=0.0):
20         Body.__init__(self, mass)
21         self.type = "planet"
22         self.semi = semimajor_axis
23         self.ecc = eccentricity
24         self.radius = RadiusEarth * (mass / MassEarth) ** 0.55
25
26     def __str__(self):
27         return f"{self.type} mass = {self.mass}, a = {self.semi}"
28
29
30 class Star(Body):
31     def __init__(self, mass):
32         Body.__init__(self, mass)
33         self.type = "star"
34         self.calculate_radius()
35
36     def __str__(self):
37         return f"{self.type} mass = {self.mass}, radius = {self.radius}"
38
39     def calculate_radius(self):
40         self.radius = self.mass**1.2

```

Code example C.2: Derived class fragments.

of this book, particularly because there are many books that do an excellent job in explaining classes and inheritance.

Classes are nice and versatile. They make programming a lot more fun, and we strongly encourage you to explore their potential. Despite these recommendations, in this book we have largely refrained from using classes, mainly to avoid going too deeply into unfamiliar territory.

```

47 class PlanetarySystem(Body):
48     def __init__(self, star, planets):
49         self.type = "planetary system"
50         self.star = star
51         self.planets = planets
52
53     def print(self):
54         print(self.star)
55         for pi in self.planets:
56             print(pi)

```

Code example C.3: A compound class.

C.3.8 Operator overloading

The redefinition in the previous section of member functions with double leading and trailing underscores is a form of overloading. The member functions

```
def __init__(self):
```

and

```
def __str__(self):
```

defined in Code example C.1 overload the previously existing member functions of the same names. Aside from these predefined functions, many other functions can be overloaded, including arithmetic and other operators. You can even define your own operators and give them specific functionality. In AMUSE we have overloaded the pipe command, `|`, to add a unit to a quantity. The `AstroPy` package overloads the `*` operator for much the same purpose.

For example, in the earlier example of the `Body` base class, we could overload the `+` sign for adding a planet to a star. This could be realized by adding the following method to the `Star` class

```
def __add__(self, planet):
    self.mass += planet.mass
    self.calculate_radius()
    return self
```

In an executable script we could then write

```
s = Star(mass=1)
p = Planet(mass=0.01)
t = s + p
print(t)
```

Which would result in

```
star mass = 1.01, radius= 1.01201196814
```

This simply returns a `Star` with a larger mass. Obviously we'd want to do more, perhaps adding functionality to compute and/or evolve the planet's orbit, but the basic procedure should be clear. Quite a number of operators can be overloaded in this fashion.

C.3.9 Properties

You will find the line

```
@property
```

in a few places in the text, and in numerous class definitions in the AMUSE source code. This construction is unfamiliar to many casual Python users, so we present a brief indication here of what it means and how it works.

The `@property` line preceding a class method definition in Python is a concise way

to build a data descriptor that triggers a function call when an attribute is accessed. What does that mean? For example, in the `Star` class, we might add the method

```
@property
def core_mass(self):
    return 0.3*self.mass
```

Without the `@property` line, we would access the core mass with

```
s = Star(1)
mc = s.core_mass()
```

With the `@property` line, the second line above would become

```
mc = s.core_mass
```

Why is it important to turn a function call into an attribute? In AMUSE, the principal reason for doing this is to make access to class data more homogeneous. For example, in the `Particles` class, calculating the `kinetic_energy` and `potential_energy` are done internally using functions, but it is convenient to use the `@properties` syntax to make these quantities accessible as class variables.

This may still seem like a minor syntactic change, but in Python this approach is part of a broader class of operations that control access to internal class data that shouldn't be accessed directly. In other object-oriented languages, such as C++, setting and getting private data in a class is commonly realized using a `set` or a `get` function. This allows the designer of a class to control how the class data are modified and used. In Python, which has “best-practice” conventions for limiting data access but doesn't actually have private data, this is best done via the `@property` paradigm and related constructs.

The syntax allows the result of a complex calculation to appear as an class attribute, as we have just seen, but it can also be used to prevent users from directly setting variables with “side effects.” For example, changing the mass of a star with a planetary system should also adjust the orbital elements of the planets orbiting it. Hence, we should not allow the mass to be modified directly, but instead intercept that attempt with a function that both changes the mass and modifies the orbits. However, since AMUSE does not use this extended functionality, we must stop the discussion here and refer the interested reader to the extensive Python documentation on the subject.

C.3.10 Testing and debugging Python scripts

Every script that is intended for anything other than testing should be tested, and the test scripts should be tested too. From personal experience, we know that any program we write is likely to have some serious shortcomings (aka bugs) if it is not thoroughly tested. Tests can be simple, but they should test fundamental functionalities of every script. It is not uncommon to spend more time writing tests than writing production code. In the end, writing extensive test functions can speed up the development of the production code enormously, and once finished, the code is much more likely to work as intended. Time devoted to writing test scripts is always time well spent.

A test code starts by importing the relevant python module

```
import unittest
```

Within the script to test, `test_get_posvel_from_binary_elements.py`, we add a test case, for example something simple like:

```
def test_get_posvel_from_binary_elements():
    a = 1 | units.au
    try:
        r, v = get_posvel_from_binary_elements(a, 0.0, 1)
        assert False
    except:
        pass
    r, v = get_posvel_from_binary_elements(a, 0.0, 1 |units.MSun)
    assert r[0] == a
```

If the `assert` statement fails, the test fails, and you should start debugging the relevant routine. You can test the script by calling:

```
%> pytest test_get_posvel_from_binary_elements.py
```

You should create tests for every routine you write. Give the test routine the same name as the routine that performs the particular test, and run **pytest** to perform the test.

Making and running a test scripts will help you to write correct and (almost) bug-free code. However, the time will probably come when you have to start a debug session (see Appendix B.1.7). Debugging is a necessity at every level of programming. It is no fun to hunt for the source of a **segmentation fault**². A run-time debugging tool may provide much needed relief. In Python this can be done by importing the **logging** library, after which it is possible to run a script in the debugger. This is done in as follows

```
import logging
logging.basicConfig(level=logging.DEBUG)
logging.getLogger("code").setLevel(logging.DEBUG)
```

There are few hard and fast practical rules on debugging. Pin down the bug using lots of `print` statements, wade through the data they produce to augment the debugger data, and good luck!

Bibliography

Aho, A. V., Kernighan, B. W., & Weinberger, P. J. 1987. *The AWK Programming Language*. Boston, MA, USA: Addison-Wesley Longman Publishing Co., Inc.
 American National Standards Institute. 1966. *ANSI Fortran X3.9-1966*. 1430 Broadway, New York, NY 10018, USA: American National Standards Institute.

²If you don't know what a segmentation fault is, make a point of never finding out (after Press *et al.*, 1992; in relation to the bubble-sort algorithm).

- Ciccarelli, E.C. 1978. *An introduction to the EMACS editor*. AI memo 447.
- Foreman-Mackey, Daniel, Hogg, David W., Lang, Dustin, & Goodman, Jonathan. 2013. emcee: The MCMC Hammer. *PASP*, **125**(925), 306.
- Gosling, J., Joy, B., & Steele, G.L. 1996. *The Java Language Specification*. 1st edn. Boston, MA, USA: Addison-Wesley Longman Publishing Co., Inc.
- Gosling, J., Joy, B., Steele, G. L., Bracha, G., & Buckley, A. 2014. *The Java Language Specification, Java SE 8 Edition*. 1st edn. Addison-Wesley Professional.
- Kernighan, Brian W. 1984. *The UNIX Programming Environment*. Prentice Hall Professional Technical Reference.
- Kernighan, Brian W., & Ritchie, Dennis M. 1988. *The C Programming Language*. 2nd edn. Prentice Hall Professional Technical Reference.
- Kluyver, Thomas, Ragan-Kelley, Benjamin, Perez, Fernando, Granger, Brian, Bussonier, Matthias, Frederic, Jonathan, Kelley, Kyle, Hamrick, Jessica, Grout, Jason, Corlay, Sylvain, Ivanov, Paul, Avila, Damián, Abdallah, Safia, Willing, Carol, & Jupyter, Project. 2016. Jupyter Notebooks: a publishing format for reproducible computational workflows. IOS Press.
- Langtangen, Hans Petter. 2009. *Python Scripting for Computational Science*. 3rd edn. Springer Publishing Company, Incorporated.
- Perez, F., & Granger, B. E. 2007. IPython: A System for Interactive Scientific Computing. *Computing in Science Engineering*, **9**(3), 21–29.
- Press, William H., Teukolsky, Saul A., Vetterling, William T., & Flannery, Brian P. 1992. *Numerical recipes in C. The art of scientific computing*.
- Rojas, S.J.G., Christensen, E.A., & Blanco-Silva, F. J. 2015. *Learning SciPy for Numerical and Scientific Computing Second Edition*. 2nd edn. Packt Publishing.
- Sterling, T.L. 2002. *Beowulf Cluster Computing with Linux*. Scientific and Computational Engineering Series. Books24x7.com.
- Stroustrup, B. 2013. *The C++ Programming Language*. 4th edn. Addison-Wesley Professional.
- The Astropy Collaboration, Robitaille, Thomas P., Tollerud, Erik J., Greenfield, Perry, Droettboom, Michael, Bray, Erik, Aldcroft, Tom, Davis, Matt, Ginsburg, Adam, Price-Whelan, Adrian M., Kerzendorf, Wolfgang E., Conley, Alexander, Crighton, Neil, Barbary, Kyle, Muna, Dimitri, Ferguson, Henry, Grollier, Frédéric, Parikh, Madhura M., Nair, Prasanth H., Günther, Hans M., Deil, Christoph, Woillez, Julien, Conseil, Simon, Kramer, Roban, Turner, James E. H., Singer, Leo, Fox, Ryan, Weaver, Benjamin A., Zabalza, Victor, Edwards, Zachary I., Azalee Bostroem, K., Burke, D. J., Casey, Andrew R., Crawford, Steven M., Dencheva, Nadia, Ely, Justin, Jenness, Tim, Labrie, Kathleen, Lim, Pey Lian, Pierfederici, Francesco, Pontzen, Andrew, Ptak, Andy, Refsdal, Brian, Servillat, Mathieu, & Streicher, Ole. 2013. Astropy: A community Python package for astronomy. *A&A*, **558**, A33.
- van der Walt, S., Colbert, S. C., & Varoquaux, G. 2011. The NumPy Array: A Structure for Efficient Numerical Computation. *Computing in Science Engineering*, **13**(2), 22–30.
- van Rossum, Guido. 1995a (Apr.). *Extending and embedding the Python interpreter*. Report CS-R9527. pub-CWI, pub-CWI:adr.
- van Rossum, Guido. 1995b (Apr.). *Python library reference*. Report CS-R9524. pub-

CWI, pub-CWI:adr.

van Rossum, Guido. 1995c (Apr.). *Python reference manual*. Report CS-R9525.

van Rossum, Guido. 1995d (Apr.). *Python tutorial*. Report CS-R9526.

How does it feel,
To be on your own,
With no direction home,
Like a complete unknown.

B. Dylan (*Like a Rolling Stone*, June 1965)

